

Министерство цифрового развития, связи и массовых коммуникаций РФ
федеральное государственное бюджетное образовательное учреждение высшего
образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Кафедра вычислительных систем
Допустить к защите
Зав. кафедрой к.т.н., доцент
_____ Перышкова Е.Н.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

**Разработка веб-приложения для прослушивания
аудиокниг и подкастов**

Пояснительная записка

Студент Илюхин В.М.

Институт ИВТ Группа ИБ-022

Руководитель Фульман В.О.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Анализ существующих решений и определение требований	6
1.1 Изучение функционала и особенностей популярных сервисов для прослушивания аудиокниг и подкастов.....	6
1.2 Обозначение основных требований к веб-приложению	7
2 Выбор технологий и инструментов для разработки, проектирование архитектуры приложения	8
2.1 Клиент-серверная архитектура	8
2.2 Определение технологий для разработки клиентской части	10
2.2.1 Фреймворк Next JS на базе библиотеки React.....	10
2.2.2 Библиотека Redux Toolkit.....	11
2.2.3 Язык программирования TypeScript.....	11
2.2.4 Фреймворк Tailwind CSS	12
2.3 Выбор базы данных для хранения информации	12
2.4 Определение технологий для разработки серверной части.....	13
2.4.1 Фреймворк Nest JS	13
2.5 Проектирование схемы базы данных	14
2.6 Выбор дополнительного программного обеспечения для реализации приложения	17
2.6.1 Платформа контейнеризации приложений Docker.....	17
2.6.2 Веб-сервер Nginx.....	18
3 Реализация клиентской и серверной частей приложения	19
3.1 Конфигурация окружения	19
3.2 Реализация серверной части приложения	21
3.2.1 Структура каталогов серверной части	21
3.2.2 Настройка подключения к базе данных.....	23
3.2.3 Разработка модуля «Пользователи»	23
3.2.4 Разработка модуля «Аутентификация и авторизация» и «Сессии».....	26
3.2.5 Разработка модулей «Аудиокниги» и «Подкасты»	27

3.2.6	Модуль «Поиск»	28
3.2.7	Функционал «История просмотров»	28
3.2.8	Модуль «Файловая система»	29
3.2.9	Модуль «История прослушивания»	30
3.2.10	Модуль «Отзывы»	30
3.2.11	Модуль «Избранное»	31
3.3	Реализация клиентской части приложения с использованием выбранных технологий	32
3.3.1	Реализация часто используемых компонентов интерфейса	33
3.3.2	Реализация функционала, связанного с пользователем	34
3.3.3	Реализация функционала, связанного с аудиокнигами, подкастами и авторами	36
3.3.4	Реализация аудиоплеера	37
4	Тестирование приложения	38
4.1	Ручное тестирование	38
4.2	Общие требования	38
	ЗАКЛЮЧЕНИЕ	43
	ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	45
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	46
	ПРИЛОЖЕНИЕ А	48
	ПРИЛОЖЕНИЕ Б	51
	ПРИЛОЖЕНИЕ В	127

ВВЕДЕНИЕ

В современном мире, где технологии развиваются с высокой скоростью, люди все чаще предпочитают получать информацию в цифровом формате. Одним из наиболее популярных видов цифрового контента являются аудиокниги и подкасты. Эти форматы позволяют людям получать новые знания и наслаждаться литературой в любое удобное время и в любом месте, не отвлекаясь от других дел.

Разработка веб-приложения для прослушивания аудиокниг и подкастов является актуальным направлением в сфере информационных технологий, которое будет способствовать развитию цифровой экономики и улучшению качества жизни людей. Это подтверждается ростом рынка аудиокниг и подкастов в последние годы. Согласно данным опроса, проведенного российской компанией ООО «Звук», 68% россиян в 2024 году стали чаще слушать аудиокниги [1]. Также по результатам исследования «Российской ассоциации электронных коммуникаций» активная аудитория подкастов составляет около 10 миллионов человек [2], а количество россиян, которые слушали подкасты хотя бы один раз, ежегодно увеличивается на 10%.

Целью работы является разработка веб-приложения для прослушивания аудиокниг и подкастов, которое будет предоставлять пользователям удобный и функциональный инструмент для доступа к аудиоконтенту и поможет удовлетворить их потребности в получении информации в удобном формате. При этом данное приложение будет иметь потенциал для дальнейшего совершенствования.

Для достижения данной цели были обозначены следующие задачи:

- 1) Выполнить анализ существующих аналогичных решений. Изучить их функционал и особенности, на основе полученных данных сформировать требования к веб-приложению

- 2) Выбрать технологии и инструменты для разработки: выбор инструментов и технологий для реализации клиентской и серверной частей, выбрать систему управления базами данных для хранения информации,

спроектировать схему базы данных, выбрать дополнительное программное обеспечение для реализации приложения

3) Реализовать серверную часть веб-приложения с использованием выбранных технологий

4) Реализовать клиентскую часть веб-приложения с использованием выбранных технологий

5) Провести тестирование веб-приложения

1 Анализ существующих решений и определение требований

1.1 Изучение функционала и особенностей популярных сервисов для прослушивания аудиокниг и подкастов

Наиболее популярными среди россиян сервисами для прослушивания аудиокниг и подкастов являются ЛитРес, MyBook, Строки, Яндекс Музыка – Bookmate, VK Музыка. Именно они были выбраны для анализа и выявления особенностей для формирования требований к итоговому приложению [3].

Представим общие характеристики каждого из перечисленных сервисов (таблица 1.1).

Таблица 1.1 – Общие характеристики сервисов для прослушивания аудиокниг и подкастов

Критерии	ЛитРес	MyBook	Строки	Яндекс Музыка - Bookmate	VK Музыка
Аудиокниги	+	+	+	+	+
Подкасты	+	+	+	+	+
Условия доступа к аудиокнигам	Подписка, от 399 рублей в месяц	Подписка, от 229 рублей в месяц	Подписка, от 249 рублей в месяц	Подписка, от 299 рублей в месяц	Подписка, от 169 рублей в месяц
Условия доступа к подкастам	0 рублей	В рамках подписки	В рамках подписки	В рамках подписки	В рамках подписки
Возможность оценивать	+	+	+	+	-
Обязательность регистрации	+	+	+	+	+
Возможность добавлять аудиокниги и подкасты в избранное	+	+	+	+	+
Поиск по сайту	+	+	+	+	+

Также у каждого из сервисов есть особенности.

ЛитРес: доступ к аудиокнигам с ценой до 399 рублей открывается по подписке за 399 рублей в месяц, также пользователям доступна возможность покупать аудиокниги отдельно, не оформляя подписку.

MyBook: из отличительных особенностей сервиса можно выделить синхронизацию прогресса читателей между устройствами, отслеживание персональной статистики пользователя.

Строки: отличительная особенность сервиса – возможность купить электронные версии школьных учебников.

Яндекс Музыка – Bookmate: особенностью сервиса является возможность слушать аудиокниги, подкасты и музыку в рамках одной универсальной подписки и сервиса.

VK Музыка: отличительная особенность – возможность прослушивания аудиокниг, музыки и подкастов в приложении социальной сети ВКонтакте без необходимости устанавливать отдельное приложение.

Все перечисленные сервисы предоставляют доступ к взаимодействию с их библиотекой требуют оформление платной подписки, что не позволяет получить доступ к библиотеке любому пользователю.

1.2 Обозначение основных требований к веб-приложению

Исходя из информации, представленной в предыдущем разделе, необходимо сформировать требования к веб-приложению.

- 1) Веб-приложение должно предоставлять доступ как к аудиокнигам, так и к подкастам
- 2) Для прослушивания содержимого необходимо требовать от пользователя прохождения регистрации
- 3) У пользователей должна быть возможность добавлять аудиокниги и подкасты в избранное
- 4) У пользователей должна быть возможность оценивать и писать отзывы к аудиокнигам и подкастам
- 5) Пользователям должен быть доступен поиск по библиотеке аудиокниг и подкастов

2 Выбор технологий и инструментов для разработки, проектирование архитектуры приложения

2.1 Клиент-серверная архитектура

Клиент-серверная архитектура — это модель организации вычислительных систем, в которой задачи распределены между клиентами и серверами. В такой архитектуре клиент, обычно являющийся пользователем или программой, запрашивает услуги или ресурсы у сервера, который отвечает на запросы, предоставляя необходимые данные или функциональность [4].

Сервер отвечает за хранение данных, обработку запросов и контроль доступа. Клиенты, в свою очередь, отправляют запросы на сервер, который предоставляет необходимые данные или выполняет определенные действия, описанные в алгоритме.

Основные преимущества клиент-серверной архитектуры [5]:

- высокая скорость обработки данных
- низкая нагрузка на сетевую инфраструктуру
- возможность быстрой работы с большим количеством клиентов
- разделение программного кода серверных и клиентских приложений

Данная архитектура также помогает повысить безопасность данных. Все критически важные данные хранятся на сервере, что облегчает контроль доступа к ним и их защиту.

Существует несколько типов клиент-серверной архитектуры: двухуровневая, многоуровневая (N-уровневая) и трехуровневая (частный случай многоуровневой).

В двухуровневой архитектуре между клиентом и базой данных обязательно должен присутствовать серверный узел, с которым этот клиент будет взаимодействовать для выполнения каких-либо операций. Пример такого типа архитектуры представлен на рисунке 2.1.

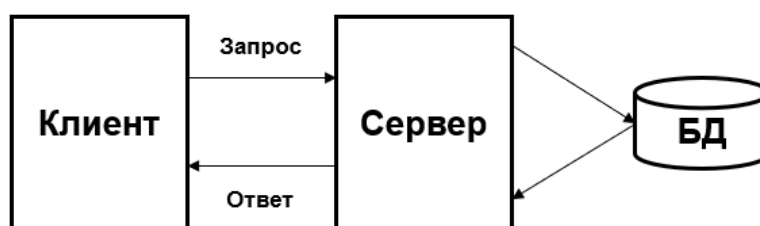


Рисунок 2.1 – пример организации уровней в двухуровневой клиент-серверной архитектуре

В трёхуровневой архитектуре сетевое приложение разделено на несколько частей, каждая из которых может выполняться на отдельном компьютере. Выделенные части приложения взаимодействуют друг с другом, обмениваясь сообщениями в заранее согласованном формате. Пример такого типа архитектуры представлен на рисунке 2.2.

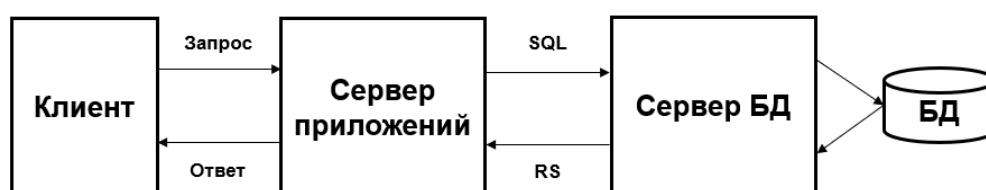


Рисунок 2.2 – пример организации уровней в трёхуровневой клиент-серверной архитектуре

Многоуровневая архитектура представляет собой расширенный вариант трёхуровневой архитектуры. На каждом уровне может находиться несколько вычислительных машин. Сервера в такой архитектуре коммуницируют между собой по определенным протоколам и интерфейсам. Пример частного случая такого типа архитектуры представлен на рисунке 2.3.

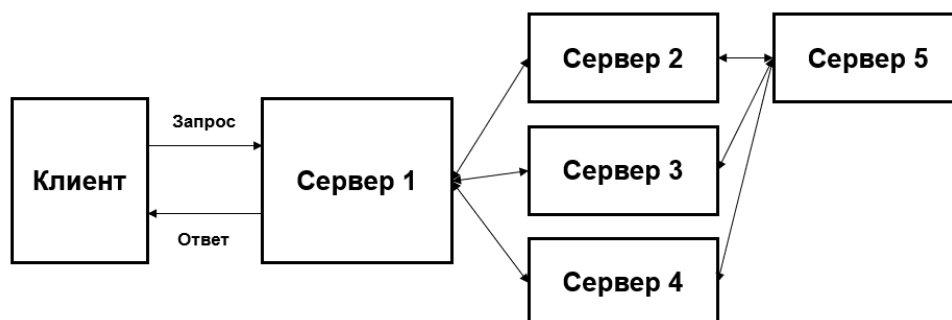


Рисунок 2.3 – организация уровней в двухуровневой клиент-серверной архитектуре

2.2 Определение технологий для разработки клиентской части

Для разработки клиентской части используются следующие технологии.

- HTML – язык гипертекстовой разметки документов для просмотра веб-страниц в браузере
- CSS – формальный язык декодирования и описания внешнего вида документа, написанного с использованием языка разметки
- JavaScript – интерпретируемый язык программирования, используемый для написания клиентской и серверной частей веб-приложений.

Эти технологии позволяют создавать веб-страницы с простой структурой и дизайном, а также обеспечивать базовую интерактивность и динамичность. Однако для разработки более сложных и масштабируемых приложений на базе данных технологий было реализовано множество библиотек и фреймворков, позволяющих эффективно взаимодействовать с браузером и сервером в сложных системах.

Данные библиотеки и фреймворки предоставляют готовые решения и инструменты для разработки клиентской части веб-приложений. Благодаря этому можно в кратчайшие сроки реализовывать базовую функциональность клиентской части. На текущий момент одной из наиболее популярных и эффективных библиотек для разработки клиентской части веб-приложений является React. Данная библиотека используется различными крупными предприятиями (VK, Яндекс, ПочтаБанк и другие), занимающимися развитием собственных высоконагруженных систем.

2.2.1 Фреймворк Next JS на базе библиотеки React

React является библиотекой JavaScript с открытым исходным кодом. Данная библиотека позволяет создавать быстрые и эффективные пользовательские интерфейсы с помощью компонентов, которые можно повторно использовать. Еще одним преимуществом React является его виртуальный DOM, который позволяет библиотеке эффективно обновлять только те части пользовательского интерфейса, которые были изменены, вместо перерисовки всего интерфейса. Это значительно улучшает

производительность приложения, особенно при работе с большими объемами данных и сложными интерфейсами [6].

Библиотека React лежит в основе фреймворка Next.js. Фреймворк предназначен для решения проблемы React, связанной с отрисовкой приложения на стороне сервера – SSR, что позволяет улучшить поисковую оптимизацию и производительность, позволяя пользователям увидеть содержимое страницы быстрее. Также фреймворк Next.js дополняет React удобными решениями по навигации между страницами с помощью файлового подхода [7].

2.2.2 Библиотека Redux Toolkit

Поскольку приложение состоит из компонентов, необходимо решить проблему управления глобальным состоянием и доступа к нему. Для централизованного управления состоянием было принято решение использовать библиотеку Redux Toolkit. Это решение позволяет хранить состояние в одном месте, а не в отдельных компонентах, что упрощает отслеживание изменений и управление состоянием из любого компонента приложения, независимо от его уровня в иерархии. Основные концепции данной библиотеки [8]:

- Единый источник состояния. Все данные о состоянии приложения хранятся в одном месте, без копий.
- Доступ к объекту состояния в режиме «только чтение». Объект глобального состояния заблокирован для записи. Компоненты приложения могут читать из него, но не записывать, что предотвращает непредсказуемые изменения.
- Компоненты могут изменять состояние только через специальные методы, именуемые «редукторами».

2.2.3 Язык программирования TypeScript

Для удобства разработки и предотвращения возможных ошибок, связанных с типами данных, было принято использовать TypeScript - язык программирования, представленный компанией Microsoft в 2012 году и

позиционируемый как средство разработки веб-приложений, расширяющее возможности JavaScript.

TypeScript отличается от JavaScript возможностью явного статического назначения типов, поддержкой использования полноценных классов (как в традиционных объектно-ориентированных языках), а также поддержкой подключения модулей, что призвано повысить скорость разработки, облегчить читаемость, рефакторинг и повторное использование кода, помочь осуществлять поиск ошибок на этапе разработки и компиляции [9].

2.2.4 Фреймворк Tailwind CSS

Для стилизации компонентов и создания пользовательского интерфейса был выбран фреймворк Tailwind CSS. Его использование обеспечивает быструю и эффективную разработку интерфейса благодаря большому набору предопределённых стилей, которые можно легко настраивать и масштабировать [10]. Данный фреймворк использует утилитарные классы, позволяющие применять стили к элементам непосредственно в HTML-коде. Также Tailwind CSS является гибким в плане конфигурации фреймворком, что позволяет переопределять различные заданные по умолчанию параметры под нужды проекта.

2.3 Выбор базы данных для хранения информации

Для веб-приложения для прослушивания аудиокниг и подкастов требуется масштабируемая и производительная системы управления базами данных. В связи с этим было принято решение использовать PostgreSQL в качестве основной СУБД.

Ключевые факторы для выбора PostgreSQL в качестве СУБД [11]:

- PostgreSQL предоставляет большой набор защищенных протоколов аутентификации, таких как GSSAPI, SSPI, LDAP, SCRAM-SHA-256 и другие. В ней реализована ролевая модель, которая позволяет разграничивать права на уровне всех сущностей, к которым относятся базы данных, схемы, таблицы и т.д. вплоть до столбца или строки.

- PostgreSQL имеет широкий набор типов данных. Среди них есть стандартные типы, которые можно найти в большинстве систем управления базами данных и языках программирования: integer (int), числовой тип (numeric), текст (text), логический тип (bool), временная метка (timestamp). Также есть более специализированные типы данных, например: списки, UUID, JSON/JSONB, геометрические типы (point, line, polygon и другие).
- PostgreSQL соответствует принципам ACID. Их соблюдение гарантирует, что система, работающая в режиме реального времени, — отказоустойчивая, в ней отсутствуют риски потери данных, сами данные согласованы между собой, параллельные манипуляции разных пользователей не оказывают влияния друг на друга. PostgreSQL поддерживает все возможные уровни изоляции транзакций, определенные в стандарте SQL, что позволяет точно контролировать параллельное выполнение операций.
- PostgreSQL распространяется под лицензией открытого исходного кода, что обеспечивает свободный доступ к её функциональности без ограничений лицензирования.

2.4 Определение технологий для разработки серверной части

В условиях ограниченного времени на реализацию приложения было принято решение реализовывать его серверную часть на том же языке программирования, что и клиентскую — JavaScript, который возможно использовать не только в качестве языка программирования для клиентской части веб-приложения благодаря Node JS — программной платформе, которая транслирует JavaScript в машинный код, исполняемый на стороне сервера.

2.4.1 Фреймворк Nest JS

Также для ускорения разработки было принято решение использовать фреймворк Nest JS. Данный фреймворк предоставляет готовую архитектуру серверных приложений, которая позволяет создавать легко тестируемые, масштабируемые и простые в обслуживании приложения.

Ключевые факторы для выбора Nest JS в качестве фреймворка [12]:

- Nest.js полностью построен на TypeScript, что позволяет писать более надежный и понятный код за счёт статической типизации. TypeScript также обеспечивает автодополнение и статический анализ, что упрощает разработку и рефакторинг кода.
- Nest.js построен на модульной архитектуре, позволяющей организовать приложение на небольшие, переиспользуемые компоненты. Это способствует улучшению поддерживаемости и масштабируемости кода.
- Nest.js способствует применению широко распространенных паттернов разработки, таких как внедрение зависимостей (Dependency Injection - DI), аспектно-ориентированное программирование (AOP) и многие другие. Это способствует повышению читаемости, тестируемости и общей структурированности кода. Внедрение зависимостей позволяет легко заменять и тестировать компоненты, а аспектно-ориентированное программирование помогает внедрять кросс-функциональные аспекты, такие как логирование или управление транзакциями, без затрагивания основной логики.

Для взаимодействия с базой данных используется библиотека Sequelize — это ORM для работы с такими СУБД, как Postgres, MySQL, MariaDB, SQLite и MSSQL. Данная библиотека предоставляет упрощенное взаимодействие с базой данных, поддерживает миграции и позволяет реализовывать валидацию и установление связей между сущностями.

2.5 Проектирование схемы базы данных

Исходя из результатов анализа существующих решений, можно обозначить ключевые сущности в каждом из них: пользователи, книги, подкасты, авторы, жанры. Каждая сущность имеет первичный ключ и может связана с другими сущностями. Получившаяся схема базы данных приведена на рисунках 2.4, 2.5 и 2.6.



Рисунок 2.5 – схема базы данных (таблицы для подкастов)

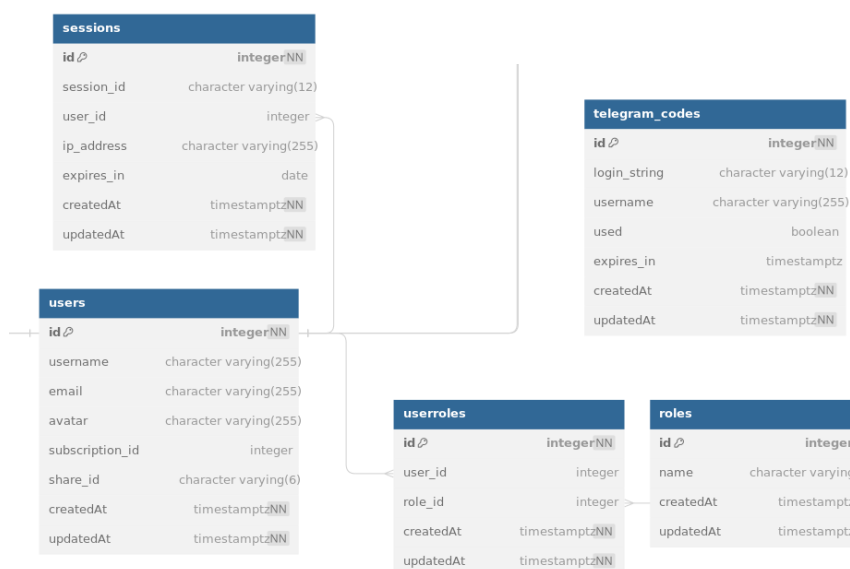


Рисунок 2.6 – схема базы данных (таблицы для пользователя)

2.6 Выбор дополнительного программного обеспечения для реализации приложения

Для обеспечения надежной и масштабируемой инфраструктуры веб-приложения было принято решение использовать Docker для упаковки приложений в так называемые контейнеры и веб-сервер Nginx для обработки HTTP-запросов и маршрутизации трафика.

2.6.1 Платформа контейнеризации приложений Docker

Docker предоставляет унифицированный способ упаковки, доставки и запуска приложений в виде контейнеров. Это упрощает разработку и позволяет разрабатывать и разворачивать приложения в изолированном окружении.

Контейнеризация (контейнерная виртуализация) — это метод виртуализации, при котором единое пользовательское пространство в ядре операционной системы разделяется на несколько независимых логических разделов. Один экземпляр такого раздела называется контейнером или зоной. В каждом контейнере можно запускать одно приложение, изолированное от остальной системы. Пример контейнерной виртуализации представлен на рисунке 2.7.

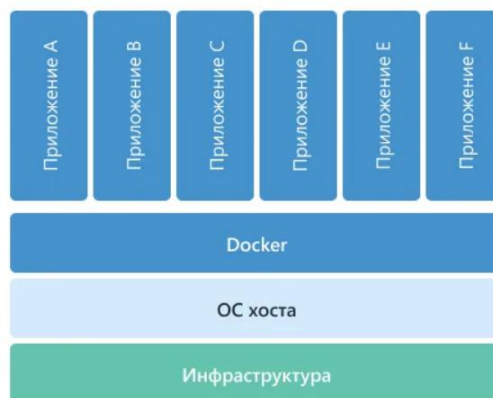


Рисунок 2.7 – пример контейнерной виртуализации

В разрабатываемом веб-приложении Docker используется для развертывания и управления компонентами приложения, включая серверную часть, базу данных, клиентскую часть. Каждый компонент приложения упаковывается в собственный контейнер, что упрощает управление и развертывание приложения в различных окружениях [13].

Поскольку приложение будет представлять несколько сервисов, было принято решение использовать Docker Compose - средство, предоставляющее способ организации управления несколькими контейнерами для совместной работы.

2.6.2 Веб-сервер Nginx

Nginx — это веб-сервер с открытым исходным кодом, созданный работать под высокой нагрузкой, чаще всего используемый для отдачи статического контента, например, html страниц, медиафайлов, документов, архивов, картинок и т.д.

Основная функциональность HTTP-сервера [14]:

- Обслуживание статических запросов, индексных файлов, автоматическое создание списка файлов, кэш дескрипторов открытых файлов;
- Поддержка SSL и расширения TLS SNI;
- Поддержка HTTP/2 и HTTP/3

3 Реализация клиентской и серверной частей приложения

3.1 Конфигурация окружения

Разработанная структура проекта представлена на рисунке 3.1.

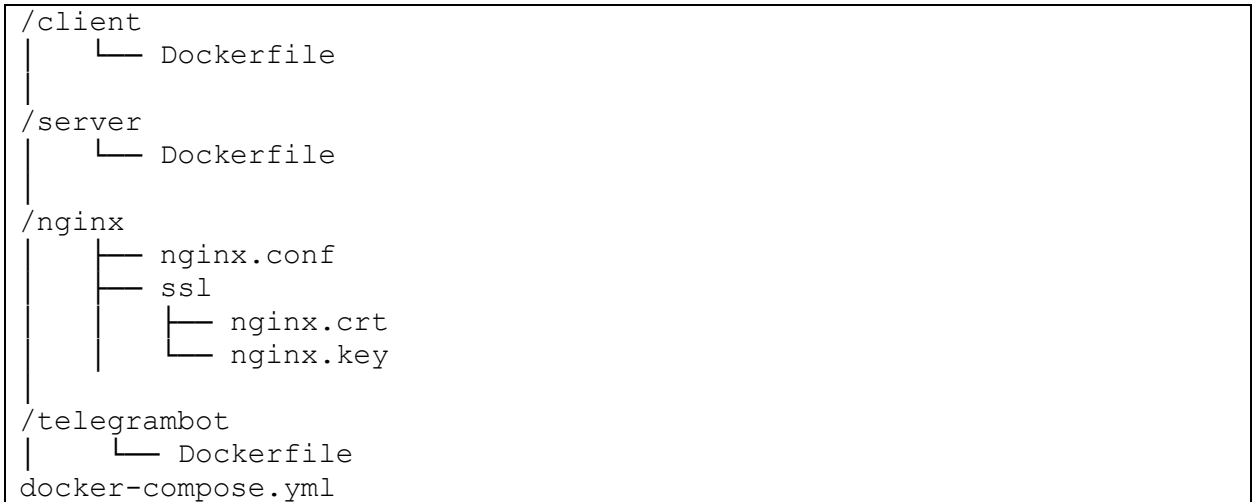


Рисунок 3.1 – структура проекта

Dockerfile - текстовый файл, который содержит инструкции для сборки образа Docker. Этот файл используется для определения того, как будет построен образ Docker, включая установку зависимостей, настройку среды выполнения, копирование файлов.

Поскольку для клиентской и серверной частей используется Node JS для сборки и запуска кода на JavaScript, было принято использовать официальный docker-образ Node JS (листинг 3.1 и листинг 3.2).

Листинг 3.1 – Dockerfile для client

```
FROM node:21
WORKDIR /var/www
COPY . /var/www/
RUN npm install
EXPOSE 3000
CMD [ "npm", "run", "dev" ]
```

Листинг 3.2 - Dockerfile для server

```
FROM node:21
WORKDIR /var/www
COPY . .
RUN npm install
EXPOSE 5000
CMD [ "npm", "run", "start:dev" ]
```

Telegram-бот будет написан на языке программирования Python, поэтому для него используем официальный docker-образ Python (листинг 3.3).

Листинг 3.3 - Dockerfile для telegrambot

```
FROM python:3.12.3-alpine3.19

WORKDIR /var/www

COPY requirements.txt /var/www/
RUN pip3 install --no-cache-dir -r /var/www/requirements.txt

COPY . /var/www/

CMD [ "python", "./main.py" ]
```

Также на проекте используется Nginx, содержимое файла конфигурации представлено на листинге 3.4.

Листинг 3.4 - содержимое файла nginx/nginx.conf

```
http {
    server {
        listen 80;
        server_name books.com www.books.com;
        return 301 https://books.com$request_uri;
    }

    server {

        listen 80;

        server_name books.com;

        location / {
            proxy_pass http://frontend:3000;
        }

        location /api {
            proxy_pass http://backend:5000;
        }

        listen 443 ssl;
        ssl_certificate /etc/nginx/ssl/nginx.crt;
        ssl_certificate_key /etc/nginx/ssl/nginx.key;
    }
}
```

Также необходимо создать файл конфигурации Docker Compose, который соберет описанные сервисы воедино.

Также для разработки используется Docker Compose. Его конфигурация задается в специальном YAML-файле – `docker-compose.yml` (листинге А.1). В данном файле определены настройки нескольких сервисов: `backend`, `frontend`, `nginx`, `database`, `adminer`, `telegrambot`. Все эти сервисы находятся в одной подсети с IP адресами в диапазоне `172.20.0.0/16`. Сетевой шлюз - `172.20.0.1`. Итоговый вариант конфигурации проиллюстрирован на рисунке 3.2.

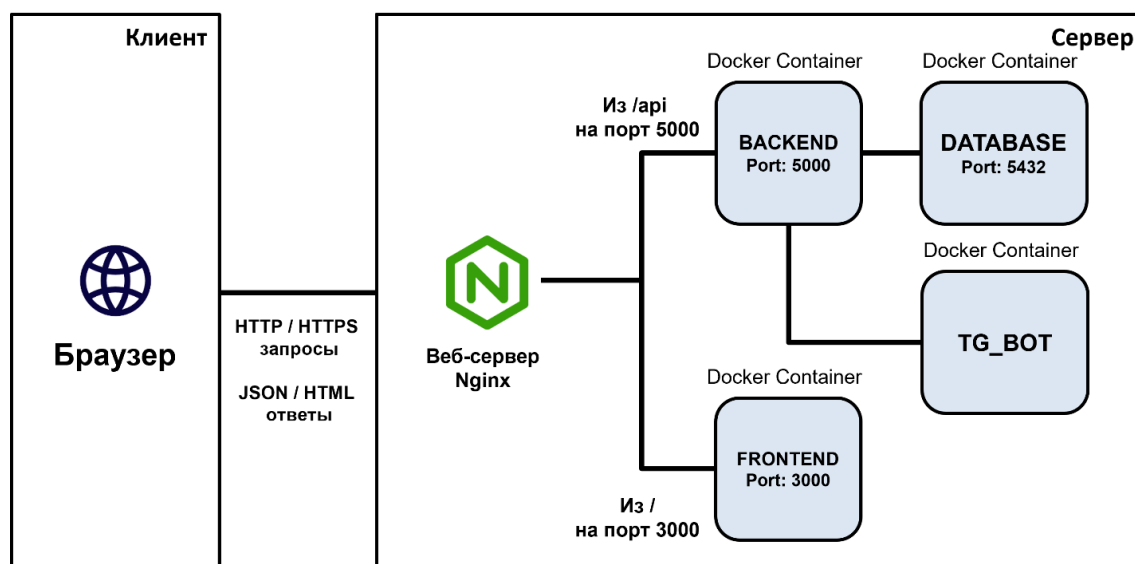


Рисунок 3.2 – итоговая конфигурация Docker Compose

3.2 Реализация серверной части приложения

3.2.1 Структура каталогов серверной части

Для реализации серверной части веб-приложения был выбран фреймворк Nest JS. Структура каталогов серверной части представлена на рисунке 3.3.



Рисунок 3.3 – структура папок для серверной части

Каталог `src` используется для организации кода и файлов, относящихся к приложению, содержит следующие каталоги:

- Каталог `auth` содержит код модуля, отвечающий за аутентификацию и авторизацию пользователей
- Каталог `authors` содержит код модуля, связанного с авторами книг или подкастов
- Каталог `books` содержит код модуля, связанного с книгами
- Каталог `booksparts` содержит код модуля, отвечающего за главы книг
- Каталог `favorites` содержит код модуля, отвечающего за избранные элементы пользователя
- Каталог `filesystem` содержит код модуля, отвечающего за взаимодействие приложения с файловой системой
- Каталог `genres` содержит код модуля, отвечающего за жанры аудиокниг и подкастов
- Каталог `helpers` содержит вспомогательные функции и утилиты, которые могут использоваться в разных частях приложения
- Каталог `history` содержит код модуля, отвечающего за историю воспроизведения пользователей
- Каталог `podcasts` содержит код модуля, связанного с подкастами
- Каталог `podcastsepisodes` содержит код модуля, связанного с эпизодами подкастов
- Каталог `reviews` содержит код модуля, связанного с отзывами об аудиокнигах и подкастах
- Каталог `roles` содержит код модуля, связанного с ролями пользователей
- Каталог `search` содержит код модуля, отвечающего за функционал поиска

- Каталог sessions содержит код модуля, отвечающего за сессии пользователей
- Каталог users содержит код модуля, отвечающего за пользователей

3.2.2 Настройка подключения к базе данных

Для взаимодействия сервера веб-приложения с базой данных используется ORM-библиотека Sequelize. Для подключения сервера к базе данных необходимо в качестве аргумента метода SequelizeModule.forRoot передать объект, содержащий необходимую для этого информацию, а именно: диалект языка SQL, адрес хоста, имя пользователя СУБД, пароль пользователя СУБД, название базы данных и другое. Объект с заданными параметрами представлен на листинге 3.5.

Листинг 3.5 – подключение к базе данных

```
SequelizeModule.forRoot({  
  dialect: 'postgres',  
  host: process.env.DB_HOST,  
  username: process.env.DB_USER,  
  password: process.env.DB_USER_PASSWORD,  
  database: process.env.DB,  
  autoLoadModels: true,  
}),
```

Данный объект передает методу информацию об используемом диалекте SQL, информацию для подключения к базе данных и флаг, который информирует о необходимости создания таблицы в базе данных для импортируемых моделей.

Чувствительная информация задается в переменных окружения. Это отвечает требованиям безопасности, позволяет обеспечить переносимость кода и легко изменять данные без необходимости пересборки приложения.

3.2.3 Разработка модуля «Пользователи»

Разработка серверной части приложения началась с основной сущности – пользователей. Необходимо описать сущность Пользователь, создав модель с помощью библиотеки Sequelize и указав атрибуты таблицы (листинг 3.6). Модель пользователей связана со следующими моделями: сессии, просмотры страниц книг и подкастов.

Листинг 3.6 – описание атрибутов таблицы, хранящей информацию о пользователях

```
@Table({ tableName: 'users' })
export class Users extends Model<Users> {
  @Column({type: DataType.STRING, unique: true})
  username: string;

  @Column({
    type: DataType.STRING,
    unique: true,
  })
  avatar: string;
}
// продолжение описания
```

В объект, передаваемый в качестве аргумента декоратора `Module`, по ключу `imports` необходимо в качестве значения передать массив, в который нужно добавить вызов метода `SequelizeModule.forFeature` для выполнения создания и обновления схем таблиц для обозначенных моделей (листинг 3.7).

Листинг 3.7 – передача информации о моделях, описанных в модуле

```
@Module({
  controllers: [UsersController],
  providers: [UsersService],
  imports: [SequelizeModule.forFeature([Users])],
  exports: [UsersService]
})
```

Также для пользователей были реализованы методы создания новой записи в БД, получения информации. Для этого в Nest JS используются контроллеры (листинг 3.8), отвечающие за обработку входящих запросов и возврат ответов. Они обрабатывают входящие HTTP-запросы и делегируют более сложные операции (например, взаимодействие с базой данных или вычисления) сервисам. Контроллеры определяют маршруты, которые сопоставляются с определенными методами контроллера. Маршруты определяются с помощью декораторов `Get()`, `Post()`, `Put()`, `Delete()` и другими. В качестве аргумента декоратора передается относительный путь, по которому будет выполняться логика контроллера, которую, в свою очередь, рекомендуется выноситься в сервисы. Для пользователей определена конечная точка `«/user/:username»`, которая возвращает информацию о пользователе по его имени. Пример обработки HTTP-запросов в NestJS представлен на рисунке 3.4.

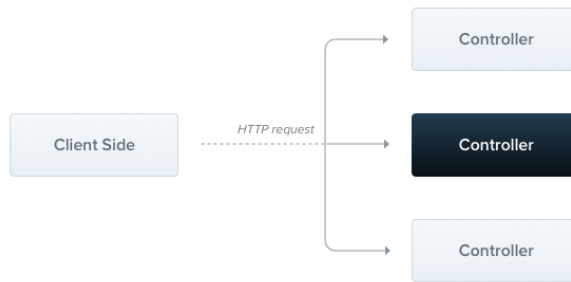


Рисунок 3.4 – обработка HTTP-запросов в Nest JS

Сервисы в NestJS отвечают за бизнес-логику приложения. Они содержат методы, которые выполняют операции, такие как выборка данных из базы данных, выполнение вычислений и другие операции. Сервисы предназначены для выполнения сложных операций и получения записей из базы данных и используются как контроллерами, так и частями других модулей приложения. Для сервиса пользователей было реализовано несколько методов, выполняющих основные операции управления записями в таблице с пользователями.

Листинг 3.8 – пример кода контроллера модуля, отвечающего за пользователей

```

export class UsersController {
  constructor(private userService: UserService) {}

  @Get('/user/:username')
  @HttpCode(200)
  async getUserInfo(@Param('username') username: string) {
    const result = await this.userService.getByUsername(username);
    if (result.status !== 200) {
      throw new HttpException('', result.status);
    }

    return { id: result.user.id, username: result.user.username,
    avatar: result.user.avatar };
  }
}
  
```

Для унификации возвращаемых значений всех сервисов всех модулей был создан интерфейс `IServiceAnswer` (листинг Б.38), который содержит основные свойства и который будут дополнять другие интерфейсы:

- Свойство `status`, представляющее собой HTTP статус-код

– Свойство `error`, представляющее собой строку, содержащую информацию об ошибке

3.2.4 Разработка модуля «Аутентификация и авторизация» и «Сессии»

Далее необходимо разработать модуль для аутентификации и авторизации пользователей. Регистрация и аутентификация происходит через бот в приложении Telegram, это позволяет пользователям быстро создать аккаунт на сайте или войти в него, а у веб-приложения, в свою очередь, есть гарантия, что сайтом собирается воспользоваться реальный человек, поскольку каждый аккаунт в Telegram закреплён за реальным номером телефона. Telegram бот был вынесен в отдельный сервис и реализован на языке Python. Алгоритм регистрации и аутентификации приведен в виде схемы на рисунке 3.5.



Рисунок 3.5 – алгоритм регистрации и аутентификации

В контроллере модуля описано несколько конечных точек:

– «`/telegram/link`» - конечная точка (метод `POST`), по которой Telegram бот запрашивает пользовательскую ссылку для входа. В качестве тела запроса ожидается имя пользователя в Telegram и API-ключ, который подтверждает, что Telegram бот имеет право на использование этой конечной точки

– «`/telegram/link/:link`» - конечная точка (метод `DELETE`), по которой клиентская часть веб-приложения может уведомить сервер о том, что пользовательская ссылка использована и более не может считаться валидной

- «/logout» - конечная точка (метод DELETE), по которой клиентская часть веб-приложения запрашивает уничтожение сессии
- «/amIAuthorized» - конечная точка (метод GET), по которой клиентская часть веб-приложения может проверить активность сохраненной сессии

В модуле «Сессии» описана модель базы данных. Данная модель связана с моделью «Пользователь». Сервисы модуля «Сессии» отвечают за создание и получение сессии по ее идентификатору. Методы этого сервиса используются в модуле «Аутентификация и авторизация» в случае успешного прохождения пользователем аутентификации.

3.2.5 Разработка модулей «Аудиокниги» и «Подкасты»

Далее необходимо разработать модуль для аудиокниг и подкастов, их глав и эпизодов, жанров. Модуль должен обрабатывать входящие запросы на получение списка аудиокниг и подкастов, получение информации об отдельном элементе из базы данных. Было создано 7 отдельных таблиц в базе данных: books, books_parts, podcats, podcasts_parts, genres, books_genres, podcasts_genres. Между books и books_parts установлена связь один ко многим, для podcasts и podcasts_episodes аналогично. Между books и genres установлена связь многие ко многим, для podcasts и genres аналогично. Связи между сущностями позволяют за один запрос с клиента получить максимально подробную информацию об элементе, включая автора и жанры, к которым элемент относится. Для этого в запрос в БД через ORM Sequelize необходимо включить пару с ключом «include» и массивом, содержащим названия моделей, в качестве значения.

В контроллерах данных модулей были реализованы следующие конечные точки:

- «/search» - конечная точка (метод GET), по которой клиентская часть веб-приложения запрашивает элементы, содержащие в названии или описании определенную подстроку. Для получения этих элементов данный контроллер использует методы сервиса, реализованного в модуле «Поиск»

– «/list» - конечная точка (метод GET), по которой клиентская часть веб-приложения получает список аудиокниг и подкастов, отсортированных по атрибуту sort. Данный метод также поддерживает постраничную навигацию, на одной странице размещается 10 элементов

– «/one/:code» - конечная точка (метод GET), по которой клиентская часть веб-приложения получает всю информацию о конкретном элементе по его уникальному коду

Данные модули используют функционал других модулей, например, «История просмотров» и «Сессии».

3.2.6 Модуль «Поиск»

Для поиска по каталогу аудиокниг и подкастов был создан соответствующий модуль – search. В нем реализованы сервисы, отвечающие за обработку запросов элементов по определенной подстроке. Логика модуля построена на поиске заданной подстроки в названиях и описаниях аудиокниг и подкастов. В модуле поиска используются методы из сервисов модулей «Аудиокниги» и «Подкасты», которые, в свою очередь, выполняют поиск по заданной подстроке в своих таблицах. Далее сервис возвращает 5 наиболее релевантных результатов в каждой из категорий, либо ничего не возвращает в случае отсутствия результатов по заданному запросу. Схема взаимодействия клиента с модулем поиска представлена на рисунке 3.6.

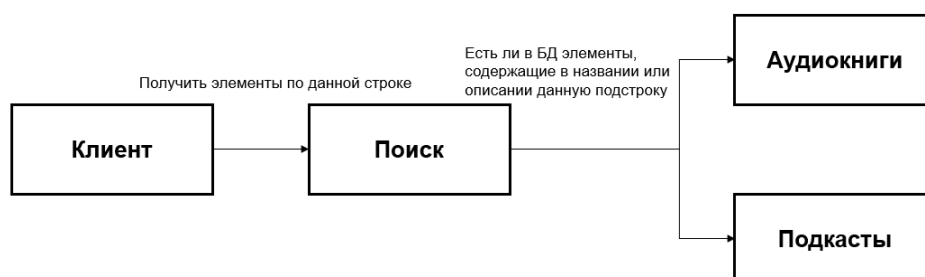


Рисунок 3.6 – схема взаимодействия клиента с модулем «Поиск» на сервере

3.2.7 Функционал «История просмотров»

Также у книг и подкастов есть атрибут «количество просмотров». Когда пользователь с активной сессией посещает на клиенте детальную страницу

книги или подкаста, его просмотр сохраняется в базу данных и считается актуальным в течение 24 часов. То есть, если пользователь в течение суток посетит страницу элемента несколько раз в системе будет зарегистрирован только один просмотр, что позволит получить на выходе более объективную статистику по элементу. Для хранения информации о просмотрах были созданы таблицы `books_views` и `podcasts_views`, связанные с пользователями и с книгами и подкастами.

3.2.8 Модуль «Файловая система»

Для взаимодействия с файлами и потоковой передачи файлов глав аудиокниг и эпизодов подкастов был создан модуль `filesystem`. В нем реализован контроллер, который позволяет клиенту получить аудиофрагмент в заданном диапазоне по уникальному строковому коду, обозначенному в таблицах `books_parts` и `podcasts_episodes`. В методе контроллера `streamBook` происходит поиск файла на основе типа (`book` или `podcast`) и уникального кода, который передается в качестве GET-параметра в URL. Затем происходит попытка получения информации о файле, чтобы определить, существует ли данный файл, и, если существует, получить его мета-данные. Если файл существует, метод проверяет, есть ли заголовок `range` в запросе, чтобы определить, требует ли клиент начать потоковую передачу аудиофайла. Если заголовок `range` присутствует, метод создает поток для чтения файла с учетом диапазона байтов, заданного в заголовке. Схема взаимодействия клиента и сервера для получения части аудиофрагмента представлена на рисунке 3.7.

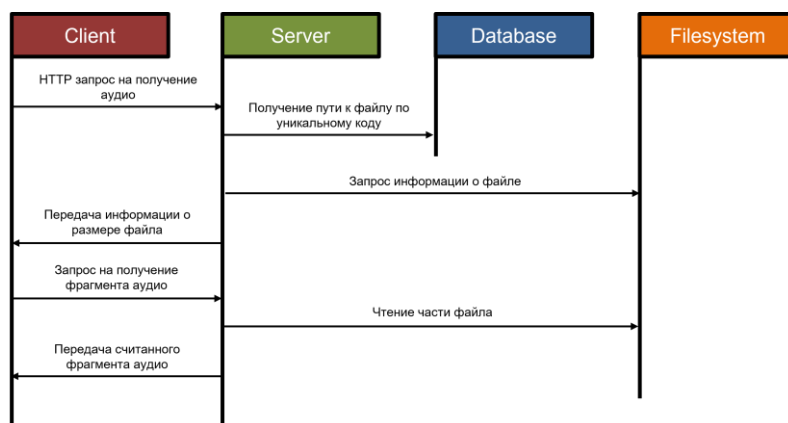


Рисунок 3.7 – схема взаимодействия клиента и сервера для получения части аудиофрагмента

3.2.9 Модуль «История прослушивания»

Для хранения истории прослушивания был создан отдельный модуль history, ответственный за нее. Информация хранится в двух таблицах базы данных: podcasts_history и books_history. Каждый пользователь с активной сессией может получить список аудиокниг и подкастов, которые были прослушаны. Элемент записывается в историю прослушивания сразу после начала воспроизведения какой-либо из частей аудиокниги или подкаста. Информацию о начале воспроизведения в данный модуль передает модуль «Файловая система». Сервисы модуля используют логику из модулей «Аудиокниги», «Подкасты» и «Сессии».

Для клиентской части веб-приложения было реализовано несколько конечных точек:

- «/books» - конечная точка (метод GET), позволяющая клиентской части веб-приложения получить историю прослушиваний аудиокниг конкретного пользователя
- «/podcasts» - конечная точка (метод GET), позволяющая клиентской части веб-приложения получить историю прослушиваний подкастов конкретного пользователя

3.2.10 Модуль «Отзывы»

Для оценивания и отзывов был создан отдельный модуль reviews. Информация об оценках и отзывах хранится в таблицах book_reviews и podcasts_reviews. Оценить и написать отзыв об аудиокниге или подкасте может только пользователь с активной сессией, имеющий рассматриваемый элемент в истории прослушиваний. Оценку и отзыв к одному и тому же элементу пользователь может оставить только один раз. Сервис данного модуля используют логику, реализованную в модулях «Сессии» и «История прослушивания». Для клиента были реализованы следующие конечные точки:

- «/:type/isavailable/:entityId» - конечная точка (метод GET), позволяющая клиентской части веб-приложения проверить, разрешается ли пользователю написать отзыв к рассматриваемому элементу

– «/:type/add» - конечная точка (метод POST), позволяющая клиентской части веб-приложения добавить отзыв к рассматриваемому элементу

– «/:type/:entityId» - конечная точка (метод GET), позволяющая клиентской части веб-приложения получить список отзывов к рассматриваемому элементу

Пример взаимодействия клиента и модуля «Отзывы» на сервере представлен на рисунке 3.8.

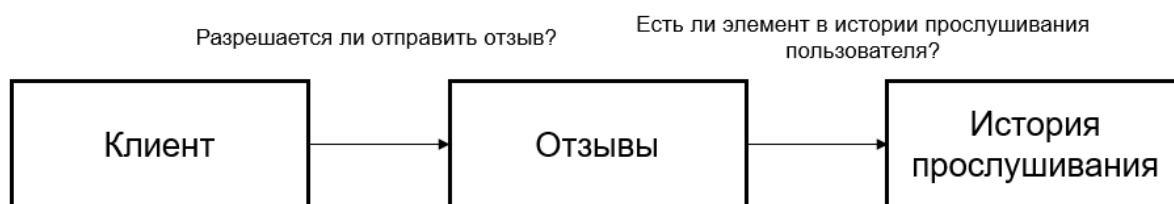


Рисунок 3.8 – пример взаимодействия клиента и модуля «Отзывы» на сервере

3.2.11 Модуль «Избранное»

Далее необходимо реализовать функционал для сохранения книг и подкастов в избранное. Для этого был создан соответствующий модуль – favorites. Добавлять аудиокниги и подкасты в избранное может только пользователь с активной сессией. Каждый пользователь может получить список избранных аудиокниг и подкастов любого другого пользователя. Сервис данного модуля использует логику, реализованную в модулях «Аудиокниги», «Подкасты» и «Сессии».

Для клиента были реализованы следующие конечные точки:

– «/:type/:id» - конечная точка (метод POST), позволяющая клиентской части веб-приложения добавить определенную книгу в избранное определенного пользователя

– «/:type/:id» - конечная точка (метод DELETE), позволяющая клиентской части веб-приложения удалить определенную книгу из избранного определенного пользователя

- «/user/:type/:id» - конечная точка (метод GET), позволяющая клиентской части веб-приложения получить список избранных элементов другого пользователя

3.3 Реализация клиентской части приложения с использованием выбранных технологий

Для реализации клиентской части веб-приложения был выбран фреймворк Next.js на базе React. Структура папок для клиентской части представлена на рисунке 3.9.

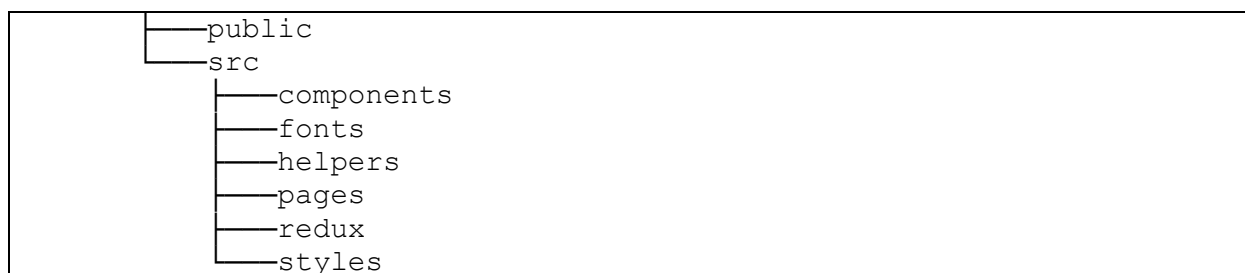


Рисунок 3.9 – структура папок для клиентской части

Папка `public` используется для хранения статических файлов, которые не требуют обработки или преобразования через Next.js. Эти файлы включают в себя изображения, шрифты, файлы JavaScript и CSS, которые должны быть доступными для клиента без каких-либо изменений.

Каталоги внутри `src` обычно используются для организации кода и файлов, относящихся к приложению:

- Каталог `components` содержит все компоненты, которые используются в приложении. Компоненты могут быть функциональными или классовыми, и они могут быть использованы в разных частях приложения.
- Каталог `fonts` содержит шрифты, используемые в приложении
- Каталог `helpers` содержит вспомогательные функции и модули, которые могут быть использованы в различных частях приложения.
- Каталог `pages` содержит все страницы приложения. Каждая страница представлена как отдельный `tsx`-файл, который Next.js автоматически преобразует в маршрут на основе имени директории и файла. Например, файл `about.js` будет доступен по маршруту `/about`.

– Каталог `redux` содержит все файлы, связанные с `Redux`, такие как действия (`actions`), редукторы (`reducers`) и хранилище (`store`). `Redux` используется для управления состоянием приложения. Доступ к этому состоянию можно получить из любой части приложения

– Каталог `styles` содержит CSS-стили, используемые в приложении

3.3.1 Реализация часто используемых компонентов интерфейса

Разработка клиентской части приложения началась с реализации основных компонентов пользовательского интерфейса: шапка клиентской части веб-приложения (`header`) с меню, компонент для модального окна, компонент кнопки и компонент подвала (`footer`). Подразумевается, что данные компоненты будут использоваться на большинстве страниц клиентской части веб-приложения, соответственно, необходимо реализовать возможность настройки данных компонентов под конкретную страницу. Для этого каждый компонент в качестве аргумента должен принимать определенные свойства и использовать их внутри себя. Например, компонент кнопки принимает в качестве аргументов дополнительные стили, функцию-обработчик события `onClick`, тип кнопки, состояние кнопки (листинг 3.9 и рисунок 3.10).

Листинг 3.9 – компонент «Кнопка»

```
const Button: React.FC<IButtonAttributes> = ({children, styles,
onClick, type = "primary", disabled = false, width = "full",}:
IButtonAttributes) => {
  const disabledStyle = `opacity-50`;
  const classesSet = {
    primary: `bg-main w-${width} rounded-2xl py-3 text-white
transition enabled: hover:-translate-y-px
enabled: hover:brightness-95 enabled: active: translate-y-px
disabled: cursor-not-allowed ${
      disabled ? disabledStyle : ""
    } ${styles ?? ""}` ,
    secondary: `bg-secondary w-${width} rounded-2xl py-3 text-
black transition enabled: hover:-translate-y-px
enabled: hover:brightness-95 enabled: active: translate-y-px
disabled: cursor-not-allowed ${
      disabled ? disabledStyle : ""
    } ${styles ?? ""}` ,
    text: `bg-white rounded-2xl w-${width} transition-all
enabled: hover:brightness-95 disabled: cursor-not-allowed
${styles}`
```

```

    };
    return (
      <button onClick={onClick} className={classesSet[type]}
disabled={disabled}>
        {children}
      </button>
    );
  };
};

```

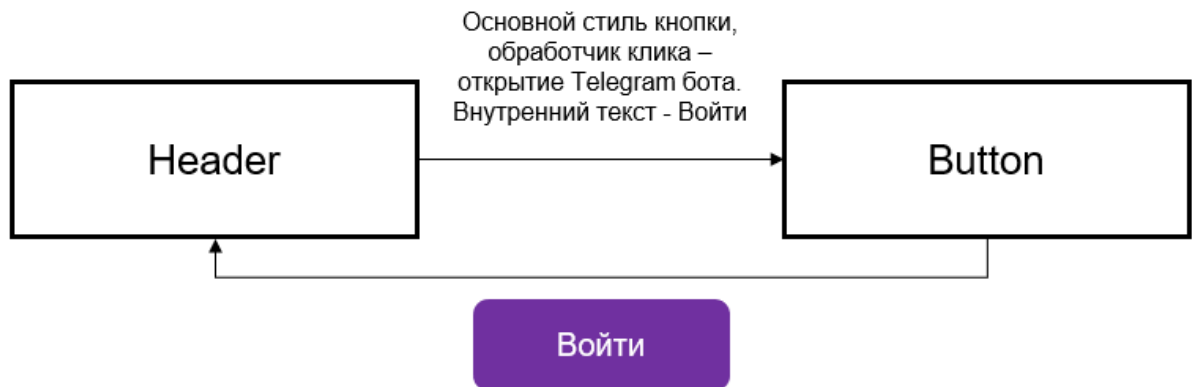


Рисунок 3.10 – пример подключения компонента интерфейса внутри другого компонента

Для того, чтобы компонент шапки и подвала было возможно было подключить на большинстве страниц клиентской части веб-приложения, было принято решение создать компонент Layout, который на страницах клиентской части будет оборачивать основное содержимое. Пример компонента Layout представлен на листинге 3.10.

Листинг 3.10 – код компонента Layout

```

const Layout: React.FC<{ children: React.ReactNode }> = ({
children }) => {
  return (
    <>
      <Header />
      {children}
      <Footer />
    </>
  );
};

```

3.3.2 Реализация функционала, связанного с пользователем

Далее необходимо реализовать функционал, связанный с пользователем, а именно – вход в аккаунт, хранение информации, выход из аккаунта. Как было сказано ранее, вход в аккаунт производится через Telegram бот. Поскольку

полный доступ к веб-приложению пользователь получает только после аутентификации, это означает, что после ее прохождения клиентская часть веб-приложения должны получить информацию о пользователях и сохранить ее в глобальное состояние, чтобы она была доступна любому компоненту клиентской части веб-приложения. Соответственно, для предоставления доступа к информации о пользователе всем компонентам приложения было решено использовать возможности библиотеки Redux Toolkit – слайс-объект, который содержит логику, связанную с определенной частью состояния приложения. Доступ к состоянию и его изменение обеспечивают функции `useSelector` – позволяет извлекать данные из хранилища Redux – и `useDispatch` – позволяет отправлять действия (actions) в хранилище Redux.

Получение информации о пользователе происходит путем отправки запроса на сервер, к запросу прикрепляется cookie с идентификатором сессии. Полученные с сервера данные сохраняются в состояние Redux (листинг В.54). Для отправки запросов на сервер используются возможности библиотеки `axios`. Конфигурация стандартного запроса включает в себя указание базового URL, максимальное время ожидания ответа от сервера и другое (листинг В.36). Пример отправки запроса с клиента на сервер представлен на рисунке 3.11.

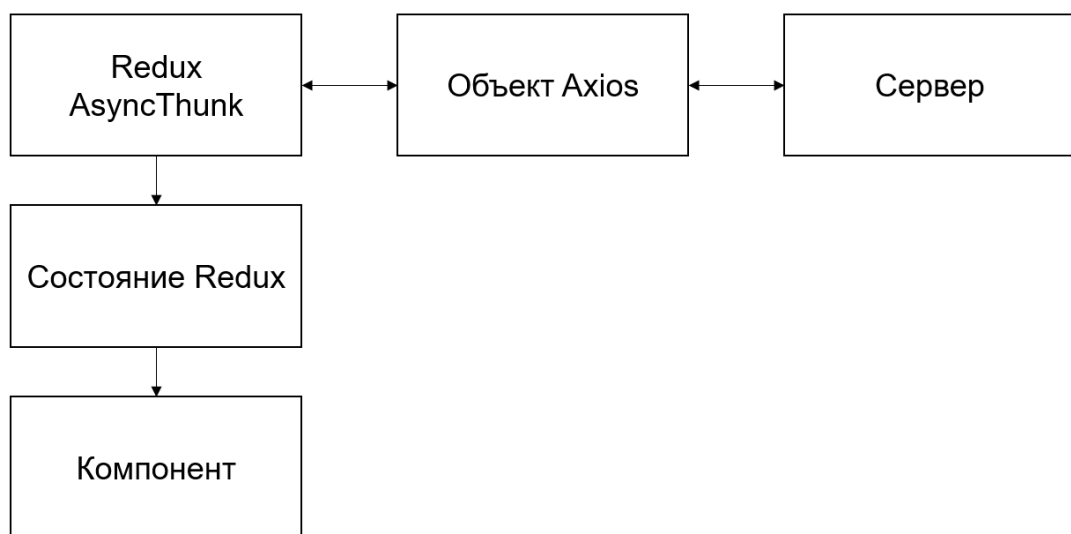


Рисунок 3.11 – пример отправки запроса с клиента на сервер

3.3.3 Реализация функционала, связанного с аудиокнигами, подкастами и авторами

Еще три основных сущности приложения – аудиокниги, подкасты и авторы. Необходимо реализовать логику получения информации об аудиокнигах и подкастах с сервера, страницы с детальной информацией об элементе, логику поиска по каталогу аудиокниг и подкастов. В список аудиокниг и подкастов должна быть вынесена основная информация об элементах – обложка, название и автор. По этой причине было принято решение сделать переиспользуемые компоненты для отдельных элементов списка (листинг В.6).

Данные компоненты будут использованы на главной странице, на страницах со списком аудиокниг и подкастов, странице поиска, профиле пользователя и на странице с историей прослушивания.

На детальной странице необходимо вывести основную информацию о книге или подкасте, вкладку с эпизодами или главами для воспроизведения, а также вкладку с отзывами об аудиокниге или подкасте, предусмотреть возможность добавить элемент в избранное. Для эпизодов и глав был создан отдельный компонент, который показывает пользователю название главы или эпизода и его длительность. Он же отвечает за начало воспроизведения аудиокниги или подкаста. Начать прослушивание может только пользователь с активной сессией, в то время как пользователю без нее будет показано диалоговое окно с предложением пройти аутентификацию. Вторая вкладка содержит список отзывов и форму для создания отзыва. Оценить элемент и написать отзыв может только пользователь с активной сессией, который начал прослушивание данной аудиокниги или подкаста. Выставить оценку с отзывом к одной и той же книге или одному и тому же подкасту пользователь может только один раз. В последующие разы при посещении вкладки с отзывами пользователю будет показываться предупреждение, что он уже оставил отзыв к данному элементу.

3.3.4 Реализация аудиоплеера

Далее необходимо реализовать аудиоплеер для проигрывания эпизодов. В Next JS есть обязательный компонент, который называется App. Содержимое данного компонента применяется абсолютно для всех страниц клиентской части веб-приложения. Поэтому, чтобы воспроизведение не прерывалось при переходе между страницами, было принято решение вынести плеер в отдельный компонент, который в последствии был добавлен в основной компонент App. Также необходимо, чтобы было общее состояние для плеера, которое будет доступно из любой части приложения, поэтому было принято решение создать Redux-слайс для плеера. Аудиоплеер реализован на базе HTML-тега `<audio>` с задействованием его возможностей.

Аудиоплеер появляется на странице только после начала воспроизведения. Воспроизведение начинается после нажатия кнопки в списке эпизодов подкаста или глав книги: по нажатию кнопки в состояние плеера записывается очередь данной книги или подкаста (листинг В.18), код, по которому клиент может получать аудиофайл с сервера, состояние, обозначающее статус воспроизведения – проигрывается или остановлено – и состояние, отвечающее за уровень громкости. Переход к следующему или предыдущему эпизоду осуществляется по нажатию соответствующих кнопок. Он возможен только в случае наличия эпизодов после текущего и до текущего соответственно.

4 Тестирование приложения

4.1 Ручное тестирование

В качестве метода тестирования веб-приложения для прослушивания аудиокниг и подкастов было выбрано ручное тестирование по следующим причинам:

- Ручное тестирование подразумевает взаимодействие с системой от лица пользователя, что позволяет оценить такие параметры, как: отзывчивость интерфейса, скорость работы и другие
- Приложение подразумевает воспроизведение аудио, соответственно, ручное тестирование позволяет оценить качество воспроизводимого файла, скорость передачи и другое
- Ручное тестирование позволяет воспроизвести нестандартные сценарии использования веб-приложения и оценить работу клиента и сервера при определенном сценарии взаимодействия пользователя с системой

4.2 Общие требования

Проведение тестирования приложения является важным этапом в процессе его разработки и обеспечения качества. В ходе ручного тестирования была осуществлена проверка функциональности, а также взаимодействия компонентов веб-приложения.

Первая часть тестирования – проверка корректности работы системы аутентификации пользователей. Ожидаемое поведение от клиента: переадресация на страницу с Telegram ботом по нажатию кнопки «Войти» в шапке клиентской части веб-приложения, получение пользовательской ссылки в Telegram боте. Страница, открытая по данной ссылке, должна выполнить отправку запроса на получение идентификатора сессии в виде cookie от сервера. Ожидаемое поведение от Telegram бота: выполнение запроса на сервер для получения пользовательской ссылки, отправка полученной валидной ссылки для входа запросившему ее пользователю. Ожидаемое поведение от сервера: обработка запросы от Telegram бота,

возврат созданной пользовательской ссылки для входа создание пользователя в случае отсутствии сведений о нем в базе данных, либо получение информации об уже созданном пользователе, обработка запроса с клиента на создании сессии и получение ее идентификатора в виде cookie.

Работа реализованного функционала соответствует ожиданиям (рисунки 5.1, 5.2 и 5.3).

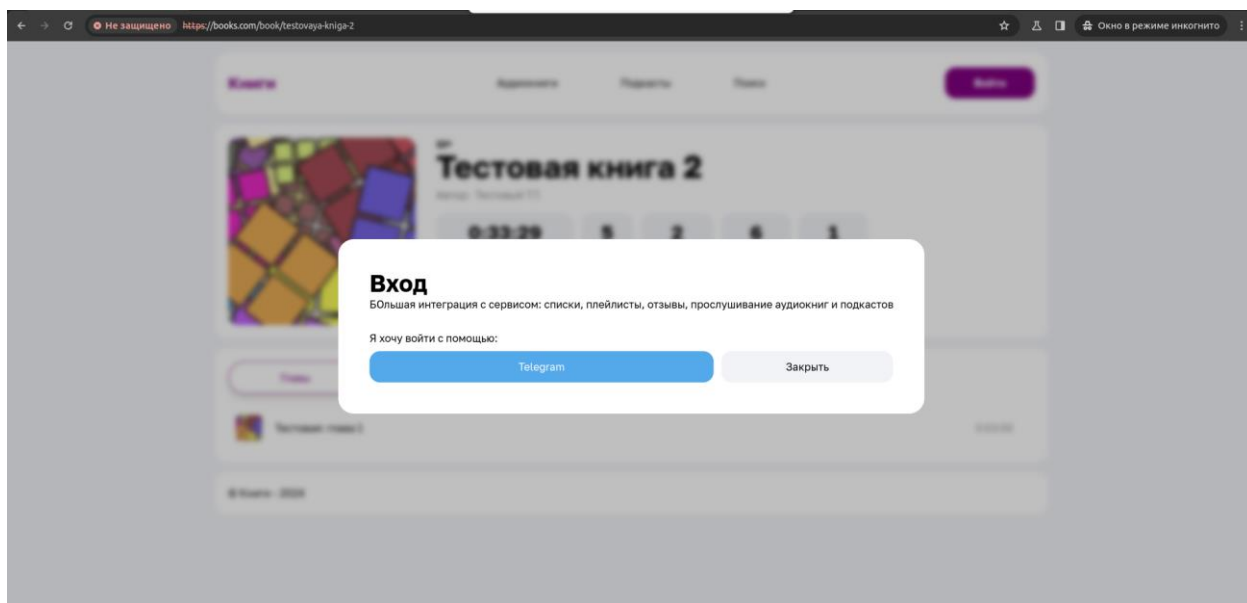


Рисунок 5.1 – диалоговое окно с предложением пройти аутентификацию

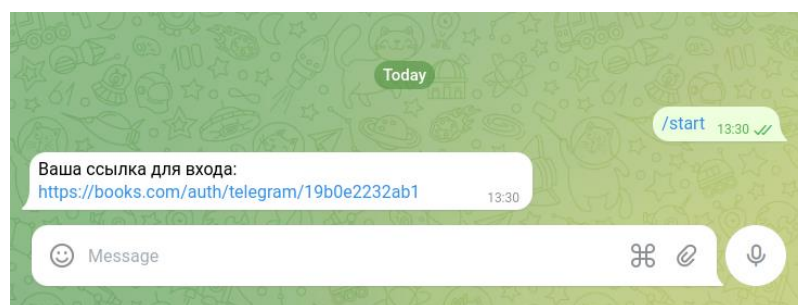


Рисунок 5.2 – получение пользовательской ссылки от Telegram бота

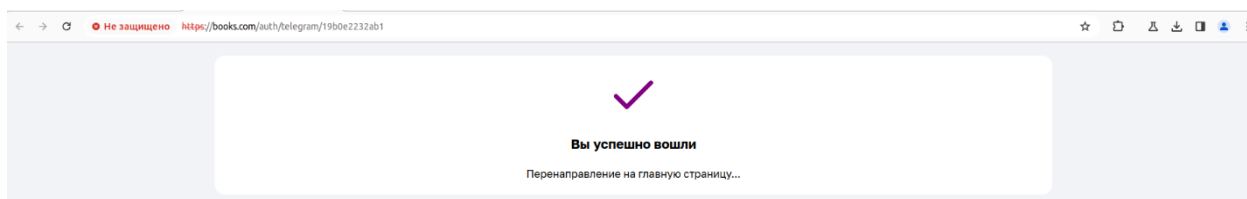


Рисунок 5.3 – страница с результатом аутентификации

Вторая часть тестирования – проверка системы поиска аудиокниг и подкастов. Ожидаемое поведение от клиента: разблокировка кнопки «Поиск»

только после введения запроса, отправка запроса на сервер для получения списка аудиокниг и подкастов по заданной подстроке. Ожидаемое поведение от сервера: возврат списка аудиокниг и подкастов по данной подстроке, либо возврат ошибки со статус-кодом 404 в случае отсутствия релевантных результатов.

Работа реализованного функционала соответствует ожиданиям (рисунок 5.4).

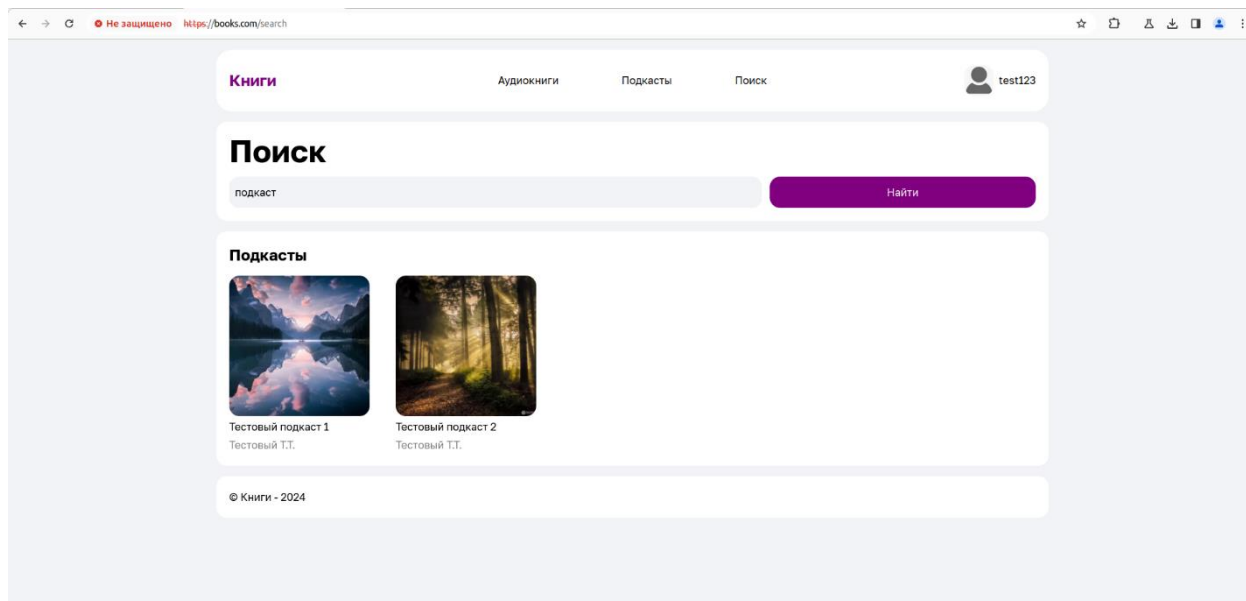


Рисунок 5.4 – результат работы системы поиска

Третья часть тестирования – проверка воспроизведения частей книги и эпизодов подкаста. Ожидаемое поведение от клиента: формирование очереди воспроизведения на основе списка частей данного элемента, начало воспроизведения путем получения фрагмента аудиофайла, переключение между эпизодами кнопками на плеере, если количество эпизодов больше одного. Если у пользователя отсутствует активная сессия, то предложить ему пройти аутентификацию. Ожидаемое поведение от сервера: поиск файла по переданному началу потоковой передачи в случае наличия активной сессии у пользователя.

Работа реализованного функционала соответствует ожиданиям (рисунок 5.5).

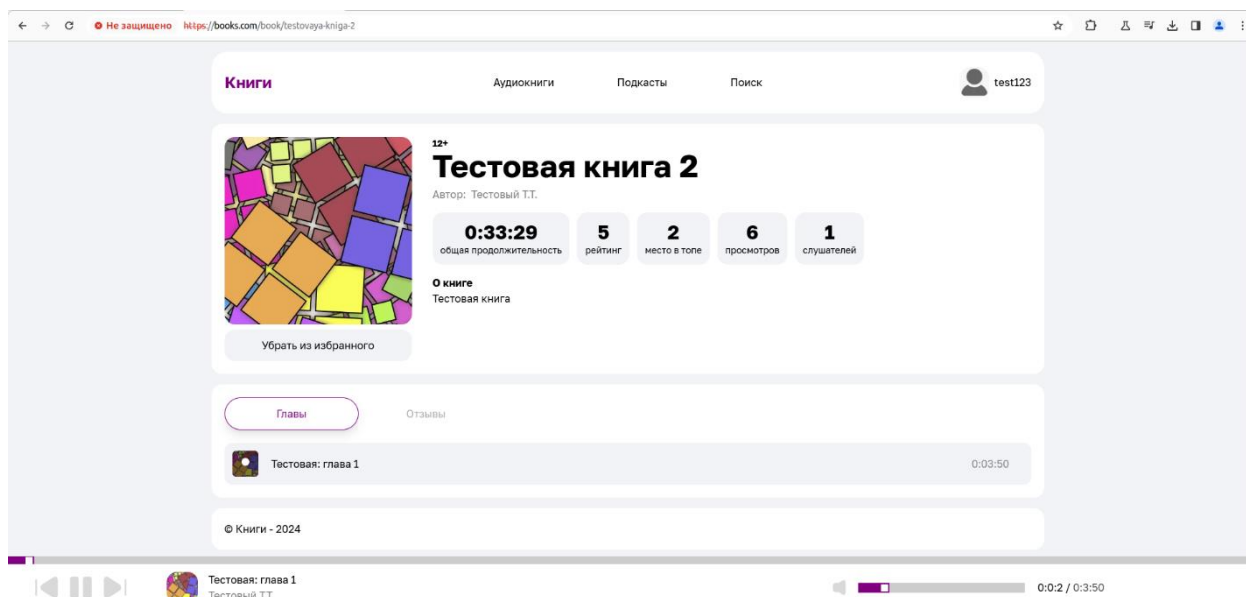


Рисунок 5.5 – результат работы системы воспроизведения

Четвертая часть тестирования – проверка системы добавления отзывов и добавления в избранное. Ожидаемое поведение от клиента: отправка запросов на сервер, изменение состояния в соответствии с ответом сервера. Предложение пройти аутентификацию для пользователей без активной сессии. Ожидаемое поведение от сервера: изменение информации в базе данных в случае, если действие выполняется пользователем с активной сессией.

Работа реализованного функционала соответствует ожиданиям (рисунки 5.6 и 5.7).

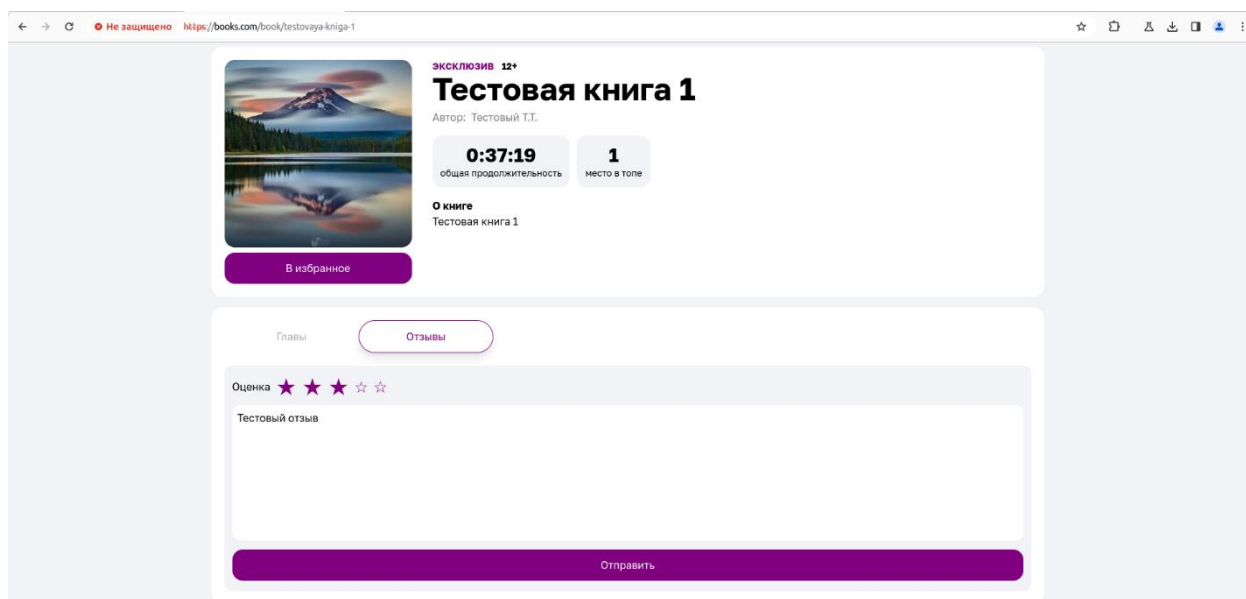


Рисунок 5.6 – форма добавления отзыва

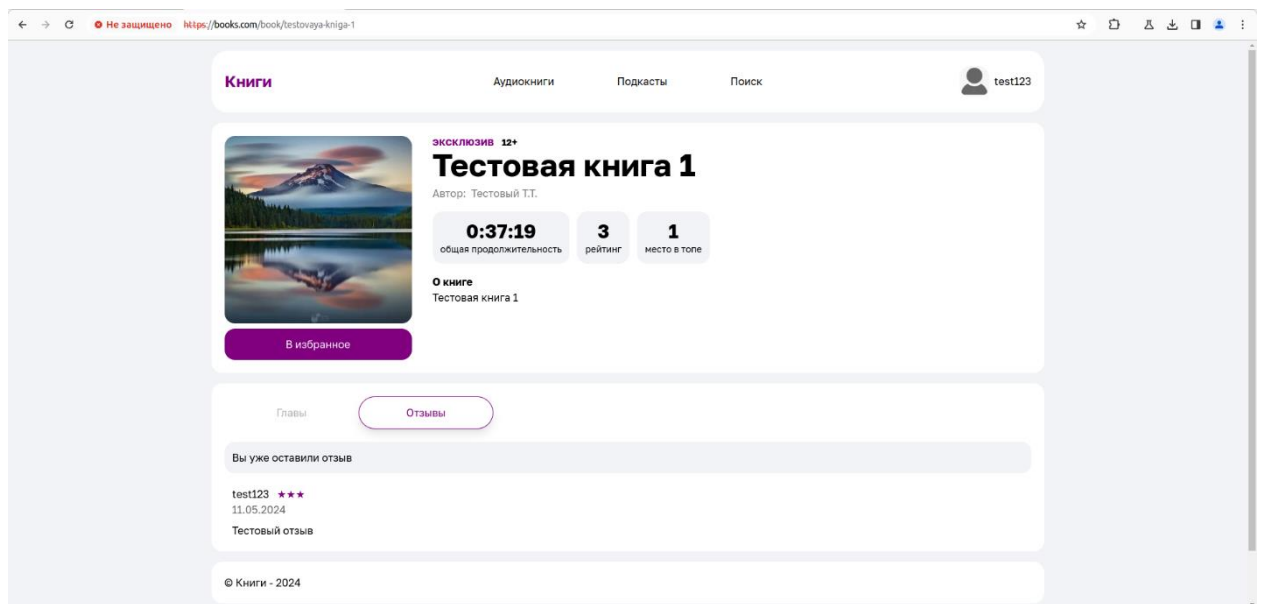


Рисунок 5.7 – уведомление пользователя о том, что отзыв уже был добавлен

ЗАКЛЮЧЕНИЕ

В результате выполненной работы удалось достичь поставленной цели, которая заключалась в анализе существующих решений, определении требований к веб-приложению, выборе инструментов и технологий для разработки, проектировании архитектуры приложения, разработке веб-приложения с использованием выбранных инструментов и технологий, а также проведении тестирования реализованного функционала.

Были выполнены следующие задачи:

- Проведен анализ существующих решений и определены требования к веб-приложению. В ходе выполнения этого этапа были изучены функционал и особенности популярных сервисов для прослушивания аудиокниг и подкастов, а также обозначены основные требования к веб-приложению.

- Выбраны технологии и инструменты для разработки, а также спроектирована архитектура приложения. На этом этапе были определены технологии и инструменты для разработки клиентской и серверной частей, выбрана база данных для хранения информации, спроектирована схема базы данных и определен дополнительный программный продукт для реализации приложения.

- Реализованы клиентская и серверная части приложения. На этом этапе была произведена конфигурация окружения, реализована серверная часть приложения с использованием выбранных технологий, а также реализована клиентская часть приложения с использованием выбранных технологий.

- Проведено тестирование приложения.

В итоге, в соответствии с поставленной целью и выполненными задачами, было разработано веб-приложение для прослушивания аудиокниг и подкастов, соответствующее обозначенным требованиям. По итогам

выполненной работы была написана пояснительная записка к выпускной квалификационной работе

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В пояснительной записке применяются следующие сокращения и обозначения:

HTML (HyperText Markup Language) – язык гипертекстовой разметки

CSS (Cascading Style Sheets) – каскадные таблицы стилей

JS – JavaScript

DOM (Document Object Model) – объектная модель документа

SEO (Search Engine Optimization) – поисковая оптимизация

SQL (Structured Query Language) - язык структурированных запросов

ORM (Object-Relational Mapping) - объектно-реляционное отображение

СУБД – система управления базами данных

DB (Database) – база данных

HTTP (HyperText Transfer Protocol) - протокол передачи гипертекста

YML (YAML Ain't Markup Language) - формат сериализации данных

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Две трети опрошенных россиян предпочитают слушать аудиокниги, а не читать бумажные // ТАСС URL: <https://tass.ru/obschestvo/19975145> (дата обращения: 30.04.2024).
- 2 Подкасты слушают более 10 миллионов россиян // Российская Газета URL: <https://rg.ru/2023/10/20/podkasty-slushaiut-bolee-10-millionov-rossiiian.html> (дата обращения: 30.04.2024).
- 3 На замену бумаге: 6 удобных сервисов для чтения и прослушивания книг // Тинькофф Журнал URL: <https://journal.tinkoff.ru/list/reading-apps/> (дата обращения: 30.04.2024).
- 4 Клиент-серверная архитектура // ServerGate URL: <https://servergate.ru/articles/klient-servernaya-arkhitektura/> (дата обращения: 30.04.2024).
- 5 Что такое технология клиент–сервер // GlavInform URL: <https://glav-inform.ru/chto-takoe-tehnologiya-klient-server/> (дата обращения: 01.05.2024).
- 6 Введение в React // Хабр URL: <https://habr.com/ru/articles/461541/> (дата обращения: 01.05.2024).
- 7 Что такое Next.js // Timeweb Community URL: <https://timeweb.com/ru/community/articles/chto-takoe-next-js> (дата обращения: 01.05.2024).
- 8 Redux // Skillfactory Media URL: <https://blog.skillfactory.ru/glossary/redux/> (дата обращения: 01.05.2024).
- 9 Компания Microsoft представила TypeScript, новую открытую альтернативу JavaScript // OpenNet URL: <https://www.opennet.ru/opennews/art.shtml?num=34981> (дата обращения: 01.05.2024).
- 10 Обзор фреймворка TailwindCSS: чем он хорош и кому будет полезен // Timeweb Community URL: <https://timeweb.com/ru/community/articles/chto->

takoe-tailwindcss-zachem-nuzhen-i-chem-horosh (дата обращения: 01.05.2024).

- 11 Что такое PostgreSQL // Академия Selectel URL: <https://selectel.ru/blog/postgresql/> (дата обращения: 01.05.2024).
- 12 What Is Nest.js? A Look at the Lightweight JavaScript Framework // kinsta URL: <https://kinsta.com/knowledgebase/nestjs/> (дата обращения: 01.05.2024).
- 13 А.В. Ефимов, А.В. Гонцова Разработка высоконагруженных сервисов распределенной обработки и хранения информации: учебное пособие / Сибирский государственный университет телекоммуникаций и информатики; каф. Вычислительных систем. – Новосибирск, 2020. – 83 с.
- 14 nginx // nginx URL: <https://nginx.org/ru/> (дата обращения: 02.05.2024).

ПРИЛОЖЕНИЕ А

Листинги, относящиеся к конфигурации Docker и Nginx

Листинг А.1 – docker.compose.yml

```
services:
  backend:
    build:
      context: ./server
      dockerfile: Dockerfile
    ports:
      - "5000:5000"
    environment:
      - PORT=5000
      - NODE_ENV=development
      - DB=books_app
      - DB_USER=admin
      - DB_USER_PASSWORD=LcmyiXctWnAW
      - DB_HOST=database
      - DB_PORT=5432
      - BOT_API_TOKEN=g2S4A0evF3Vx7
    volumes:
      - ./server/src:/var/www/src
    restart: always
    depends_on:
      - database
    networks:
      - appnet
    extra_hosts:
      - "books.com:172.20.1.100"
  frontend:
    build:
      context: ./client
      dockerfile: Dockerfile
    ports:
      - "3000:3000"
    environment:
      - NEXT_PUBLIC_BACKEND_URL=https://books.com/api
    volumes:
      - ./client:/var/www
    restart: always
    depends_on:
      - backend
    networks:
      - appnet
    extra_hosts:
      - "books.com:172.20.1.100"
  nginx:
    image: nginx:latest
    ports:
      - "80:80"
      - "443:443"
    volumes:
```



```

    - ./nginx/nginx.conf:/etc/nginx/nginx.conf
    - ./nginx/ssl:/etc/nginx/ssl
  restart: always
  networks:
    appnet:
      ipv4_address: 172.20.1.100
database:
  image: postgres
  restart: always
  ports:
    - 5432:5432
  environment:
    - POSTGRES_DB=books_app
    - POSTGRES_USER=admin
    - POSTGRES_PASSWORD=LcmyiXctWnAW
    - PGDATA=/data/postgres
  volumes:
    - ./pgdata:/data/postgres
  networks:
    - appnet
adminer:
  image: adminer
  restart: always
  ports:
    - 8080:8080
  networks:
    - appnet
telegrambot:
  build:
    context: ./telegrambot
    dockerfile: Dockerfile
  environment:
    - BOT_TOKEN=7045320214:AAESQEITD_fP1fJl281UvtEI9jiUxozFRns
    - API_TOKEN=g2S4A0evF3Vx7
    - WEB_URL=https://books.com
  volumes:
    - ./telegrambot/main.py:/var/www/main.py
  restart: always
  networks:
    - appnet
  extra_hosts:
    - "books.com:172.20.1.100"

networks:
  appnet:
    ipam:
      config:
        - subnet: 172.20.0.0/16
          gateway: 172.20.0.1
          ip_range: 172.20.1.0/24

```

Листинг A.2 – client/Dockerfile

```
FROM node:21
WORKDIR /var/www
COPY . /var/www/
RUN npm install
EXPOSE 3000
CMD [ "npm", "run", "dev" ]
```

Листинг A.3 – client/Dockerfile

```
FROM node:21
WORKDIR /var/www
COPY . .
RUN npm install
EXPOSE 5000
CMD [ "npm", "run", "start:dev" ]
```

Листинг A.4 – telegrambot/Dockerfile

```
FROM python:3.12.3-alpine3.19
WORKDIR /var/www
COPY requirements.txt /var/www/
RUN pip3 install --no-cache-dir -r /var/www/requirements.txt
COPY . /var/www/
CMD [ "python", "./main.py" ]
```

Листинг A.5 – nginx/nginx.conf

```
events {}
http {
    server {
        listen 80;
        server_name books.com www.books.com;
        return 301 https://books.com$request_uri;
    }
    server {
        listen 80;
        server_name books.com;

        location / {
            proxy_pass http://frontend:3000;
        }
        location /api {
            proxy_pass http://backend:5000;
        }
        # SSL configuration
        listen 443 ssl;
        ssl_certificate /etc/nginx/ssl/nginx.crt;
        ssl_certificate_key /etc/nginx/ssl/nginx.key;

        # Enable HTTP/2
        http2 on;
    }
}
```

ПРИЛОЖЕНИЕ Б

Листинги, относящиеся к серверной части приложения

Листинг Б.1 – telegrambot/main.py

```
import os
import telebot
import requests
from telebot import types

def getLink(username):
    payload = {"username": username, "botApiToken":
os.environ["API_TOKEN"]}
    result = requests.post(os.environ['WEB_URL'] +
"/api/auth/telegram/link/", verify=False, data=payload)
    return result.json()['link'];

bot = telebot.TeleBot(os.environ["BOT_TOKEN"])

@bot.message_handler(func=lambda message: message.text=='/start'
or message.text=='Получить ссылку для входа')
def send_welcome(message):
    link = getLink(message.from_user.username)
    markup = types.ReplyKeyboardMarkup(row_width=1)
    itemAuth = types.KeyboardButton("Получить ссылку для входа")
    markup.row(itemAuth)
    answer = "Ваша ссылка для входа: " + os.environ['WEB_URL'] +
'/auth/telegram/' + link
    bot.send_message(message.chat.id, answer,
reply_markup=markup)

bot.infinity_polling()
```

Листинг Б.2 – server/src/app.module.ts

```
import { Module } from '@nestjs/common';
import { SequelizeModule } from '@nestjs/sequelize';
import { AppService } from './app.service';
import { AuthModule } from './auth/auth.module';
import { RolesModule } from './roles/roles.module';
import { SessionsModule } from './sessions/sessions.module';
import { UsersModule } from './users/users.module';
import { BooksModule } from './books/books.module';
import { AuthorsModule } from './authors/authors.module';
import { PodcastsModule } from './podcasts/podcasts.module';
import { SearchModule } from './search/search.module';
import { GenresModule } from './genres/genres.module';
import { BookspartsModule } from
'./booksparts/booksparts.module';
```

```

import { PodcastsepisodesModule } from
'./podcastsepisodes/podcastsepisodes.module';
import { FilesystemModule } from
'./filesystem/filesystem.module';
import { HistoryModule } from './history/history.module';
import { ReviewsModule } from './reviews/reviews.module';
import { FavoritesModule } from './favorites/favorites.module';
@Module({
  imports: [
    SequelizeModule.forRoot({
      dialect: 'postgres',
      host: process.env.DB_HOST,
      username: process.env.DB_USER,
      password: process.env.DB_USER_PASSWORD,
      database: process.env.DB,
      autoLoadModels: true,
    }),
    UsersModule,
    RolesModule,
    SessionsModule,
    AuthModule,
    BooksModule,
    AuthorsModule,
    PodcastsModule,
    SearchModule,
    GenresModule,
    BookspartsModule,
    PodcastsepisodesModule,
    FilesystemModule,
    HistoryModule,
    ReviewsModule,
    FavoritesModule,
  ],
  providers: [AppService],
})
export class AppModule {}

```

Листинг Б.3 – server/src/app.service.ts

```

import { Injectable } from '@nestjs/common';
@Injectable()
export class AppService {
  getHello(): string {
    return 'Hello World!';
  }
}

```

Листинг Б.4 – server/src/main.ts

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from '../app.module';
import { ValidationPipe } from '@nestjs/common';
import * as cookieParser from 'cookie-parser';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.useGlobalPipes(new ValidationPipe());
  app.setGlobalPrefix('api');
  app.use(cookieParser());
  await app.listen(process.env.PORT);
}
bootstrap();
```

Листинг Б.5 – server/src/auth/auth.controller.ts

```
import {
  BadRequestException,
  Body,
  Controller,
  Delete,
  Get,
  HttpStatusCode,
  HttpException,
  InternalServerErrorException,
  Ip,
  Param,
  Post,
  Req,
  Res,
  UnauthorizedException,
} from '@nestjs/common';
import { Request } from 'express';
import { SessionsService } from 'src/sessions/sessions.service';
import { AuthService } from '../auth.service';
import { CreateTgLinkDto } from '../dto/create-tg-link.dto';

@Controller('auth')
export class AuthController {
  constructor(
    private authService: AuthService,
    private sessionsService: SessionsService,
  ) {}

  @Post('/telegram/link')
  @HttpCode(200)
```

```

async createTelegramLink(@Body() dto: CreateTgLinkDto) {
    if (dto.botApiToken !== process.env.BOT_API_TOKEN) {
        throw new UnauthorizedException();
    }

    const link = await this.authService.createLink(dto);
    if (!link) {
        throw new InternalServerErrorException();
    }

    return { link };
}

@Delete('/telegram/link/:link')
@HttpCode(200)
async deleteTelegramLink(
    @Ip() ip: string,
    @Param('link') link: string,
    @Res() res,
) {
    if (!link.trim()) {
        throw new BadRequestException();
    }
    const result = await this.authService.markLinkAsUsed(link,
ip);
    console.log(result);
    if (result.status !== 200) {
        throw new HttpException(result.error, result.status);
    }
    return res
        .cookie('session', result.sessionCode, {
            httpOnly: true,
            maxAge: 2 * 30 * 24 * 60 * 60 * 1000,
        })
        .json(result);
}

@Delete('/logout')
@HttpCode(200)
logout(@Res() res) {
    return res.clearCookie('session').json();
}

@Get('/amIAuthorized')
async amIAuthorized(@Req() request: Request) {
    if (!request.cookies.session) {

```

```

        throw new UnauthorizedException();
    }
    const session = await
this.sessionsService.getActualSessionByCode(
    request.cookies.session,
);
    if (!session) {
        throw new UnauthorizedException();
    }

    return session.users;
}
}

```

Листинг Б.6 – server/src/auth/auth.module.ts

```

import { Module } from '@nestjs/common';
import { AuthController } from '../auth.controller';
import { AuthService } from '../auth.service';
import { SequelizeModule } from '@nestjs/sequelize';
import { TelegramCodes } from '../models/telegram-codes.model';
import { SessionsModule } from 'src/sessions/sessions.module';

@Module({
  controllers: [AuthController],
  providers: [AuthService],
  imports: [SequelizeModule.forFeature([TelegramCodes]),
SessionsModule]
})
export class AuthModule {}

```

Листинг Б.7 – server/src/auth/auth.service.ts

```

import { Injectable } from '@nestjs/common';
import { InjectModel } from '@nestjs/sequelize';
import { randomBytes } from 'crypto';
import { SessionsService } from 'src/sessions/sessions.service';
import { CreateTgLinkDto } from '../dto/create-tg-link.dto';
import { ITgCodeStatus } from '../dto/tg-code-status.dto';
import { TelegramCodes } from '../models/telegram-codes.model';

@Injectable()
export class AuthService {
  constructor(
    @InjectModel(TelegramCodes)
    private telegramCodesRepository: typeof TelegramCodes,
    private sessionService: SessionsService,

```

```

) {}

private async getLink(link: string): Promise<TelegramCodes> {
  try {
    const foundLink = this.telegramCodesRepository.findOne({
      where: { login_string: link, used: false },
    });
    return foundLink;
  } catch (e) {
    return null;
  }
}

async createLink(dto: CreateTgLinkDto): Promise<string> {
  try {
    const link = randomBytes(12).toString('hex').slice(0, 12);
    let date = new Date();
    date.setDate(date.getDate() + 3);
    const dbFriendlyDate = date.toISOString();

    await this.telegramCodesRepository.create({
      login_string: link,
      expires_in: dbFriendlyDate,
      username: dto.username,
    });
    return link;
  } catch (e) {
    console.error(e);
    return null;
  }
}

async markLinkAsUsed(link: string, ip: string):
Promise<ITgCodeStatus> {
  const linkFromDb = await this.getLink(link);

  if (!linkFromDb) {
    return { status: 404, error: 'Ссылка не найдена' };
  }
  try {
    await this.telegramCodesRepository.update(
      { used: true },
      { where: { login_string: link, used: false } },
    );
    const sessionCodeResult = await
this.sessionService.createSession(

```



```

        ip,
        linkFromDb.username,
    );
    return { status: 200, sessionCode:
sessionCodeResult.sessionCode };
    } catch (e) {
        console.error(e);
        return { status: 500 };
    }
}
}
}

```

Листинг Б.8 – server/src/auth/dto/create-tg-link.dto.ts

```

import { Equals, IsNotEmpty } from 'class-validator';

export class CreateTgLinkDto {
    @IsNotEmpty()
    readonly username: string;

    @IsNotEmpty()
    readonly botApiToken: string;
}

```

Листинг Б.9 – server/src/auth/dto/tg-code-status.dto.ts

```

import { IServiceAnswer } from 'src/helpers/service-
answer.interface';
import { TelegramCodes } from '../models/telegram-codes.model';

export interface ITgCodeStatus extends IServiceAnswer {
    sessionCode?: string;
    tgCode?: TelegramCodes;
}

```

Листинг Б.10 – server/src/auth/models/telegram-codes.model.ts

```

import { Column, DataType, Model, Table } from 'sequelize-
typescript';

@Table({ tableName: 'telegram_codes' })
export class TelegramCodes extends Model<TelegramCodes> {
    @Column({
        type: DataType.INTEGER,
        unique: true,
        autoIncrement: true,
        primaryKey: true,
    })

```

```

    id: number;

    @Column({
      type: DataType.STRING(12),
      unique: true,
    })
    login_string: string;

    @Column({
      type: DataType.STRING,
    })
    username: string;

    @Column({
      type: DataType.BOOLEAN,
      defaultValue: false
    })
    used: boolean;

    @Column({
      type: DataType.DATE
    })
    expires_in: string
  }

```

Листинг Б.11 – server/src/authors/authors.controller.ts

```

import { Controller, Get, HttpException, Param } from
'@nestjs/common';
import { AuthorsService } from '../authors.service';

@Controller('authors')
export class AuthorsController {
  constructor(private authorsService: AuthorsService) {}
  @Get('/one/:code')
  async getOne(@Param('code') code: string) {
    const authorResult = await
this.authorsService.getAuthorWhere({ code }, true);
    if (authorResult.status !== 200) {
      throw new HttpException(authorResult.error,
authorResult.status);
    }

    return authorResult.author;
  }
}

```

Листинг Б.12 – server/src/authors/authors.module.ts

```
import { Module } from '@nestjs/common';
import { AuthorsController } from '../authors.controller';
import { AuthorsService } from '../authors.service';
import { SequelizeModule } from '@nestjs/sequelize';
import { Authors } from '../models/authors.model';

@Module({
  controllers: [AuthorsController],
  providers: [AuthorsService],
  imports: [SequelizeModule.forFeature([Authors])],
})
export class AuthorsModule {}
```

Листинг Б.13 – server/src/authors/authors.service.ts

```
import { Injectable } from '@nestjs/common';
import { InjectModel } from '@nestjs/sequelize';
import { CyrillicLatinTransliteration } from
'src/helpers/cyrillic-latin.helper';
import { IAuthorStatus } from '../dto/author-status.interface';
import { CreateAuthorDto } from '../dto/create-author.dto';
import { Authors } from '../models/authors.model';
import { UpdateAuthorDto } from '../dto/update-author.dto';
import { FindOptions } from 'sequelize';
import { Books } from 'src/books/models/books.model';
import { Podcasts } from 'src/podcasts/models/podcasts.model';

@Injectable()
export class AuthorsService {
  constructor(
    @InjectModel(Authors) private authorsRepository: typeof
Authors,
  ) {}

  async getAuthorWhere(
    where: any,
    includeAssociations?: boolean,
  ): Promise<IAuthorStatus> {
    try {
      let findOneOptions: FindOptions<Authors> = {
        where,
        attributes: { exclude: ['createdAt', 'updatedAt'] },
      };
      if (includeAssociations) {
        findOneOptions = {
          ...findOneOptions,
```

```

        include: [
            {
                model: Books,
                attributes: { exclude: ['author_id', 'createdAt',
'updatedAt'] },
            },
            {
                model: Podcasts,
                attributes: { exclude: ['author_id', 'createdAt',
'updatedAt'] },
            },
        ],
    };
}

    const author = await
this.authorsRepository.findOne(findOneOptions);
    console.log(author);
    if (!author) {
        return { status: 404, error: 'Автор не найден' };
    }
    return { author, status: 200 };
} catch (e) {
    return { status: 500 };
}
}

    async createAuthor(dto: CreateAuthorDto):
Promise<IAuthorStatus> {
        const code =
CyrillicLatinTransliteration.translate(dto.name);
        let authorState = dto;
        const existedAuthor = await this.getAuthorWhere({ code });
        if (existedAuthor.status === 200) {
            return {
                author: null,
                status: 409,
                error: 'Автор с таким именем или кодом уже есть',
            };
        }
        authorState['code'] = code;

        try {
            const newAuthor = await
this.authorsRepository.create(authorState);
            return { author: newAuthor, status: 200 };
        }
    }
}

```

```

        } catch (e) {
            return { author: null, status: 500 };
        }
    }

    async updateAuthor(dto: UpdateAuthorDto):
    Promise<IAuthorStatus> {
        try {
            await this.authorsRepository.update(dto, { where: { id:
dto.id } });
            return { status: 200 };
        } catch (e) {
            return { status: 500 };
        }
    }
}

```

Листинг Б.14 – server/src/authors/dto/author-status.interface.ts

```

import { IServiceAnswer } from 'src/helpers/service-
answer.interface';
import { Authors } from '../models/authors.model';

export interface IAuthorStatus extends IServiceAnswer {
    author?: Authors;
    authors?: Authors[];
}

```

Листинг Б.15 – server/src/authors/dto/create-author.dto.ts

```

export class CreateAuthorDto {
    name: string;
    bio?: string;
    type?: string;
}

```

Листинг Б.16 – server/src/authors/dto/update-author.dto.ts

```

export class UpdateAuthorDto {
    id: number;
    name?: string;
    bio?: string;
    type?: string;
    code?: string;
}

```

Листинг Б.17 – server/src/authors/models/authors.model.ts

```
import { Column, DataType, HasMany, Model, Table } from
'sequelize-typescript';
import { Books } from 'src/books/models/books.model';
import { Podcasts } from 'src/podcasts/models/podcasts.model';

@Table({ tableName: 'authors' })
export class Authors extends Model<Authors> {
  @Column({
    type: DataType.INTEGER,
    unique: true,
    autoIncrement: true,
    primaryKey: true,
  })
  id: number;

  @Column({
    type: DataType.STRING,
  })
  name: string;

  @Column({
    type: DataType.STRING,
    unique: true,
  })
  code: string;

  @Column({
    type: DataType.TEXT,
  })
  bio: string;

  @Column({
    type: DataType.STRING,
  })
  type: string;

  @HasMany(() => Books)
  books: Books[];

  @HasMany(() => Podcasts)
  podcasts: Podcasts[];
}
```

Листинг Б.18 – server/src/books/books.controller.ts

```
import {
  Controller,
  Get,
  HttpException,
  Param,
  Query,
  Req
} from '@nestjs/common';
import { Request } from 'express';
import { BooksService } from '../books.service';
import { QuerySortBookDto } from '../dto/query-sort-books.dto';

@Controller('books')
export class BooksController {
  constructor(private booksService: BooksService) {}

  @Get('/search')
  async searchBook(
    @Query('searchQuery') query: string,
    @Query('page') page: number,
    @Req() req: Request,
  ) {
    const booksResult = await this.booksService.searchBook(
      page,
      query,
      req.cookies.session,
    );
    if (booksResult.status !== 200) {
      throw new HttpException(booksResult.error,
booksResult.status);
    }

    return booksResult.books;
  }

  @Get('/list')
  async getBooks(@Query() query: QuerySortBookDto, @Req() req:
Request) {
    const booksResult = await
this.booksService.getSortedBooksList(
      query,
      req.cookies.session,
    );
    if (booksResult.status !== 200) {
      throw new HttpException(booksResult.error,
booksResult.status);
    }
  }
}
```

```

    }

    return { books: booksResult.books, count: booksResult.count
};
}

@Get('/trending')
async getTrendingBooks(@Req() req: Request) {
    const booksResult = await
this.booksService.getPopularBooksList(req.cookies.session);
    if (booksResult.status !== 200) {
        throw new HttpException(booksResult.error,
booksResult.status);
    }

    return booksResult.books;
}

@Get('/one/:code')
async getBook(@Req() req: Request, @Param('code') code:
string) {
    const bookResult = await this.booksService.getBookWhere({
code }, req.cookies.session);
    if (bookResult.status !== 200) {
        throw new HttpException(bookResult.error,
bookResult.status);
    }

    await this.booksService.addView(req.cookies.session,
bookResult.book.id);

    return bookResult.book;
}
}

```

Листинг Б.19 – server/src/books/books.module.ts

```

import { Module } from '@nestjs/common';
import { SequelizeModule } from '@nestjs/sequelize';
import { SessionsModule } from 'src/sessions/sessions.module';
import { BooksController } from '../books.controller';
import { BooksService } from '../books.service';
import { BooksViews } from '../models/books-views.model';
import { Books } from '../models/books.model';

@Module({
    controllers: [BooksController],

```



```

    providers: [BooksService],
    imports: [
        SequelizeModule.forFeature([Books, BooksViews]),
        SessionsModule,
    ],
    exports: [BooksService],
})
export class BooksModule {}

```

Листинг Б.20 – server/src/books/books.service.ts

```

import { Inject, Injectable } from '@nestjs/common';
import { InjectModel } from '@nestjs/sequelize';
import { Includeable, Op } from 'sequelize';
import { Authors } from 'src/authors/models/authors.model';
import { Genres } from 'src/genres/models/genres.model';
import { CyrillicLatinTransliteration } from
'src/helpers/cyrillic-latin.helper';
import { SessionsService } from 'src/sessions/sessions.service';
import { IBookStatus } from '../dto/book-status.interface';
import { CreateBookDto } from '../dto/create-book.dto';
import { QuerySortBookDto } from '../dto/query-sort-books.dto';
import { UpdateBookDto } from '../dto/update-book.dto';
import { BooksViews } from '../models/books-views.model';
import { Books } from '../models/books.model';
import { BooksReviews } from 'src/reviews/model/books-
reviews.model';
import { Sequelize } from 'sequelize-typescript';
import { Users } from 'src/users/models/users.model';
import { BooksFavorites } from 'src/favorites/models/books-
favorites.model';

@Injectable()
export class BooksService {
    constructor(
        @InjectModel(Books) private booksRepository: typeof Books,
        @InjectModel(BooksViews) private viewsRepository: typeof
BooksViews,
        @Inject(SessionsService) private sessionsService:
SessionsService,
    ) {}

    async commonInclude(sessionCode: string | null):
Promise<Includeable[]> {
        let userId = null;
        if (sessionCode) {
            userId = (await

```

```

this.sessionsService.getActualSessionByCode(sessionCode))
    .user_id;
}
return [
    { model: Authors, attributes: ['name', 'code'] },
    {
        model: BooksFavorites,
        attributes: ['user_id', 'book_id'],
        where: { user_id: userId },
        include: [{ model: Users, attributes: ['username'] }],
        required: false,
    },
];
}

async getBookWhere(
    where: any,
    sessionCode?: string | null,
): Promise<IBookStatus> {
    try {
        let userId = null;
        if (sessionCode) {
            userId = (
                await
this.sessionsService.getActualSessionByCode(sessionCode)
            ).user_id;
        }
        const book = await this.booksRepository.findOne({
            where,
            include: [
                { model: Authors, attributes: ['name', 'code'] },
                {
                    model: BooksReviews,
                    include: [{ model: Users, attributes: ['username']
}],
                    attributes: ['id', 'text', 'rating', 'createdAt'],
                },
                {
                    model: BooksFavorites,
                    attributes: ['user_id', 'book_id'],
                    where: { user_id: userId },
                    include: [{ model: Users, attributes: ['username']
}],
                    required: false,
                },
            ],
        },
    ],

```

```

    });
    if (!book) {
        return { status: 404 };
    }

    if (book.reviews.length > 0) {
        const totalRating = book.reviews.reduce(
            (sum, review) => sum + review.rating,
            0,
        );
        const averageRating = parseFloat(
            (totalRating / book.reviews.length).toFixed(2),
        );
        book.dataValues.rating = averageRating;
    }
    return { book, status: 200 };
} catch (e) {
    console.log(e);
    return { status: 500 };
}
}

async searchBook(
    page: number,
    searchQuery?: string,
    sessionCode?: string | null,
): Promise<IBookStatus> {
    let whereCondition = searchQuery
        ? {
            [Op.or]: {
                name: { [Op.substring]: searchQuery },
                description: { [Op.substring]: searchQuery },
            },
        }
        : {};
    try {
        const books = await this.booksRepository.findAll({
            include: await this.commonInclude(sessionCode),
            attributes: { exclude: ['author_id', 'createdAt',
'updatedAt'] },
            where: whereCondition,
            order: [['sort', 'ASC']],
            limit: 10,
            offset: (page - 1) * 10,
        });
        if (books.length == 0) {

```

```

        return { status: 404, error: 'Нет результатов по данному
запросу' };
    }
    return { books: books, status: 200 };
} catch (e) {
    return { status: 500, error: 'Произошла ошибка на сервере'
};
}
}

async getSortedBooksList(
    dto: QuerySortBookDto,
    sessionCode?: string | null,
): Promise<IBookStatus> {
    try {
        const books = await this.booksRepository.findAll({
            order: [['sort', 'ASC']],
            include: await this.commonInclude(sessionCode),
            attributes: {
                exclude: ['author_id', 'createdAt', 'updatedAt'],
            },
            limit: 10,
            offset: (dto.page - 1) * 10,
        });

        const count = await this.booksRepository.count();
        return { books, count, status: 200 };
    } catch (e) {
        return { status: 500 };
    }
}

async getPopularBooksList(sessionCode: string | null):
Promise<IBookStatus> {
    try {
        const books = await this.booksRepository.findAll({
            order: [['trending', 'ASC']],
            include: await this.commonInclude(sessionCode),
            attributes: { exclude: ['author_id', 'createdAt',
'updatedAt'] },
            limit: 5,
        });

        return { books, status: 200 };
    } catch (e) {
        return { status: 500 };
    }
}

```

```

    }
}

async createBook(dto: CreateBookDto): Promise<IBookStatus> {
    const code =
CyrillicLatinTransliteration.translate(dto.name);
    const bookResult = await this.getBookWhere({ code });
    if (bookResult.status !== 200) {
        return bookResult;
    }
    let bookValues = dto;
    bookValues['code'] = code;

    try {
        const newBook = await
this.booksRepository.create(bookValues, {
            raw: true,
        });
        return { book: newBook, status: 200 };
    } catch (e) {
        return { status: 500 };
    }
}

async updateBook(dto: UpdateBookDto): Promise<IBookStatus> {
    try {
        await this.booksRepository.update(dto, { where: { id:
dto.id } });
        return { status: 200 };
    } catch (e) {
        return { status: 500 };
    }
}

async removeBook(id: number): Promise<IBookStatus> {
    try {
        await this.booksRepository.destroy({ where: { id } });
        return { status: 200 };
    } catch (e) {
        return { status: 500 };
    }
}

async addView(sessionCode: string, bookId: number):
Promise<IBookStatus> {
    try {

```

```

    if (!sessionCode) {
        return;
    }
    const session =
        await
this.sessionsService.getActualSessionByCode(sessionCode);

    const now = new Date();

    const dayAgo = new Date(now.getTime() - 24 * 60 * 60 *
1000);

    const isViewExist = await this.viewsRepository.findOne({
        where: {
            user_id: session.user_id,
            book_id: bookId,
            createdAt: {
                [Op.gte]: dayAgo,
            },
        },
        raw: true,
    });

    if (isViewExist) {
        return;
    }

    await this.viewsRepository.create({
        user_id: session.user_id,
        book_id: bookId,
    });

    const book = await this.booksRepository.findOne({
        where: { id: bookId },
    });
    await this.booksRepository.update(
        { views: book.views ? book.views + 1 : 1 },
        { where: { id: bookId } },
    );

    return { status: 200 };
} catch (e) {
    return { status: 500 };
}
}

```

```
}
```

Листинг Б.21 – server/src/books/dto/book-status.interface.ts

```
import { IServiceAnswer } from 'src/helpers/service-  
answer.interface';  
import { Books } from '../models/books.model';  
import { BooksHistory } from '../history/models/books-  
history.model';  
  
export interface IBookStatus extends IServiceAnswer {  
  book?: Books;  
  books?: Books[];  
  history?: BooksHistory[];  
  count?: number;  
}
```

Листинг Б.22 – server/src/books/dto/create-book.dto.ts

```
export class CreateBookDto {  
  readonly name: string;  
  readonly cover?: string;  
  readonly description: string;  
  readonly minAge: string;  
  readonly exclusive: boolean;  
  readonly sort?: number;  
  readonly authorId: number;  
}
```

Листинг Б.23 – server/src/books/dto/query-sort-books.dto.ts

```
import { IsNotEmpty } from "class-validator";  
  
export class QuerySortBookDto {  
  readonly name?: 'ASC' | 'DESC';  
  readonly sort?: 'ASC' | 'DESC';  
  readonly trending?: 'ASC' | 'DESC';  
  readonly listens?: 'ASC' | 'DESC';  
  readonly views?: 'ASC' | 'DESC';  
  @IsNotEmpty()  
  readonly page: number;  
}
```

Листинг Б.24 – server/src/books/dto/udpate-book.dto.ts

```
export class UpdateBookDto {  
  readonly id: number;  
  readonly name?: string;
```

```

    readonly cover?: string;
    readonly description: string;
    readonly minAge?: string;
    readonly exclusive?: boolean;
    readonly sort?: number;
    readonly authorId?: number;
}

```

Листинг Б.25 – server/src/books/models/books-views.model.ts

```

import {
    Column,
    DataType,
    ForeignKey,
    Model,
    Table,
} from 'sequelize-typescript';
import { Users } from 'src/users/models/users.model';
import { Books } from './books.model';

@Table({ tableName: 'books_views' })
export class BooksViews extends Model<BooksViews> {
    @Column({
        type: DataType.INTEGER,
        unique: true,
        autoIncrement: true,
        primaryKey: true,
    })
    id: number;

    @ForeignKey(() => Users)
    @Column({
        type: DataType.INTEGER,
    })
    user_id: number;

    @ForeignKey(() => Books)
    @Column({
        type: DataType.INTEGER,
    })
    book_id: number;
}

```

Листинг Б.26 – server/src/books/models/books.model.ts

```

import {
    BelongsTo,

```



```

    BelongsToMany,
    Column,
    DataType,
    ForeignKey,
    HasMany,
    Model,
    Table,
} from 'sequelize-typescript';
import { Authors } from 'src/authors/models/authors.model';
import { BooksParts } from 'src/booksparts/models/books-
parts.model';
import { BooksGenres } from 'src/genres/models/books-
genres.model';
import { Genres } from 'src/genres/models/genres.model';
import { BooksHistory } from '../history/models/books-
history.model';
import { BooksReviews } from 'src/reviews/model/books-
reviews.model';
import { BooksFavorites } from 'src/favorites/models/books-
favorites.model';

@Table({ tableName: 'books' })
export class Books extends Model<Books> {
    @Column({
        type: DataType.INTEGER,
        unique: true,
        autoIncrement: true,
        primaryKey: true,
    })
    id: number;

    @Column({
        type: DataType.STRING,
    })
    name: string;

    @Column({
        type: DataType.STRING,
        unique: true,
    })
    code: string;

    @Column({
        type: DataType.INTEGER,
    })
    trending: number;

```

```

@Column({
  type: DataType.STRING,
})
cover: string;

@Column({
  type: DataType.TEXT,
})
description: string;

@Column({
  type: DataType.FLOAT,
})
rating: number;

@Column({
  type: DataType.INTEGER,
})
min_age: number;

@Column({
  type: DataType.BOOLEAN,
  defaultValue: false,
})
exclusive: boolean;

@Column({
  type: DataType.FLOAT,
})
unsub_price: number;

@Column({
  type: DataType.STRING,
})
total_duration: string;

@Column({
  type: DataType.INTEGER,
  defaultValue: 0,
})
sort: number;

@ForeignKey(() => Authors)
@Column({
  type: DataType.INTEGER,

```

```

    })
    author_id: number;

    @BelongsTo(() => Authors)
    author: Authors;

    @Column({
        type: DataType.INTEGER,
        defaultValue: 0,
    })
    listeners: number;

    @Column({
        type: DataType.INTEGER,
        defaultValue: 0,
    })
    views: number;

    @BelongsToMany(() => Genres, () => BooksGenres)
    genres: Genres[];

    @HasMany(() => BooksParts)
    bookParts: BooksParts[];

    @HasMany(() => BooksHistory)
    history: BooksHistory[];

    @HasMany(() => BooksReviews)
    reviews: BooksReviews[];

    @HasMany(() => BooksFavorites)
    favorites: BooksFavorites[];
}

```

Листинг Б.27 – server/src/favorites/favorites.controller.ts

```

import {
    BadRequestException,
    Controller,
    Delete,
    Get,
    HttpStatusCode,
    HttpException,
    Param,
    Post,
    Query,
    Req,
}

```

```

    UseGuards,
  } from '@nestjs/common';
import { FavoritesService } from '../favorites.service';
import { AuthGuard } from 'src/helpers/auth.guard';
import { Request } from 'express';

@Controller('favorites')
export class FavoritesController {
  constructor(private favoritesService: FavoritesService) {}

  @Post('/:type/:id')
  @HttpCode(200)
  @UseGuards(AuthGuard)
  async addFavorite(
    @Param('type') type: 'podcast' | 'book',
    @Param('id') id: number,
    @Req() req: Request,
  ) {
    if (type !== 'book' && type !== 'podcast') {
      throw new BadRequestException();
    }

    const favoriteResult = await
this.favoritesService.addOrRemoveFavorite(
      req.cookies.session,
      id,
      type,
    );

    if (favoriteResult.status !== 200) {
      throw new HttpException(favoriteResult.error,
favoriteResult.status);
    }

    return { message: 'ok' };
  }

  @Delete('/:type/:id')
  @HttpCode(200)
  @UseGuards(AuthGuard)
  async removeFavorite(
    @Param('type') type: 'podcast' | 'book',
    @Param('id') id: number,
    @Req() req: Request,
  ) {
    if (type !== 'book' && type !== 'podcast') {

```

```

        throw new BadRequestException();
    }

    const favoriteResult = await
this.favoritesService.addOrRemoveFavorite(
    req.cookies.session,
    id,
    type,
);

    if (favoriteResult.status !== 200) {
        throw new HttpException(favoriteResult.error,
favoriteResult.status);
    }

    return { message: 'ok' };
}

@Get('/user/:type/:id')
@UseGuards(AuthGuard)
async getList(
    @Param('type') type: 'podcast' | 'book',
    @Param('id') userId: number,
    @Query('page') page: number = 1,
) {
    if (type !== 'book' && type !== 'podcast') {
        throw new BadRequestException();
    }

    const favoriteResult = await this.favoritesService.getList(
        userId,
        type,
        page,
    );

    if (favoriteResult.status !== 200) {
        throw new HttpException(favoriteResult.error,
favoriteResult.status);
    }

    return { favorites: favoriteResult.favorites, count:
favoriteResult.count };
}
}

```

Листинг Б.28 – server/src/favorites/favorites.module.ts

```
import { Module } from '@nestjs/common';
import { FavoritesController } from '../favorites.controller';
import { FavoritesService } from '../favorites.service';
import { SequelizeModule } from '@nestjs/sequelize';
import { BooksFavorites } from '../models/books-favorites.model';
import { PodcastsFavorites } from '../models/podcasts-favorites.model';
import { SessionsModule } from 'src/sessions/sessions.module';
import { BooksModule } from 'src/books/books.module';
import { PodcastsModule } from 'src/podcasts/podcasts.module';

@Module({
  controllers: [FavoritesController],
  providers: [FavoritesService],
  imports: [
    SequelizeModule.forFeature([BooksFavorites,
PodcastsFavorites]),
    SessionsModule,
    BooksModule,
    PodcastsModule
  ],
})
export class FavoritesModule {}
```

Листинг Б.29 – server/src/favorites/favorites.service.ts

```
import { Inject, Injectable } from '@nestjs/common';
import { InjectModel } from '@nestjs/sequelize';
import { PodcastsFavorites } from '../models/podcasts-favorites.model';
import { BooksFavorites } from '../models/books-favorites.model';
import { SessionsService } from 'src/sessions/sessions.service';
import { IFavoritesStatus } from '../dto/favorites-status.dto';
import { BooksService } from 'src/books/books.service';
import { PodcastsService } from 'src/podcasts/podcasts.service';
import { Books } from 'src/books/models/books.model';
import { Podcasts } from 'src/podcasts/models/podcasts.model';

@Injectable()
export class FavoritesService {
  constructor(
    @InjectModel(PodcastsFavorites)
    private podcastsFavoritesRepository: typeof
PodcastsFavorites,
    @InjectModel(BooksFavorites)
    private booksFavoritesRepository: typeof BooksFavorites,
```

```

    @Inject(SessionsService) private sessionsService:
SessionsService,
    @Inject(BooksService) private booksService: BooksService,
    @Inject(PodcastsService) private podcastsService:
PodcastsService,
  ) {}

  async getOneWhere(
    entityId: number,
    userId: number,
    type: 'podcast' | 'book',
  ): Promise<IFavoritesStatus> {
    try {
      const favs =
        type === 'book'
          ? await this.booksFavoritesRepository.findOne({
              where: { book_id: entityId, user_id: userId },
              raw: true,
            })
          : await this.podcastsFavoritesRepository.findOne({
              where: { podcast_id: entityId, user_id: userId },
              raw: true,
            });
      if (!favs) {
        return { status: 404 };
      }
      return { status: 200, favorite: favs };
    } catch (e) {
      return { status: 500 };
    }
  }

  async getList(
    userId: number,
    type: 'podcast' | 'book',
    page: number,
  ): Promise<IFavoritesStatus> {
    try {
      const favs =
        type === 'book'
          ? await this.booksFavoritesRepository.findAll({
              where: { user_id: userId },
              include: [
                {
                  model: Books,
                  include: await

```

```

this.booksService.commonInclude(null),
    attributes: ['name', 'cover', 'code'],
  },
],
  attributes: { exclude: ['createdAt', 'updatedAt']
},
    limit: 10,
    offset: (page - 1) * 10,
  })
: await this.podcastsFavoritesRepository.findAll({
  where: { user_id: userId },
  include: [
    {
      model: Podcasts,
      attributes: ['name', 'cover', 'code'],
      include: await
this.podcastsService.commonInclude(null),
    },
  ],
  attributes: { exclude: ['createdAt', 'updatedAt']
},
    limit: 10,
    offset: (page - 1) * 10,
  });

const count =
  type == 'book'
    ? await this.booksFavoritesRepository.count({
      where: { user_id: userId },
    })
    : await this.podcastsFavoritesRepository.count({
      where: { user_id: userId },
    });
if (favs.length === 0) {
  return { status: 404 };
}
return { count, status: 200, favorites: favs };
} catch (e) {
  return { status: 500 };
}
}

async addOrRemoveFavorite(
  sessionCode: string,
  entityId: number,
  type: 'podcast' | 'book',

```



```

): Promise<IFavoritesStatus> {
  const session =
    await
this.sessionsService.getActualSessionByCode(sessionCode);

  const existingFavs = await this.getOneWhere(
    entityId,
    session.user_id,
    type,
  );

  if (existingFavs.status === 200) {
    type === 'book'
      ? await this.booksFavoritesRepository.destroy({
        where: { book_id: entityId, user_id: session.user_id
},
        })
      : await this.podcastsFavoritesRepository.destroy({
        where: { podcast_id: entityId, user_id:
session.user_id },
        });
    return { status: 200 };
  }

  type === 'book'
    ? await this.booksFavoritesRepository.create({
      book_id: entityId,
      user_id: session.user_id,
    })
    : await this.podcastsFavoritesRepository.create({
      podcast_id: entityId,
      user_id: session.user_id,
    });
  return { status: 200 };
}
}

```

Листинг Б.30 – server/src/favorites/dto/favorites-status.dto.ts

```

import { IServiceAnswer } from 'src/helpers/service-
answer.interface';
export interface IFavoritesStatus extends IServiceAnswer {
  favorite?: any;
  favorites?: any[];
  count?: number;
}

```

Листинг Б.31 – server/src/favorites/models/books-favorites.model.ts

```
import {
  BelongsTo,
  Column,
  DataType,
  ForeignKey,
  Model,
  Table,
} from 'sequelize-typescript';
import { Books } from 'src/books/models/books.model';
import { Users } from 'src/users/models/users.model';

@Table({ tableName: 'books_favorites' })
export class BooksFavorites extends Model<BooksFavorites> {
  @Column({
    type: DataType.INTEGER,
    unique: true,
    autoIncrement: true,
    primaryKey: true,
  })
  id: number;

  @ForeignKey(() => Users)
  @Column({
    type: DataType.INTEGER,
  })
  user_id: number;

  @ForeignKey(() => Books)
  @Column({
    type: DataType.INTEGER,
  })
  book_id: number;

  @BelongsTo(() => Books)
  book: Books;

  @BelongsTo(() => Users)
  user: Users;
}
```

Листинг Б.32 – server/src/favorites/models/podcasts-favorites.model.ts

```
import {
  BelongsTo,
  Column,
  DataType,
```

```

    ForeignKey,
    Model,
    Table,
} from 'sequelize-typescript';
import { Podcasts } from 'src/podcasts/models/podcasts.model';
import { Users } from 'src/users/models/users.model';

@Table({ tableName: 'podcasts_favorites' })
export class PodcastsFavorites extends Model<PodcastsFavorites>
{
    @Column({
        type: DataType.INTEGER,
        unique: true,
        autoIncrement: true,
        primaryKey: true,
    })
    id: number;

    @ForeignKey(() => Users)
    @Column({
        type: DataType.INTEGER,
    })
    user_id: number;

    @ForeignKey(() => Podcasts)
    @Column({
        type: DataType.INTEGER,
    })
    podcast_id: number;

    @BelongsTo(() => Podcasts)
    podcast: Podcasts;

    @BelongsTo(() => Users)
    user: Users;
}

```

Листинг Б.33 – server/src/filesystem/filesystem.controller.ts

```

import {
    BadRequestException,
    Controller,
    Get,
    HttpException,
    Inject,
    NotFoundException,
    Param,
}

```

```

    Req,
    Res,
    UseGuards,
  } from '@nestjs/common';
import { Request, Response } from 'express';
import * as fs from 'fs';
import { Stats } from 'fs';
import { BookspartsService } from
'src/booksparts/booksparts.service';
import { AuthGuard } from 'src/helpers/auth.guard';
import { HistoryService } from 'src/history/history.service';
import { PodcastsepisodesService } from
'src/podcastsepisodes/podcastsepisodes.service';

@Controller('filesystem')
export class FilesystemController {
  constructor(
    @Inject(BookspartsService) private partsService:
BookspartsService,
    @Inject(PodcastsepisodesService)
    private episodesService: PodcastsepisodesService,
    @Inject(HistoryService) private historyService:
HistoryService,
  ) {}
  @Get('/stream/:type/:code')
  @UseGuards(AuthGuard)
  async streamBook(
    @Param('code') code: string,
    @Param('type') type: 'podcast' | 'book',
    @Req() req: Request,
    @Res() res: Response,
  ) {
    let file;
    if (type == 'book') {
      const bookpartResult = await
this.partsService.getOneWhere({ code }, [
        'filepath',
        'book_id',
      ]);
      if (bookpartResult.status !== 200) {
        throw new HttpException(bookpartResult.error,
bookpartResult.status);
      }
      this.historyService.addToHistoryExternalModules(
        req.cookies.session,
        'book',

```

```

        bookpartResult.part.book_id,
    );
    file = process.cwd() + bookpartResult.part.filepath;
} else if (type == 'podcast') {
    const podcastResult = await
this.episodesService.getOneWhere({ code }, [
        'filepath',
        'podcast_id',
    ]);
    if (podcastResult.status !== 200) {
        throw new HttpException(podcastResult.error,
podcastResult.status);
    }
    this.historyService.addToHistoryExternalModules(
        req.cookies.session,
        'podcast',
        podcastResult.episode.podcast_id,
    );
    file = process.cwd() + podcastResult.episode.filepath;
} else {
    throw new BadRequestException();
}

let stat: Stats;

try {
    stat = fs.statSync(file);
} catch (e) {
    throw new NotFoundException();
}

const range = req.headers['range'];
let readStream: fs.ReadStream;

if (range) {
    const parts = range.replace(/bytes=/, '').split('-');
    const partialStart = parts[0];
    const partialEnd = parts[1];

    const start = parseInt(partialStart, 10);
    const end = partialEnd ? parseInt(partialEnd, 10) :
stat.size - 1;
    const contentLength = end - start + 1;

    res.status(206).header({
        'Content-Type': 'audio/mpeg',

```

```

        'Content-Length': contentLength,
        'Content-Range': 'bytes ' + start + '-' + end + '/' +
stat.size,
    });

    readStream = fs.createReadStream(file, { start, end });
  } else {
    res.header({ 'Content-Type': 'audio/mpeg', 'Content-
Length': stat.size });
    readStream = fs.createReadStream(file);
  }

  readStream.pipe(res);
}
}

```

Листинг Б.34 – server/src/filesystem/filesystem.module.ts

```

import { Module } from '@nestjs/common';
import { FileSystemController } from '../filesystem.controller';
import { FileSystemService } from '../filesystem.service';
import { BookspartsModule } from
'src/booksparts/booksparts.module';
import { PodcastsepisodesModule } from
'src/podcastsepisodes/podcastsepisodes.module';
import { SessionsModule } from 'src/sessions/sessions.module';
import { PodcastsModule } from 'src/podcasts/podcasts.module';
import { BooksModule } from 'src/books/books.module';
import { HistoryModule } from 'src/history/history.module';

@Module({
  controllers: [FileSystemController],
  providers: [FileSystemService],
  imports: [
    BookspartsModule,
    PodcastsepisodesModule,
    SessionsModule,
    PodcastsModule,
    BooksModule,
    HistoryModule,
  ],
})
export class FileSystemModule {}

```

Листинг Б.35 – server/src/filesystem/filesystem.service.ts

```
import { Injectable } from '@nestjs/common';

@Injectable()
export class FilesystemService {}
```

Листинг Б.36 – server/src/helpers/auth.guard.ts

```
import {
  CanActivate,
  ExecutionContext,
  Inject,
  Injectable,
} from '@nestjs/common';
import { Observable } from 'rxjs';
import { SessionsService } from 'src/sessions/sessions.service';

@Injectable()
export class AuthGuard implements CanActivate {
  constructor(
    @Inject(SessionsService) private sessionsService:
SessionsService,
  ) {}

  canActivate(
    context: ExecutionContext,
  ): boolean | Promise<boolean> | Observable<boolean> {
    const request = context.switchToHttp().getRequest();
    const sessionCode = request.cookies?.session;

    if (!sessionCode) {
      return false;
    }

    const session =
this.sessionsService.getActualSessionByCode(sessionCode);
    if (session) {
      return true;
    } else {
      return false;
    }
  }
}
```

Листинг Б.37 – server/src/helpers/not_auth.guard.ts

```
import {
  CanActivate,
```

```

    ExecutionContext,
    Inject,
    Injectable,
} from '@nestjs/common';
import { Observable } from 'rxjs';
import { SessionsService } from 'src/sessions/sessions.service';

@Injectable()
export class NoAuthGuard implements CanActivate {
  constructor(
    @Inject(SessionsService) private sessionService:
SessionsService,
  ) {}

  canActivate(
    context: ExecutionContext,
  ): boolean | Promise<boolean> | Observable<boolean> {
    const request = context.switchToHttp().getRequest();
    const sessionCode = request.cookies?.session;

    if (!sessionCode) {
      return true;
    }

    const session =
this.sessionService.getActualSessionByCode(sessionCode);
    if (!session) {
      return true;
    } else {
      return false;
    }
  }
}

```

Листинг Б.38 – server/src/helpers/service-answer.interface.ts

```

export interface IServiceAnswer {
  error?: string;
  status?: number;
}

```

Листинг Б.39 – server/src/history/history.controller.ts

```

import {
  Controller,
  Get,
  HttpException,

```



```

    Inject,
    Query,
    Req,
    UseGuards,
  } from '@nestjs/common';
import { InjectModel } from '@nestjs/sequelize';
import { BooksHistory } from '../models/books-history.model';
import { PodcastsHistory } from '../models/podcasts-
history.model';
import { AuthGuard } from 'src/helpers/auth.guard';
import { Request } from 'express';
import { HistoryService } from '../history.service';

@Controller('history')
export class HistoryController {
  constructor(
    @InjectModel(BooksHistory)
    private booksHistoryRepository: typeof BooksHistory,
    @InjectModel(PodcastsHistory)
    private podcastsHistoryRepository: typeof PodcastsHistory,
    private historyService: HistoryService,
  ) {}

  @Get('/books')
  @UseGuards(AuthGuard)
  async getBooksHistory(@Req() req: Request, @Query('page')
page: number = 1) {
    const historyResult = await
this.historyService.getMyHistory(
      req.cookies.session,
      this.booksHistoryRepository,
      'book',
      page,
    );

    if (historyResult.status !== 200) {
      throw new HttpException(historyResult.error,
historyResult.status);
    }

    return { history: historyResult.history, count:
historyResult.count };
  }

  @Get('/podcasts')
  @UseGuards(AuthGuard)

```

```

    async getPodcastsHistory(
      @Req() req: Request,
      @Query('page') page: number = 1,
    ) {
      const historyResult = await
this.historyService.getMyHistory(
        req.cookies.session,
        this.podcastsHistoryRepository,
        'podcast',
        page,
      );

      if (historyResult.status !== 200) {
        throw new HttpException(historyResult.error,
historyResult.status);
      }

      return { history: historyResult.history, count:
historyResult.count };
    }
  }
}

```

Листинг Б.40 – server/src/history/history.module.ts

```

import { Module } from '@nestjs/common';
import { HistoryService } from '../history.service';
import { SessionsModule } from 'src/sessions/sessions.module';
import { SequelizeModule } from '@nestjs/sequelize';
import { BooksHistory } from '../models/books-history.model';
import { PodcastsHistory } from '../models/podcasts-
history.model';
import { HistoryController } from '../history.controller';

@Module({
  providers: [HistoryService],
  exports: [HistoryService],
  imports: [
    SequelizeModule.forFeature([BooksHistory, PodcastsHistory]),
    SessionsModule,
  ],
  controllers: [HistoryController],
})
export class HistoryModule {}

```

Листинг Б.41 – server/src/history/history.service.ts

```
import { Inject, Injectable } from '@nestjs/common';
import { Authors } from 'src/authors/models/authors.model';
import { Books } from 'src/books/models/books.model';
import { Podcasts } from 'src/podcasts/models/podcasts.model';
import { SessionsService } from 'src/sessions/sessions.service';
import { IHistoryStatus } from '../dto/history-status.dto';
import { BooksHistory } from '../models/books-history.model';
import { PodcastsHistory } from '../models/podcasts-
history.model';
import { InjectModel } from '@nestjs/sequelize';

@Injectable()
export class HistoryService {
  constructor(
    @Inject(SessionsService) private sessionsService:
SessionsService,
    @InjectModel(BooksHistory)
    private booksHistoryRepository: typeof BooksHistory,
    @InjectModel(PodcastsHistory)
    private podastsHistoryRepository: typeof PodcastsHistory,
  ) {}

  async addToHistoryExternalModules(
    sessionCode: string,
    type: 'podcast' | 'book',
    entityId: number,
  ) {
    const repository =
      type === 'book'
        ? this.booksHistoryRepository
        : this.podastsHistoryRepository;

    await this.addToHistory(sessionCode, repository, type,
entityId);
  }

  async getMyHistory(
    sessionCode: string,
    historyRepository: any,
    type: 'podcast' | 'book',
    page: number,
  ): Promise<IHistoryStatus> {
    try {
      const session =
        await
```

```

this.sessionsService.getActualSessionByCode(sessionCode);

const include =
  type === 'book'
    ? [
      {
        model: Books,
        attributes: ['name', 'code', 'cover'],
        include: [{ model: Authors, attributes: ['name']
}],
    },
    ]
    : [
      {
        model: Podcasts,
        attributes: ['name', 'code', 'cover'],
        include: [{ model: Authors, attributes: ['name']
}],
    },
    ];

const attributesExclude =
  type === 'book'
    ? ['createdAt', 'updatedAt', 'id', 'book_id',
'user_id']
    : ['createdAt', 'updatedAt', 'id', 'podcast_id',
'user_id'];

const history = await historyRepository.findAll({
  order: [['updatedAt', 'DESC']],
  where: { user_id: session.user_id },
  include,
  attributes: {
    exclude: attributesExclude,
  },
  limit: 10,
  offset: (page - 1) * 10,
});

const count = await historyRepository.count({
  where: { user_id: session.user_id },
});

return { count, history, status: 200 };
} catch (e) {
return { status: 500 };
}

```

```

    }
}

async addToHistory(
  sessionCode: string,
  historyRepository: any,
  type: 'podcast' | 'book',
  entityId: number,
): Promise<IHistoryStatus> {
  try {
    const session =
      await
this.sessionsService.getActualSessionByCode(sessionCode);
    let status = 200;

    const where =
      type === 'book'
        ? { user_id: session.user_id, book_id: entityId }
        : { user_id: session.user_id, podcast_id: entityId };

    const createObject =
      type === 'book'
        ? { user_id: session.user_id, book_id: entityId }
        : { user_id: session.user_id, podcast_id: entityId };

    const historyRow = await historyRepository.findOne({
      where,
      raw: true,
    });

    if (!historyRow) {
      await historyRepository.create(createObject);
    } else {
      await historyRepository.update(createObject, { where });
      status = 409;
    }

    return { status };
  } catch (e) {
    return { status: 500 };
  }
}

async getOneWhere(
  user_id: number,
  entityId: number,

```

```

    type: 'book' | 'podcast',
  ): Promise<IHistoryStatus> {
    try {
      const history =
        type === 'book'
          ? await this.booksHistoryRepository.findOne({
              where: { user_id, book_id: entityId },
              raw: true,
            })
          : await this.podcastsHistoryRepository.findOne({
              where: { user_id, podcast_id: entityId },
              raw: true,
            });
      if (!history) {
        return { status: 404 };
      }
      return { oneItem: history, status: 200 };
    } catch (e) {
      return { status: 500 };
    }
  }
}

```

Листинг Б.42 – server/src/history/dto/history-status.dto.ts

```

import { IServiceAnswer } from 'src/helpers/service-
answer.interface';

export interface IHistoryStatus extends IServiceAnswer {
  history?: any[];
  oneItem?: any;
  count?: number;
}

```

Листинг Б.43 – server/src/history/models/books-history.model.ts

```

import {
  BelongsTo,
  Column,
  DataType,
  ForeignKey,
  Model,
  Table,
} from 'sequelize-typescript';
import { Users } from 'src/users/models/users.model';
import { Books } from '../../../books/models/books.model';

```

```

@Table({ tableName: 'books_history' })
export class BooksHistory extends Model<BooksHistory> {
  @Column({
    type: DataType.INTEGER,
    unique: true,
    autoIncrement: true,
    primaryKey: true,
  })
  id: number;

  @ForeignKey(() => Users)
  @Column({
    type: DataType.INTEGER,
  })
  user_id: number;

  @ForeignKey(() => Books)
  @Column({
    type: DataType.INTEGER,
  })
  book_id: number;

  @BelongsTo(() => Books)
  book: Books;
}

```

Листинг Б.44 – server/src/history/models/podcasts-history.model.ts

```

import {
  BelongsTo,
  Column,
  DataType,
  ForeignKey,
  HasOne,
  Model,
  Table,
} from 'sequelize-typescript';
import { Users } from 'src/users/models/users.model';
import { Podcasts } from '../../podcasts/models/podcasts.model';

@Table({ tableName: 'podcasts_history' })
export class PodcastsHistory extends Model<PodcastsHistory> {
  @Column({
    type: DataType.INTEGER,
    unique: true,
    autoIncrement: true,
    primaryKey: true,
  })

```

```

    })
    id: number;

    @ForeignKey(() => Users)
    @Column({
        type: DataType.INTEGER,
    })
    user_id: number;

    @ForeignKey(() => Podcasts)
    @Column({
        type: DataType.INTEGER,
    })
    podcast_id: number;

    @BelongsTo(() => Podcasts)
    podcast: Podcasts;
}

```

Листинг Б.45 – server/src/podcasts/podcasts.controller.ts

```

import {
    Controller,
    Get,
    HttpException,
    Param,
    Post,
    Query,
    Req,
    UseGuards,
} from '@nestjs/common';
import { Request } from 'express';
import { PodcastsService } from '../podcasts.service';
import { AuthGuard } from 'src/helpers/auth.guard';

@Controller('podcasts')
export class PodcastsController {
    constructor(private podcastsService: PodcastsService) {}

    @Get('/one/:code')
    async getOne(@Req() req: Request, @Param('code') code: string)
    {
        const podcastResult = await
this.podcastsService.getPodcastWhere({
            code,
        }, req.cookies.session);
    }
}

```



```

        if (podcastResult.status !== 200) {
            throw new HttpException(podcastResult.error,
podcastResult.status);
        }
        await this.podcastsService.addView(
            req.cookies.session,
            podcastResult.podcast.id,
        );

        return podcastResult.podcast;
    }

    @Get('/search')
    async searchPodcast(
        @Query('searchQuery') searchQuery: string,
        @Query('page') page: number,
        @Req() req: Request,
    ) {
        const podcastsResult = await
this.podcastsService.searchPodcast(
            page,
            searchQuery,
            req.cookies.session,
        );
        if (podcastsResult.status !== 200) {
            throw new HttpException(podcastsResult.error,
podcastsResult.status);
        }

        return podcastsResult.podcasts;
    }

    @Get('/list')
    async getList(@Query('page') page: number = 1, @Req() req:
Request) {
        const podcastsResult = await
this.podcastsService.getSortedPodcastsList(
            page,
            req.cookies.session,
        );
        if (podcastsResult.status !== 200) {
            throw new HttpException(podcastsResult.error,
podcastsResult.status);
        }

        return { podcasts: podcastsResult.podcasts, count:

```

```

podcastsResult.count };
    }

    @Get('/trending')
    async getPopularPodcasts(@Req() req: Request) {
        const podcastsResult = await
this.podcastsService.getPopularPodcastsList(
        req.cookies.session,
    );
        if (podcastsResult.status !== 200) {
            throw new HttpException(podcastsResult.error,
podcastsResult.status);
        }

        return podcastsResult.podcasts;
    }
}

```

Листинг Б.46 – server/src/podcasts/podcasts.module.ts

```

import { Module } from '@nestjs/common';
import { SequelizeModule } from '@nestjs/sequelize';
import { SessionsModule } from 'src/sessions/sessions.module';
import { PodcastsSubs } from './models/podcasts-sub.model';
import { PodcastsViews } from './models/podcasts-views.model';
import { Podcasts } from './models/podcasts.model';
import { PodcastsController } from './podcasts.controller';
import { PodcastsService } from './podcasts.service';

@Module({
    controllers: [PodcastsController],
    providers: [PodcastsService],
    imports: [
        SequelizeModule.forFeature([Podcasts, PodcastsSubs,
PodcastsViews]),
        SessionsModule,
    ],
    exports: [PodcastsService],
})
export class PodcastsModule {}

```

Листинг Б.47 – server/src/podcasts/podcasts.service.ts

```

import { Inject, Injectable } from '@nestjs/common';
import { InjectModel } from '@nestjs/sequelize';
import { Includeable, Op } from 'sequelize';
import { Authors } from 'src/authors/models/authors.model';

```

```

import { Genres } from 'src/genres/models/genres.model';
import { ISessionStatus } from 'src/sessions/dto/session-
status.dto';
import { SessionsService } from 'src/sessions/sessions.service';
import { IPodcastStatus } from './dto/podcast-status.interface';
import { PodcastsViews } from './models/podcasts-views.model';
import { Podcasts } from './models/podcasts.model';
import { PodcastsReviews } from 'src/reviews/model/podcasts-
reviews.model';
import { Users } from 'src/users/models/users.model';
import { PodcastsFavorites } from
'src/favorites/models/podcasts-favorites.model';

@Injectable()
export class PodcastsService {
  constructor(
    @InjectModel(Podcasts) private podcastsRepository: typeof
Podcasts,
    @InjectModel(PodcastsViews) private viewsRepository: typeof
PodcastsViews,
    @Inject(SessionsService) private sessionsService:
SessionsService,
  ) {}

  async commonInclude(
    sessionCode: string | null,
  ): Promise<Includeable[]> {
    let userId = null;
    if (sessionCode) {
      userId = (await
this.sessionsService.getActualSessionByCode(sessionCode))
        .user_id;
    }
    return [
      { model: Authors, attributes: ['name', 'code'] },
      {
        model: PodcastsFavorites,
        attributes: ['user_id', 'podcast_id'],
        where: { user_id: userId },
        include: [{ model: Users, attributes: ['username'] }],
        required: false,
      },
    ],
  ];
}

  async getPodcastWhere(

```

```

    where: any,
    sessionCode?: string | null,
  ): Promise<IPodcastStatus> {
    try {
      let userId = null;
      if (sessionCode) {
        userId = (
          await
this.sessionsService.getActualSessionByCode(sessionCode)
        ).user_id;
      }
      const podcast = await this.podcastsRepository.findOne({
        where,
        include: [
          { model: Authors, attributes: ['name', 'code'] },
          {
            model: Genres,
            attributes: { exclude: ['createdAt', 'updatedAt'] },
          },
          {
            model: PodcastsReviews,
            include: [{ model: Users, attributes: ['username']
}],
            attributes: ['id', 'text', 'rating', 'createdAt'],
          },
          {
            model: PodcastsFavorites,
            attributes: ['user_id', 'podcast_id'],
            where: { user_id: userId },
            include: [{ model: Users, attributes: ['username']
}],
            required: false,
          },
        ],
        attributes: {
          exclude: ['createdAt', 'updatedAt'],
        },
      });
      if (!podcast) {
        return { status: 404, error: 'Подкаст не найден' };
      }
      if (podcast.reviews.length > 0) {
        const totalRating = podcast.reviews.reduce(
          (sum, review) => sum + review.rating,
          0,
        );

```

```

        const averageRating = parseFloat(
            (totalRating / podcast.reviews.length).toFixed(2),
        );
        podcast.dataValues.rating = averageRating;
    }
    return { podcast, status: 200 };
} catch (e) {
    return { status: 500 };
}
}

async searchPodcast(
    page: number,
    searchQuery: string,
    sessionCode?: string | null,
): Promise<IPodcastStatus> {
    let whereCondition = searchQuery
        ? {
            [Op.or]: {
                name: { [Op.substring]: searchQuery },
                description: { [Op.substring]: searchQuery },
            },
        }
        : {};

    try {
        const podcasts = await this.podcastsRepository.findAll({
            include: await this.commonInclude(sessionCode),
            attributes: { exclude: ['author_id', 'createdAt',
'updatedAt'] },
            where: whereCondition,
            order: [['sort', 'ASC']],
            limit: 10,
            offset: (page - 1) * 10,
        });

        if (podcasts.length === 0) {
            return { status: 404, error: 'Нет результатов по такому
запросу' };
        }
        return { podcasts, status: 200 };
    } catch (e) {
        return { status: 500 };
    }
}
}

```

```

async getSortedPodcastsList(
  page: number,
  sessionCode?: string | null,
): Promise<IPodcastStatus> {
  try {
    const podcasts = await this.podcastsRepository.findAll({
      order: [['sort', 'ASC']],
      include: await this.commonInclude(sessionCode),
      attributes: { exclude: ['author_id', 'createdAt',
'updatedAt'] },
      limit: 10,
      offset: (page - 1) * 10,
    });

    const count = await this.podcastsRepository.count();
    return { podcasts, count, status: 200 };
  } catch (e) {
    console.log(e);
    return { status: 500 };
  }
}

async getPopularPodcastsList(
  sessionCode?: string | null,
): Promise<IPodcastStatus> {
  try {
    const podcasts = await this.podcastsRepository.findAll({
      order: [['trending', 'ASC']],
      include: await this.commonInclude(sessionCode),
      attributes: { exclude: ['author_id', 'createdAt',
'updatedAt'] },
      limit: 5,
    });

    return { podcasts, status: 200 };
  } catch (e) {
    return { status: 500 };
  }
}

async addView(
  sessionCode: string,
  podcastId: number,
): Promise<ISessionStatus> {
  try {
    if (!sessionCode) {

```

```

        return;
    }
    const session =
        await
this.sessionsService.getActualSessionByCode(sessionCode);

    const now = new Date();

    const dayAgo = new Date(now.getTime() - 24 * 60 * 60 *
1000);

    const isViewExist = await this.viewsRepository.findOne({
        where: {
            user_id: session.user_id,
            createdAt: {
                [Op.gte]: dayAgo,
            },
        },
        raw: true,
    });

    if (isViewExist) {
        return;
    }

    await this.viewsRepository.create({
        user_id: session.user_id,
        podcast_id: podcastId,
    });

    const podcast = await this.podcastsRepository.findOne({
        where: { id: podcastId },
    });
    await this.podcastsRepository.update(
        { views: podcast.views ? podcast.views + 1 : 1 },
        { where: { id: podcastId } },
    );

    return { status: 200 };
} catch (e) {
    return { status: 500 };
}
}
}

```

Листинг Б.48 – server/src/podcasts/dto/podcast-status.interface.ts

```
import { IServiceAnswer } from 'src/helpers/service-  
answer.interface';  
import { Podcasts } from '../models/podcasts.model';  
import { PodcastsHistory } from '../../history/models/podcasts-  
history.model';  
  
export interface IPodcastStatus extends IServiceAnswer {  
  podcast?: Podcasts;  
  podcasts?: Podcasts[];  
  history?: PodcastsHistory[];  
  count?: number;  
}
```

Листинг Б.49 – server/src/podcasts/models/podcasts-subscribers.model.ts

```
import {  
  Column,  
  DataType,  
  ForeignKey,  
  Model,  
  Table,  
} from 'sequelize-typescript';  
import { Users } from 'src/users/models/users.model';  
import { Podcasts } from './podcasts.model';  
  
@Table({ tableName: 'podcasts_subscribers' })  
export class PodcastsSubs extends Model<PodcastsSubs> {  
  @Column({  
    type: DataType.INTEGER,  
    unique: true,  
    autoIncrement: true,  
    primaryKey: true,  
  })  
  id: number;  
  @ForeignKey(() => Users)  
  @Column({  
    type: DataType.INTEGER,  
  })  
  user_id: number;  
  @ForeignKey(() => Podcasts)  
  @Column({  
    type: DataType.INTEGER,  
  })  
  podcast_id: number;  
}
```


Листинг Б.50 – server/src/podcasts/models/podcasts-views.model.ts

```
import {
  Column,
  DataType,
  ForeignKey,
  Model,
  Table,
} from 'sequelize-typescript';
import { Users } from 'src/users/models/users.model';
import { Podcasts } from '../podcasts.model';

@Table({ tableName: 'podcasts_views' })
export class PodcastsViews extends Model<PodcastsViews> {
  @Column({
    type: DataType.INTEGER,
    unique: true,
    autoIncrement: true,
    primaryKey: true,
  })
  id: number;

  @ForeignKey(() => Users)
  @Column({
    type: DataType.INTEGER,
  })
  user_id: number;

  @ForeignKey(() => Podcasts)
  @Column({
    type: DataType.INTEGER,
  })
  podcast_id: number;
}
```

Листинг Б.51 – server/src/podcasts/models/podcasts.model.ts

```
import {
  BelongsTo,
  BelongsToMany,
  Column,
  DataType,
  ForeignKey,
  HasMany,
  Model,
  Table,
} from 'sequelize-typescript';
import { Authors } from 'src/authors/models/authors.model';
```

```

import { Books } from 'src/books/models/books.model';
import { Users } from 'src/users/models/users.model';
import { PodcastsSubs } from './podcasts-sub.model';
import { Genres } from 'src/genres/models/genres.model';
import { PodcastsGenres } from 'src/genres/models/podcasts-
genres.model';
import { PodcastsEpisodes } from
'src/podcastsepisodes/model/podcasts-episodes.model';
import { PodcastsHistory } from '../../history/models/podcasts-
history.model';
import { PodcastsReviews } from 'src/reviews/model/podcasts-
reviews.model';
import { PodcastsFavorites } from
'src/favorites/models/podcasts-favorites.model';

@Table({ tableName: 'podcasts' })
export class Podcasts extends Model<Podcasts> {
  @Column({
    type: DataType.INTEGER,
    unique: true,
    autoIncrement: true,
    primaryKey: true,
  })
  id: number;

  @Column({
    type: DataType.STRING,
  })
  name: string;

  @Column({
    type: DataType.STRING,
    unique: true,
  })
  code: string;

  @Column({
    type: DataType.INTEGER,
  })
  sort: number;

  @Column({
    type: DataType.INTEGER,
  })
  trending: number;

```

```

@Column({
  type: DataType.STRING,
})
cover: string;

@Column({
  type: DataType.TEXT,
})
description: string;

@Column({
  type: DataType.FLOAT,
})
rating: number;

@Column({
  type: DataType.INTEGER,
})
min_age: number;

@Column({
  type: DataType.BOOLEAN,
})
exclusive: boolean;

@Column({
  type: DataType.FLOAT,
})
unsub_price: number;

@Column({
  type: DataType.INTEGER,
})
views: number;

@ForeignKey(() => Authors)
@Column({
  type: DataType.INTEGER,
})
author_id: number;

@BelongsTo(() => Authors)
author: Authors;

@BelongsToMany(() => Users, () => PodcastsSubs)
subs: Users[];

```

```

@BelongsToMany(() => Genres, () => PodcastsGenres)
genres: Genres[];

@HasMany(() => PodcastsEpisodes)
episodes: PodcastsEpisodes[];

@HasMany(() => PodcastsHistory)
history: PodcastsHistory[];

@HasMany(() => PodcastsReviews)
reviews: PodcastsReviews[];

@HasMany(() => PodcastsFavorites)
favorites: PodcastsFavorites[];
}

```

Листинг Б.52 – server/src/reviews/reviews.controller.ts

```

import {
  BadRequestException,
  Body,
  Controller,
  Get,
  HttpException,
  Param,
  Post,
  Query,
  Req,
  UseGuards,
} from '@nestjs/common';
import { ReviewsService } from '../reviews.service';
import { AuthGuard } from 'src/helpers/auth.guard';
import { Request } from 'express';
import { CreateReviewDto } from '../dto/create-review.dto';

@Controller('reviews')
export class ReviewsController {
  constructor(private reviewsService: ReviewsService) {}

  @Get('/:type/isavailable/:entityId')
  @UseGuards(AuthGuard)
  async isAvailable(
    @Req() req: Request,
    @Param('entityId') entityId: number,
    @Param('type') type: 'podcast' | 'book',
  ) {

```

```

        if (type !== 'book' && type !== 'podcast') {
            throw new BadRequestException();
        }
        const reviewResult = await
this.reviewsService.canUserAddReview(
            type,
            req.cookies.session,
            entityId,
        );

        if (reviewResult.status !== 200) {
            throw new HttpException(reviewResult.error,
reviewResult.status);
        }

        return { status: 200 };
    }

    @Post('/:type/add')
    @UseGuards(AuthGuard)
    async addReview(
        @Req() req: Request,
        @Body() dto: CreateReviewDto,
        @Param('type') type: 'podcast' | 'book',
    ) {
        if (type !== 'book' && type !== 'podcast') {
            throw new BadRequestException();
        }

        const reviewResult = await this.reviewsService.addReview(
            req.cookies.session,
            type,
            dto,
        );

        if (reviewResult.status !== 200) {
            throw new HttpException(reviewResult.error,
reviewResult.status);
        }

        return { review: reviewResult.review };
    }

    @Get('/:type/:entityId')
    async getReviews(
        @Param('entityId') entityId: number,

```

```

    @Param('type') type: 'podcast' | 'book',
    @Query('page') page: number = 1,
  ) {
    if (type !== 'book' && type !== 'podcast') {
      throw new BadRequestException();
    }

    const reviewResult = await this.reviewsService.getReviews(
      entityId,
      type,
      page,
    );

    if (reviewResult.status !== 200) {
      throw new HttpException(reviewResult.error,
reviewResult.status);
    }

    return { reviews: reviewResult.reviews, count:
reviewResult.count };
  }
}

```

Листинг Б.53 – server/src/reviews/reviews.module.ts

```

import { Module } from '@nestjs/common';
import { ReviewsController } from '../reviews.controller';
import { ReviewsService } from '../reviews.service';
import { SequelizeModule } from '@nestjs/sequelize';
import { PodcastsReviews } from '../model/podcasts-
reviews.model';
import { BooksReviews } from '../model/books-reviews.model';
import { HistoryModule } from 'src/history/history.module';
import { SessionsModule } from 'src/sessions/sessions.module';

@Module({
  controllers: [ReviewsController],
  providers: [ReviewsService],
  imports: [
    SequelizeModule.forFeature([PodcastsReviews, BooksReviews]),
    HistoryModule,
    SessionsModule,
  ],
})
export class ReviewsModule {}

```

Листинг Б.54 – server/src/reviews/reviews.service.ts

```
import { Inject, Injectable } from '@nestjs/common';
import { InjectModel } from '@nestjs/sequelize';
import { PodcastsReviews } from '../model/podcasts-reviews.model';
import { BooksReviews } from '../model/books-reviews.model';
import { SessionsService } from 'src/sessions/sessions.service';
import { HistoryService } from 'src/history/history.service';
import { IReviewsStatus } from '../dto/reviews-status.dto';
import { CreateReviewDto } from '../dto/create-review.dto';

@Injectable()
export class ReviewsService {
  constructor(
    @InjectModel(PodcastsReviews)
    private podcastsReviewsRepository: typeof PodcastsReviews,
    @InjectModel(BooksReviews)
    private booksReviewsRepository: typeof BooksReviews,
    @Inject(SessionsService) private sessionsService:
SessionsService,
    @Inject(HistoryService) private historyService:
HistoryService,
  ) {}

  async canUserAddReview(
    type: 'podcast' | 'book',
    sessionCode: string,
    entityId: number,
  ): Promise<IReviewsStatus> {
    const session =
      await
this.sessionsService.getActualSessionByCode(sessionCode);

    const historyResult = await this.historyService.getOneWhere(
      session.user_id,
      entityId,
      type,
    );

    if (historyResult.status !== 200) {
      return {
        status: 403,
        error: 'Невозможно оставить отзыв к этому элементу',
      };
    }
  }
}
```

```

    return { status: 200 };
}

async getReviews(
  entityId: number,
  type: 'book' | 'podcast',
  page: number,
): Promise<IReviewsStatus> {
  try {
    const reviews =
      type === 'book'
        ? await this.booksReviewsRepository.findAll({
            where: { book_id: entityId },
            limit: 10,
            offset: (page - 1) * 10,
            raw: true,
          })
        : await this.podcastsReviewsRepository.findAll({
            where: { podcast_id: entityId },
            limit: 10,
            offset: (page - 1) * 10,
            raw: true,
          });

    const count =
      type === 'book'
        ? await this.booksReviewsRepository.count({
            where: { book_id: entityId },
          })
        : await this.podcastsReviewsRepository.count({
            where: { podcast_id: entityId },
          });

    return { status: 200, reviews, count };
  } catch (e) {
    return { status: 500 };
  }
}

async addReview(
  sessionCode: string,
  type: 'book' | 'podcast',
  dto: CreateReviewDto,
): Promise<IReviewsStatus> {
  try {
    const session =

```



```

        await
this.sessionsService.getActualSessionByCode(sessionCode);

        const review =
            type === 'book'
                ? await this.booksReviewsRepository.findOne({
                    where: { user_id: session.user_id, book_id:
dto.entityId },
                    raw: true,
                })
                : await this.podcastsReviewsRepository.findOne({
                    where: { user_id: session.user_id, podcast_id:
dto.entityId },
                    raw: true,
                });
        if (review) {
            return { status: 403, error: 'Отзыв уже добавлен' };
        }

        const newReview =
            type === 'book'
                ? await this.booksReviewsRepository.create({
                    book_id: dto.entityId,
                    user_id: session.user_id,
                    rating: dto.rating,
                    text: dto.text,
                })
                : await this.podcastsReviewsRepository.create({
                    podcast_id: dto.entityId,
                    user_id: session.user_id,
                    rating: dto.rating,
                    text: dto.text,
                });

        return { review: newReview, status: 200 };
    } catch (e) {
        return { status: 500 };
    }
}
}

```

Листинг Б.55 – server/src/reviews/dto/create-review.dto.ts

```

import { IsNumber, IsString, Max, Min } from 'class-validator';

export class CreateReviewDto {
    @IsNumber()

```

```

    entityId: number;
    @IsNumber()
    @Min(1)
    @Max(5)
    rating: number;
    @IsString()
    text: string;
}

```

Листинг Б.56 – server/src/reviews/dto/reviews-status.dto.ts

```

import { IServiceAnswer } from 'src/helpers/service-
answer.interface';

export interface IReviewsStatus extends IServiceAnswer {
    review?: any;
    reviews?: any;
    count?: number;
}

```

Листинг Б.57 – server/src/reviews/model/books-reviews.model.ts

```

import {
    BelongsTo,
    Column,
    DataType,
    ForeignKey,
    Model,
    Table,
} from 'sequelize-typescript';
import { Books } from 'src/books/models/books.model';
import { Users } from 'src/users/models/users.model';

@Table({ tableName: 'books_reviews' })
export class BooksReviews extends Model<BooksReviews> {
    @Column({
        type: DataType.INTEGER,
        unique: true,
        autoIncrement: true,
        primaryKey: true,
    })
    id: number;

    @ForeignKey(() => Users)
    @Column({
        type: DataType.INTEGER,
    })

```

```

    user_id: number;

    @ForeignKey(() => Books)
    @Column({
        type: DataType.INTEGER,
    })
    book_id: number;

    @Column({
        type: DataType.INTEGER,
        allowNull: false,
    })
    rating: number;

    @Column({
        type: DataType.TEXT,
    })
    text: string;

    @BelongsTo(() => Books)
    book: Books;

    @BelongsTo(() => Users)
    user: Users;
}

```

Листинг Б.58 – server/src/reviews/model/podcasts-reviews.model.ts

```

import {
    BelongsTo,
    Column,
    DataType,
    ForeignKey,
    Model,
    Table,
} from 'sequelize-typescript';
import { Books } from 'src/books/models/books.model';
import { Podcasts } from 'src/podcasts/models/podcasts.model';
import { Users } from 'src/users/models/users.model';

@Table({ tableName: 'podcasts_reviews' })
export class PodcastsReviews extends Model<PodcastsReviews> {
    @Column({
        type: DataType.INTEGER,
        unique: true,
        autoIncrement: true,
        primaryKey: true,
    })

```

```

    })
    id: number;

    @ForeignKey(() => Users)
    @Column({
      type: DataType.INTEGER,
    })
    user_id: number;

    @ForeignKey(() => Podcasts)
    @Column({
      type: DataType.INTEGER,
    })
    podcast_id: number;

    @Column({
      type: DataType.INTEGER,
      allowNull: false,
    })
    rating: number;

    @Column({
      type: DataType.TEXT,
    })
    text: string;

    @BelongsTo(() => Podcasts)
    podcast: Podcasts;

    @BelongsTo(() => Users)
    user: Users;
  }

```

Листинг Б.59 – server/src/search/search.controller.ts

```

import { Controller, Get, Query } from '@nestjs/common';
import { SearchService } from '../search.service';

@Controller('search')
export class SearchController {
  constructor(private searchService: SearchService) {}

  @Get()
  searchBooksPodcasts(@Query('query') query: string) {
    return this.searchService.search(query);
  }
}

```

```
}
```

Листинг Б.60 – server/src/search/search.module.ts

```
import { Module } from '@nestjs/common';
import { BooksModule } from 'src/books/books.module';
import { PodcastsModule } from 'src/podcasts/podcasts.module';
import { SearchController } from '../search.controller';
import { SearchService } from '../search.service';

@Module({
  controllers: [SearchController],
  providers: [SearchService],
  imports: [BooksModule, PodcastsModule],
})
export class SearchModule {}
```

Листинг Б.61 – server/src/search/search.service.ts

```
import { Injectable } from '@nestjs/common';
import { BooksService } from 'src/books/books.service';
import { PodcastsService } from 'src/podcasts/podcasts.service';
import { ISearchResult } from '../dto/search-result.dto';

@Injectable()
export class SearchService {
  constructor(
    private booksService: BooksService,
    private podcastsService: PodcastsService,
  ) {}

  async search(query: string): Promise<ISearchResult> {
    const books = (await this.booksService.searchBook(1,
query)).books;
    const podcasts = (await
this.podcastsService.searchPodcast(1, query))
.podcasts;
    return { books, podcasts, status: 200 };
  }
}
```

Листинг Б.62 – server/src/search/dto/search-result.dto.ts

```
import { Books } from 'src/books/models/books.model';
import { IServiceAnswer } from 'src/helpers/service-
answer.interface';
import { Podcasts } from 'src/podcasts/models/podcasts.model';
```

```
export interface ISearchResult extends IServiceAnswer {
  books?: Books[];
  podcasts?: Podcasts[];
}
```

Листинг Б.63 – server/src/sessions/sessions.module.ts

```
import { Module } from '@nestjs/common';
import { SequelizeModule } from '@nestjs/sequelize';
import { UsersModule } from 'src/users/users.module';
import { Sessions } from '../models/sessions.model';
import { SessionsService } from '../sessions.service';

@Module({
  providers: [SessionsService],
  imports: [SequelizeModule.forFeature([Sessions],
UsersModule)],
  exports: [SessionsService]
})
export class SessionsModule {}
```

Листинг Б.64 – server/src/sessions/sessions.service.ts

```
import { Injectable } from '@nestjs/common';
import { InjectModel } from '@nestjs/sequelize';
import { randomBytes } from 'crypto';
import { UsersService } from 'src/users/users.service';
import { Sessions } from '../models/sessions.model';
import { Session } from 'inspector';
import { Op } from 'sequelize';
import { ISessionStatus } from '../dto/session-status.dto';
import { Users } from 'src/users/models/users.model';

@Injectable()
export class SessionsService {
  constructor(
    @InjectModel(Sessions) private sessionRepository: typeof
Sessions,
    private usersService: UsersService,
  ) {}

  async getActualSessionByCode(code: string): Promise<Sessions>
{
  try {
    const session = await this.sessionRepository.findOne({
      where: {
        session_id: code,

```

```

        expires_in: { [Op.gt]: new Date().toISOString() },
    },
    include: [{model: Users, attributes: ['id', 'username',
'avatar']]},
    });
    return session;
} catch (e) {
    return null;
}
}

async createSession(ip: string, username: string):
Promise<ISessionStatus> {
    const sessionCode: string =
randomBytes(12).toString('hex').slice(0, 12);
    let curDate: Date = new Date();
    curDate.setDate(curDate.getDate() + 30);
    let expires_in: string = curDate.toISOString();
    const userResult = await this.usersService.createUser({
username });
    try {
        await this.sessionRepository.create({
            user_id: userResult.user.id,
            expires_in,
            session_id: sessionCode,
            ip_address: ip,
        });
        return { sessionCode, status: 200 };
    } catch (e) {
        return { status: 500 };
    }
}
}

```

Листинг Б.65 – server/src/sessions/dto/session-status.dto.ts

```

import { IServiceAnswer } from 'src/helpers/service-
answer.interface';

export interface ISessionStatus extends IServiceAnswer {
    sessionCode?: string;
}

```

Листинг Б.66 – server/src/sessions/models/sessions.model.ts

```

import { Table, Model, Column, DataType, ForeignKey, BelongsTo }
from "sequelize-typescript";

```

```

import { Users } from "src/users/models/users.model";

@Table({tableName: 'sessions'})
export class Sessions extends Model<Sessions> {

    @Column({
        type: DataType.INTEGER,
        unique: true,
        autoIncrement: true,
        primaryKey: true
    })
    id: number;

    @Column({
        type: DataType.STRING(12),
        unique: true,
    })
    session_id: string;

    @ForeignKey(() => Users)
    @Column({
        type: DataType.INTEGER
    })
    user_id: number;

    @BelongsTo(() => Users)
    users: Users;

    @Column({
        type: DataType.STRING
    })
    ip_address: string;

    @Column({
        type: DataType.DATEONLY
    })
    expires_in: string;

}

```

Листинг Б.67 – server/src/users/users.controller.ts

```

import {
    Body,
    Controller,
    Get,
    HttpStatusCode,

```



```

    HttpException,
    Param,
    Post,
    Res,
} from '@nestjs/common';
import { UsersService } from '../users.service';
import { CreateUserDto } from '../dto/create-user.dto';
import { Users } from '../models/users.model';

@Controller('users')
export class UsersController {
  constructor(private userService: UsersService) {}

  @Get('/user/:username')
  @HttpCode(200)
  async getUserInfo(@Param('username') username: string) {
    const result = await
this.userService.getByUsername(username);
    if (result.status !== 200) {
      throw new HttpException('', result.status);
    }

    return { id: result.user.id, username: result.user.username,
avatar: result.user.avatar };
  }
}

```

Листинг Б.68 – server/src/users/users.module.ts

```

import { Module } from '@nestjs/common';
import { SequelizeModule } from '@nestjs/sequelize';
import { Users } from '../models/users.model';
import { UsersController } from '../users.controller';
import { UsersService } from '../users.service';

@Module({
  controllers: [UsersController],
  providers: [UsersService],
  imports: [SequelizeModule.forFeature([Users])],
  exports: [UsersService]
})
export class UsersModule {}

```

Листинг Б.69 – server/src/users/users.service.ts

```

import { Injectable } from '@nestjs/common';
import { InjectModel } from '@nestjs/sequelize';

```

```

import { Users } from '../models/users.model';
import { CreateUserDto } from '../dto/create-user.dto';
import { UpdateUserDto } from '../dto/update-user.dto';
import { IUserWithStatus } from '../dto/user-with-status.interface';

@Injectable()
export class UsersService {
  constructor(@InjectModel(Users) private usersRepository:
    typeof Users) {}

  async isExists(searchJson): Promise<boolean> {
    try {
      const user = await this.usersRepository.findOne({
        where: searchJson,
        raw: true,
      });
      if (user) {
        return true;
      }
    } catch (e) {
      return false;
    }
  }

  async getById(id: number | string): Promise<IUserWithStatus> {
    try {
      const user = await this.usersRepository.findOne({
        where: { id },
        raw: true,
      });
      return { user, status: 200 };
    } catch (e) {
      return { status: 500 };
    }
  }

  async getByUsername(username: string):
    Promise<IUserWithStatus> {
    try {
      const user = await this.usersRepository.findOne({
        where: { username },
        raw: true,
      });
      if (!user) {
        return { status: 404 };
      }
    }
  }

```

```

        }
        return { user, status: 200 };
    } catch (e) {
        return { status: 500 };
    }
}

async updateUser(dto: UpdateUserDto): Promise<IUserWithStatus>
{
    try {
        await this.usersRepository.update(dto, { where: { id:
dto.id } });
        return { status: 200 };
    } catch (e) {
        return { status: 500 };
    }
}

async createUser(dto: CreateUserDto): Promise<IUserWithStatus>
{
    const existingUser = await this.getByUsername(dto.username);
    if (existingUser.status === 200) {
        return { user: existingUser.user, status: 200 };
    }

    try {
        const newUser = await this.usersRepository.create(dto, {
raw: true });
        return { user: newUser, status: 200 };
    } catch (e) {
        return { status: 500 };
    }
}
}

```

Листинг Б.70 – server/src/users/dto/create-user.dto.ts

```

import { IsEmail, IsOptional, IsString } from "class-validator";

export class CreateUserDto {
    @IsString()
    readonly username: string;

    @IsOptional()
    @IsEmail()
    readonly email?: string;
}

```

```

    @IsOptional()
    readonly avatar?: string;
}

```

Листинг Б.71 – server/src/users/dto/update-user.dto.ts

```

import { IsEmail, IsString } from 'class-validator';

export class UpdateUserDto {
    readonly id: number;
    @IsString()
    readonly username: string;
    @IsEmail()
    readonly email?: string;
    @IsString()
    readonly avatar?: string;
    readonly subscription_id?: number;
    readonly share_id?: string;
}

```

Листинг Б.72 – server/src/users/dto/user-with-status.interface.ts

```

import { IServiceAnswer } from 'src/helpers/service-answer.interface';
import { Users } from '../models/users.model';

export interface IUserWithStatus extends IServiceAnswer {
    user?: Users;
}

```

Листинг Б.73 – server/src/users/models/users.model.ts

```

import {
    Table,
    Model,
    Column,
    DataType,
    HasMany,
    BelongsToMany,
} from 'sequelize-typescript';
import { BooksViews } from 'src/books/models/books-views.model';
import { PodcastsSubs } from 'src/podcasts/models/podcasts-subs.model';
import { PodcastsViews } from 'src/podcasts/models/podcasts-views.model';
import { Podcasts } from 'src/podcasts/models/podcasts.model';
import { Roles } from 'src/roles/models/roles.model';
import { UserRoles } from 'src/roles/models/userroles.model';

```

```

import { Sessions } from 'src/sessions/models/sessions.model';

interface UserCreationAttribute {
  username: string;
  email?: string;
  avatar?: string;
  subscription_id?: number;
}

@Table({ tableName: 'users' })
export class Users extends Model<Users> {
  @Column({
    type: DataType.INTEGER,
    unique: true,
    autoIncrement: true,
    primaryKey: true,
  })
  id: number;

  @Column({
    type: DataType.STRING,
    unique: true,
  })
  username: string;

  @Column({
    type: DataType.STRING,
    unique: true,
  })
  email: string;

  @Column({
    type: DataType.STRING,
    unique: true,
  })
  avatar: string;

  @Column({
    type: DataType.INTEGER,
  })
  subscription_id: number;

  @Column({
    type: DataType.STRING(6),
    unique: true,
  })

```

```
share_id: string;

@HasMany(() => Sessions)
sessions: Sessions[];

@BelongsToMany(() => Roles, () => UserRoles)
roles: Roles[];

@BelongsToMany(() => Podcasts, () => PodcastsSubs)
podcasts: Podcasts[];

@HasMany(() => BooksViews)
booksViews: BooksViews[];

@HasMany(() => PodcastsViews)
podcastsViews: PodcastsViews[];
}
```

ПРИЛОЖЕНИЕ В

Листинги, относящиеся к клиентской части приложения

Листинг В.1 – client/src/components/Layout.tsx

```
import Footer from "../footer/Footer";
import Header from "../header/Header";

const Layout: React.FC<{ children: React.ReactNode }> = ({
  children }) => {
  return (
    <>
      <Header />
      {children}
      <Footer />
    </>
  );
};
export default Layout;
```

Листинг В.2 – client/src/components/Player.tsx

```
import {
  playAudio,
  setCurrentQueuePosition,
  setCurrentTime,
  setDuration,
  setVolume,
  stopAudio,
} from "@redux/slices/playerSlice";
import Image from "next/image";
import React, { useEffect } from "react";
import { useDispatch, useSelector } from "react-redux";
import PauseButton from "../icons/PauseButton";
import PlayButton from "../icons/PlayButton";
import SkipToNext from "../icons/SkipToNext";
import VolumeIcon from "../icons/VolumeIcon";

let audio;

const Player: React.FC = () => {
  const playerState = useSelector((state) => state.player);
  const dispatch = useDispatch();
  useEffect(() => {
    if (localStorage.getItem("volume")) {
      dispatch(setVolume(Number(localStorage.getItem("volume"))));
    }
  });
};
```

```

    }
    if (!audio) {
      audio = new Audio();
    } else {
      setAudio();
    }
  }, []);

  const setAudio = () => {
    if (playerState.playQueue) {
      audio.src =
playerState.playQueue[playerState.currentQueuePosition].src;
      audio.volume = playerState.volume / 100;
      audio.onloadedmetadata = () => {
        dispatch(setDuration(audio.duration));
      };
      audio.ontimeupdate = () => {
        dispatch(setCurrentTime(audio.currentTime));
      };
      toggleMode();
    }
  };

  useEffect(() => {
    setAudio();
  }, [playerState.playQueue, playerState.currentQueuePosition]);

  const toggleMode = () => {
    if (playerState.isPlaying === false) {
      dispatch(playAudio());
      audio.play();
    } else {
      dispatch(stopAudio());
      audio.pause();
    }
  };

  const changeVolume = (e: React.ChangeEvent<HTMLInputElement>)
=> {
    audio.volume = Number(e.target.value) / 100;
    dispatch(setVolume(Number(e.target.value)));
    localStorage.setItem("volume",
Number(e.target.value).toString());
  };

  const changeCurrentTime = (e:

```



```

React.ChangeEvent<HTMLInputElement>) => {
  audio.currentTime = Number(e.target.value);
  dispatch(setCurrentTime(e.target.value));
};

const getNext = () => {
  if (
    playerState.currentQueuePosition + 1 >
    playerState.playQueue.length - 1
  ) {
    return;
  }
  toggleMode();
  let nextPos = playerState.currentQueuePosition + 1;
  dispatch(setCurrentQueuePosition(nextPos));
  setAudio();
};

const getPrev = () => {
  if (playerState.currentQueuePosition == 0) {
    return;
  }
  toggleMode();
  let prevPos = playerState.currentQueuePosition - 1;
  dispatch(setCurrentQueuePosition(prevPos));
  setAudio();
};

if (!playerState.playQueue) {
  return null;
}

return (
  <div className="fixed bottom-0 left-0 right-0 flex flex-
col">
    <input
      type="range"
      min={0}
      max={playerState.duration}
      value={playerState.currentTime}
      onChange={changeCurrentTime}
      className="w-full h-3 bg-[#D9D9D9] cursor-pointer"
    />
    <div className="bg-white py-3 px-10 flex justify-between
gap-3">
      <div className="flex gap-5 items-center w-2/3">

```

```

<div className="flex gap-3 mr-10 items-center">
  <div onClick={getPrev} className="cursor-pointer">
    <SkipToNext className="h-10 rotate-180" />
  </div>
  <div onClick={() => toggleMode()} className="cursor-
pointer">
    {playerState.isPlaying ? (
      <PauseButton className="h-10" />
    ) : (
      <PlayButton className="h-10" />
    )}
  </div>
  <div className="cursor-pointer" onClick={getNext}>
    <SkipToNext className="h-10" />
  </div>
</div>

<div className="flex gap-4 items-center">
  <div className="w-12 h-12 overflow-hidden rounded-
2xl">
    {playerState.cover ? (
      <Image
        width={0}
        height={0}
        sizes="100vw"
        style={{ width: "100%", height: "auto" }}
        src={playerState.cover}
        alt="Cover"
        objectFit="cover"
      />
    ) : (
      <div className="rounded-2xl w-12 h-12 bg-
secondary"></div>
    )}
  </div>

  <div className="flex flex-col">
    <div>
      {playerState.playQueue &&
(playerState.playQueue[playerState.currentQueuePosition]
      .name ??
      "----")}
    </div>
    <div className="opacity-50">{playerState.author ??
"----"}</div>
  </div>

```

```

        </div>
      </div>
    </div>

    <div className="flex gap-5 items-center w-1/3">
      <div className="flex gap-3 w-1/2 items-center">
        <VolumeIcon className="h-10" />
        <input
          type="range"
          min={0}
          max={100}
          value={playerState.volume}
          onChange={changeVolume}
          className="w-full"
        />
      </div>

      <div className="flex">
        <div>
          {Math.floor(playerState.currentTime / 3600)}:
          {Math.floor((playerState.currentTime % 3600) /
60)}:
          {Math.floor((playerState.currentTime % 3600) %
60)}
        </div>
        <div className="mx-1"></div>
        <div className="opacity-70">
          {Math.floor(playerState.duration / 3600)}:
          {Math.floor((playerState.duration % 3600) / 60)}:
          {Math.floor((playerState.duration % 3600) % 60)}
        </div>
      </div>
    </div>
  </div>
);
};
export default Player;

```

Листинг В.3 – client/src/components/auth/telegram/Error.tsx

```

import WarningIcon from "@components/icons/WarningIcon";
import Block from "@components/ui/Block";

const Error: React.FC = () => {
  return (
    <Block className="flex flex-col items-center mt-5 gap-5">

```

```

        <WarningIcon color="#800080" className="h-20" />
        <h3 className="text-xl font-bold">Произошла ошибка при
входе</h3>
        <p>Перенаправление на главную страницу...</p>
    </Block>
  );
};
export default Error;

```

Листинг В.4 – client/src/components/auth/telegram/Pending.tsx

```

import ClockIcon from "@components/icons/ClockIcon";
import Block from "@components/ui/Block";

const Pending: React.FC = () => {
  return (
    <Block className="flex flex-col items-center mt-5 gap-5">
      <ClockIcon color="#800080" className="h-20" />
      <h3 className="text-xl font-bold">Отправляем запрос на
сервер</h3>
    </Block>
  );
};
export default Pending;

```

Листинг В.5 – client/src/components/auth/telegram/Success.tsx

```

import CheckIcon from "@components/icons/CheckIcon";
import Block from "@components/ui/Block";

const Success: React.FC = () => {
  return (
    <Block className="flex flex-col items-center mt-5 gap-5">
      <CheckIcon color="#800080" className="h-20"/>
      <h3 className="text-xl font-bold">Вы успешно вошли</h3>
      <p>Перенаправление на главную страницу...</p>
    </Block>
  );
};
export default Success;

```

Листинг В.6 – client/src/components/books/BookItem.tsx

```

import Image from "next/image";
import Link from "next/link";

interface IBookItemProps {
  image: string;

```

```

    code: string;
    name: string;
    authorName: string;
  }

const BookItem: React.FC<IBookItemProps> = ({
  image,
  code,
  name,
  authorName,
}) => {
  return (
    <div className="flex flex-col gap-2 transition-all hover:-
translate-y-1 active:translate-y-1">
      <Link href={`\book/${code}`} className="flex flex-col gap-
1">
        <div className="w-full overflow-hidden">
          <Image
            width={0}
            height={0}
            sizes="100vw"
            style={{ width: "100%", height: "auto" }}
            src={image}
            alt={image}
            objectFit="cover"
          />
        </div>
        <div>{name}</div>
        <div className="opacity-50">{authorName}</div>
      </Link>
    </div>
  );
};

export default BookItem;

```

Листинг В.7 – client/src/components/books/BooksList.tsx

```

import React from "react";
import BookItem from "../BookItem";
import Button from "../ui/Button";

interface IBooksListProps {
  books: any[];
  paginationOnClick?: () => any;
  count?: number;
}

```

```

const BooksList: React.FC<IBooksListProps> = ({
  books,
  paginationOnClick,
  count,
}) => {
  return (
    <div className="flex flex-col w-full">
      <div className="grid grid-cols-5 gap-10">
        {books.map((book) => (
          <BookItem
            name={book.name}
            code={book.code}
            image={book.cover}
            authorName={book.author?.name}
          />
        ))}
      </div>
      {count && books.length < count && (
        <Button styles="mt-5" type="secondary"
onClick={paginationOnClick}>
          Показать еще
        </Button>
      )}
    </div>
  );
};
export default BooksList;

```

Листинг В.8 – client/src/components/footer/Footer.tsx

```

import Block from "../ui/Block";

const Footer: React.FC = () => {
  return (
    <Block className="mt-4 mb-24">
      <p>&copy; Книги - 2024</p>
    </Block>
  );
};
export default Footer;

```

Листинг В.9 – client/src/components/head/DocHead.tsx

```

import Head from "next/head";

interface IDocHeadProps {
  keywords?: string;

```

```

    description?: string;
    title?: string;
    index?: boolean;
  }

const DocHead: React.FC<IDocHeadProps> = ({
  keywords,
  description = "Обширная библиотека аудиокниг и подкастов для
прослушивания в любое время и в любом месте. Вы можете легко
найти нужную вам аудиокнигу или подкаст, используя удобный и
интуитивно понятный интерфейс нашего сайта.",
  title = "Книги",
  index = true,
}) => {
  const metaKeywords =
    keywords +
    "аудиокниги, подкасты, прослушивание, библиотека,
путешествия, спорт, отдых";

  return (
    <Head>
      <title>{title} &mdash; Книги</title>
      <meta name="description" content={description} />
      <meta name="keywords" content={metaKeywords} />
      <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
      <meta
        name="robots"
        content={index ? "index, follow" : "noindex, nofollow"}
      />
      <meta property="og:title" content={title} />
      <meta property="og:description" content={description} />
      <meta name="theme-color" content="#800080" />
    </Head>
  );
};

export default DocHead;

```

Листинг В.10 – client/src/components/header/Header.tsx

```

import { useDispatch } from "react-redux";
import HeaderDesktop from "../HeaderDesktop";
import HeaderMobile from "../HeaderMobile";
import { getUserInfo } from "@redux/slices/userSlice";
import { useEffect } from "react";

const Header: React.FC = () => {

```

```

const dispatch = useDispatch();
const fetchUserData = async () => {
  dispatch(getUserInfo());
};

useEffect(() => {
  fetchUserData();
}, [dispatch]);
return (
  <>
    <HeaderDesktop />
    <HeaderMobile />
  </>
);
};
export default Header;

```

Листинг В.11 – client/src/components/header/HeaderDesktop.tsx

```

import Link from "next/link";
import RightHeaderSide from "../RightHeaderSide";
import LoginModal from "../../modals/LoginModal";
import { useState } from "react";

const HeaderDesktop: React.FC = () => {
  const [loginModalOpen, setLoginModalOpen] =
    useState<boolean>(false);
  return (
    <>
      <LoginModal modalOpen={loginModalOpen}
        setModalOpen={setLoginModalOpen} />
      <div className="backdrop-blur-sm sm:hidden md:flex max-w-
screen-xl mt-4 justify-between items-center px-5 rounded-3xl py-
6 bg-white w-full">
        <div className="font-bold text-main text-2xl w-1/3">
          <Link href="/">Книги</Link>
        </div>
        <div className="flex justify-between w-1/3">
          <Link
            className="transition-all hover:-translate-y-px
hover:brightness-95 active:translate-y-px"
            href="/books"
          >
            Аудиокниги
          </Link>
          <Link
            className="transition-all hover:-translate-y-px

```



```

    hover:brightness-95 active:translate-y-px"
      href="/podcasts"
    >
      Подкасты
    </Link>
    <Link
      className="transition-all hover:-translate-y-px
hover:brightness-95 active:translate-y-px"
      href="/search"
    >
      Поиск
    </Link>
  </div>
  <div className="w-1/3 flex justify-end">
    <RightHeaderSide
setLoginModalOpen={setLoginModalOpen}/>
  </div>
</div>
</>
);
};
export default HeaderDesktop;

```

Листинг В.12 – client/src/components/header/HeaderMobile.tsx

```

import BookIcon from "../../ui/bottomHeaderIcons/book";
import MicIcon from "../../ui/bottomHeaderIcons/mic";
import UserIcon from "../../ui/bottomHeaderIcons/user";

const HeaderMobile: React.FC = () => {
  return (
    <div className="sm:flex md:hidden absolute bottom-0 w-full
p-5 border-t-2 border-main rounded-t-2xl bg-white">
      <div className="flex justify-between w-full">
        <div>
          <BookIcon color="#000" text="Аудиокниги" active={true}
/>
        </div>
        <div>
          <MicIcon color="#000" text="Подкасты" />
        </div>
        <div>
          <UserIcon color="#000" text="username" />
        </div>
      </div>
    </div>
  );
};

```

```
};
export default HeaderMobile;
```

Листинг В.13 – client/src/components/header/RightHeaderSide.tsx

```
import { useEffect, useRef, useState } from "react";
import { useSelector } from "react-redux";
import Button from "../../ui/Button";
import UserDropdown from "../UserDropdown";
import Username from "../Username";

const RightHeaderSide: React.FC<{
  setLoginModalOpen: (isOpen: boolean) => void;
}> = ({ setLoginModalOpen }) => {
  const userState = useSelector((state) => state.user);
  const [userDropdownVisible, setUserDropdownVisible] =
    useState<boolean>(false);
  const [viewedHtml, setViewedHtml] =
    useState(<div>Loading</div>);
  const ignoreClick = useRef(null);

  const dropDownToggler = () => {
    setUserDropdownVisible(!userDropdownVisible);
  };

  useEffect(() => {
    if (userState.username && !userState.isFetching) {
      setViewedHtml(
        <>
          <div ref={ignoreClick} className="cursor-pointer"
            onClick={dropDownToggler}>
            <Username avatar={userState.avatar}
              username={userState.username} />
          </div>
          <UserDropdown visible={userDropdownVisible} />
        </>
      );
    } else if (!userState.username && !userState.isFetching) {
      setViewedHtml(
        <Button
          onClick={() => setLoginModalOpen(true)}
          type="primary"
          styles="font-bold"
          width="1/3"
        >
```

```

        Войти
      </Button>
    );
  }
}, [userDropdownVisible, userState]));

useEffect(() => {
  document.addEventListener("click", (e) => {
    if (ignoreClick.current &&
ignoreClick.current.contains(e.target)) {
      return;
    }
    setUserDropdownVisible(false);
  });
}, []);

return viewedHtml;
};
export default RightHeaderSide;

```

Листинг В.14 – client/src/components/header/UserDropdown.tsx

```

import { userLogout } from "@redux/slices/userSlice";
import Link from "next/link";
import { useDispatch, useSelector } from "react-redux";

const UserDropdown: React.FC<{ visible: boolean }> = ({ visible
}) => {
  const dispatch = useDispatch();
  const userData = useSelector((state) => state.user);

  const logout = () => {
    dispatch(userLogout());
  };

  return (
    <div
      className={`absolute right-0 top-[110%] flex flex-col
shadow-md bg-white rounded-2xl gap-3 p-4 transition-all origin-
top ${
        visible ? "scale-100" : "scale-0"
      }`}
    >
      <Link href={` /user/${userData.username} `}>Профиль</Link>
      <Link href="/user/history">История прослушиваний</Link>
      <Link href="#">Настройки</Link>
      <div onClick={() => logout()} className="text-[red]
cursor-pointer">

```

```

        Выйти
      </div>
    </div>
  );
};
export default UserDropdown;

```

Листинг В.15 – client/src/components/header/Username.tsx

```

import Image from "next/image";

interface IUsernameProps {
  avatar: string;
  username: string;
}

const Username: React.FC<IUsernameProps> = ({ avatar, username
}) => {
  return (
    <div className="flex items-center gap-2">
      <Image
        src={avatar}
        alt="avatar"
        height={45}
        width={45}
        className="rounded-2xl"
      />
      <div>{username}</div>
    </div>
  );
};
export default Username;

```

Листинг В.16 – client/src/components/index/MainPageContent.tsx

```

import { useEffect, useState } from "react";
import BooksList from "../books/BooksList";
import PodcastsList from "../podcasts/PodcastsList";
import Block from "../ui/Block";
import { query } from "@helpers/axios";
import TypographyTitle from
"../ui/typography/headers/TypographyTitle";

const MainPageContent: React.FC = () => {
  const [books, setBooks] = useState([]);
  const [podcasts, setPodcasts] = useState([]);
  const fetchBooks = async () => {

```

```

    await query.get("/books/trending").then((res) => {
      setBooks(res.data);
    });
  };

const fetchPodcasts = async () => {
  await query
    .get("/podcasts/trending")
    .then((res) => setPodcasts(res.data));
};

useEffect(() => {
  fetchBooks();
  fetchPodcasts();
}, []);

return (
  <>
    <Block className="mt-4 py-5">
      <TypographyTitle tag="h2" className="mb-6">
        Популярные аудиокниги
      </TypographyTitle>
      {books && <BooksList books={books} />}
    </Block>
    <Block className="mt-4 py-5 mb-5">
      <TypographyTitle tag="h2" className="mb-6">
        Популярные подкасты
      </TypographyTitle>
      {podcasts && <PodcastsList podcasts={podcasts} />}
    </Block>
  </>
);
};

export default MainPageContent;

```

Листинг В.17 – client/src/components/modals/LoginModal.tsx

```

import Link from "next/link";
import Button from "../ui/Button";
import Modal from "../ui/Modal";
import { useState } from "react";

interface ILoginModalProps {
  modalOpen: boolean;
  setModalOpen: (isOpen: boolean) => void;
}

const LoginModal: React.FC<ILoginModalProps> = ({

```

```

    modalOpen,
    setModalOpen,
  }) => {
    return (
      <Modal isModalOpen={modalOpen} modalToggler={setModalOpen}>
        <h1 className="text-4xl font-bold">Вход</h1>
        <p>
          Большая интеграция с сервисом: добавление в избранное,
отзывы, прослушивание
          аудиокниг и подкастов
        </p>

        <div className="mt-7">
          <div>Я хочу войти с помощью:</div>
          <div className="flex gap-3 mt-2">
            <Link
              target="_blank"
              href="https://t.me/bookstglogin_bot"
              className="w-2/3"
            >
              <Button
                onClick={() => setModalOpen(false)}
                type="primary"
                styles="!bg-[#54a9eb]"
              >
                Telegram
              </Button>
            </Link>
            <Button
              onClick={() => setModalOpen(false)}
              type="secondary"
              width="1/3"
            >
              Закрыть
            </Button>
          </div>
        </div>
      </Modal>
    );
  };
};
export default LoginModal;

```

Листинг В.18 – client/src/components/parts/EpisodeItem.tsx

```

import { useSelector } from "react-redux";
import PlayButton from "../icons/PlayButton";
import ImagePlayStatus from "../ui/ImagePlayStatus";

```

```

import { useEffect, useState } from "react";
import LoginModal from "../modals/LoginModal";

interface IEpisodeItemProps {
  name: string;
  code: string;
  duration: string;
  itemCover: string;
  playClick: () => any;
}

const EpisodeItem: React.FC<IEpisodeItemProps> = ({
  name,
  code,
  duration,
  itemCover,
  playClick,
}) => {
  const playerState = useSelector((state) => state.player);
  const username = useSelector((state) => state.user.username);
  const [loginModalOpen, setLoginModalOpen] = useState(false);

  return (
    <>
      <LoginModal
        modalOpen={loginModalOpen}
        setModalOpen={setLoginModalOpen}
      ></LoginModal>
      <div
        className={`w-full p-3 mb-2 last:mb-0 rounded-xl ${
          playerState.playQueue &&
          code ===
playerState.playQueue[playerState.currentQueuePosition].code
            ? "bg-secondary"
            : "bg-white"
        } hover:bg-secondary flex items-center gap-5 cursor-
pointer`}
      >
        <div className="flex">
          <ImagePlayStatus
            imageSrc={itemCover}
            isPlaying={
              playerState.playQueue &&
              code ===
playerState.playQueue[playerState.currentQueuePosition].code

```

```

        }
        onClick={
            username === null ? () => setLoginModalOpen(true)
: playClick
        }
    />
</div>
<div className="w-10/12">{name}</div>
<div className="w-1/12 opacity-45 text-
right">{duration}</div>
</div>
</>
);
};
export default EpisodeItem;

```

Листинг В.19 – client/src/components/parts/ReviewForm.tsx

```

import { useState } from "react";
import Button from "../ui/Button";
import Input from "../ui/Input";
import { query } from "@helpers/axios";
import { useSelector } from "react-redux";
import LoginModal from "../modals/LoginModal";

const ReviewForm: React.FC<{ type: "podcast" | "book"; entityId:
number }> = ({
    type,
    entityId,
}) => {
    const [expanded, setExpanded] = useState(false);
    const [rating, setRating] = useState<number>(5);
    const [text, setText] = useState<string>("");

    const username = useSelector((state) => state.user.username);

    const [loginModalOpen, setLoginModalOpen] = useState(false);

    const sendReview = async () => {
        await query
            .post(`/reviews/${type}/add`, {
                entityId,
                rating,
                text,
            })
            .then(() => window.location.reload())
            .catch();
    }

```



```

};
return (
  <>
    <LoginModal
      modalOpen={loginModalOpen}
      setModalOpen={setLoginModalOpen}
    ></LoginModal>
    {expanded ? (
      <div className="bg-secondary rounded-xl p-3">
        <div className="flex items-center gap-2">
          <div>Оценка</div>
          {Array.from({ length: 5 }, (_, index) => (
            <span
              key={index}
              className={`text-main transition-all
hover:scale-110 ${
                index < rating ? "text-4xl" : "text-2xl"
              }`}
              onClick={() => setRating(index + 1)}
              style={{ cursor: "pointer" }}
            >
              {index < rating ? "★" : "☆"}
            </span>
          ))}
        </div>

        <div>
          <textarea
            rows={8}
            className="mt-2 resize-none w-full p-2 rounded-xl"
            placeholder="Напишите отзыв..."
            value={text}
            onChange={(e) => setText(e.target.value)}
          ></textarea>
        </div>

        <Button onClick={sendReview} styles="mt-2 mb-1"
type="primary">
          Отправить
        </Button>
      </div>
    ) : (
      <div className="bg-secondary rounded-2xl p-3 flex gap-3
items-center">
        <div className="w-2/3">
          Поделитесь впечатлением после прослушивания

```

```

        </div>
        <Button
          onClick={() => {
            username == null
              ? setLoginModalOpen(true)
              : setExpanded(true);
          }}
          type="primary"
          width="1/3"
        >
          ОСТАВИТЬ ОТЗЫВ
        </Button>
      </div>
    )}
  </>
);
};
export default ReviewForm;

```

Листинг B.20 – client/src/components/parts/ReviewItem.tsx

```

const ReviewItem: React.FC<{ review: any }> = ({ review }) => {
  return (
    <div className="flex-col px-3">
      <div className="flex-col">
        <div className="flex gap-3 items-center">
          <div>{review.user.username}</div>
          <div className="flex text-main items-
center">{Array(review.rating).fill("★").join("")}</div>
        </div>
        <div className="opacity-60">
          {new Date(review.createdAt).toLocaleDateString()}
        </div>
        <div className="mt-2">{review.text}</div>
      </div>
    </div>
  );
};
export default ReviewItem;

```

Листинг B.21 – client/src/components/parts/Tabs.tsx

```

import React, { useEffect, useState } from "react";
import EpisodeItem from "../EpisodeItem";
import ReviewItem from "../ReviewItem";
import { useSelector } from "react-redux";
import { query } from "@helpers/axios";

```

```

import ReviewForm from "../ReviewForm";

interface ITabsProps {
  type: "book" | "podcast";
  episodeOnClickPlay: (sort: any) => void;
  episodes: any[];
  reviews: any[];
  cover: string;
  entityId: number;
}

const Tabs: React.FC<ITabsProps> = ({
  type,
  episodeOnClickPlay,
  episodes,
  reviews,
  cover,
  entityId,
}) => {
  const [currentTab, setCurrentTab] = useState<"episodes" |
"reviews">(
    "episodes"
  );
  const [reviewStatus, setReviewStatus] = useState<
    "reviewed" | "not-allowed" | "undefined"
  >("undefined");

  const [preReviewBlock, setPreReviewBlock] =
useState<React.ReactNode>(
    <div></div>
  );

  const userState = useSelector((state) => state.user);

  const classNames = {
    active:
      "bont-bold text-main rounded-full border border-main
shadow-lg w-1/2 p-3 text-center",
    notActive:
      "border border-white text-[#adadad] w-1/2 p-3 text-center
cursor-pointer transition-all hover:text-main",
  };
  const checkAccessToReview = async () => {
    await query
      .get("/reviews/book/isavailable/" + entityId)
      .then()
  }

```

```

        .catch(() => setReviewStatus("not-allowed"));
    };

    useEffect(() => {
        if (entityId) {
            checkAccessToReview();
        }
    }, [entityId]);

    useEffect(() => {
        if (reviews.some((review) => review.user.username ===
userState.username)) {
            setReviewStatus("reviewed");
        }
    }, [reviews, userState.username]);

    useEffect(() => {
        if (reviewStatus === "undefined" && entityId) {
            setPreReviewBlock(<ReviewForm type={type}
entityId={entityId} />);
        } else if (reviewStatus === "reviewed" && entityId) {
            setPreReviewBlock(
                <div className="bg-secondary rounded-2xl p-3">
                    Вы уже оставили отзыв
                </div>
            );
        }
    }, [reviewStatus, entityId]);

    return (
        <div className="w-full">
            <div className="w-1/3 flex">
                <div
                    onClick={() => setCurrentTab("episodes")}
                    className={
                        currentTab === "episodes" ? classNames.active :
classNames.notActive
                    }
                >
                    {type === "podcast" ? "Эпизоды" : "Главы"}
                </div>
                <div
                    onClick={() => setCurrentTab("reviews")}
                    className={
                        currentTab === "reviews" ? classNames.active :
classNames.notActive

```

```

        }
      >
      ОТЗЫВЫ
    </div>
  </div>
  {currentTab === "episodes" && (
    <div className="mt-5">
      {episodes.map((episode) => (
        <EpisodeItem
          itemCover={cover}
          code={episode.code}
          duration={episode.duration}
          name={episode.name}
          playClick={() => episodeOnClickPlay(episode.sort)}
        />
      ))}
    </div>
  )}
  {currentTab === "reviews" && (
    <div className="flex flex-col w-full gap-5 mt-5">
      {preReviewBlock}
      {reviews.map((review) => {
        return <ReviewItem review={review} />;
      })}
    </div>
  )}
</div>
);
};
export default Tabs;

```

Листинг В.22 – client/src/components/podcasts/PodcastItem.tsx

```

import Image from "next/image";
import Link from "next/link";
import Button from "../ui/Button";

interface IPodcastItemProps {
  image: string;
  code: string;
  name: string;
  authorName: string;
}

const PodcastItem: React.FC<IPodcastItemProps> = ({
  image,
  code,

```

```

    name,
    authorName,
  }) => {
    return (
      <div className="flex flex-col gap-2 transition-all hover:-
translate-y-1 active:translate-y-1">
        <Link
          href={` /podcast/${code}`}
          className="flex flex-col gap-1"
        >
          <div className="w-full overflow-hidden rounded-2xl">
            <Image
              width={0}
              height={0}
              sizes="100vw"
              style={{ width: "100%", height: "auto" }}
              src={image}
              alt={image}
              objectFit="cover"
            />
          </div>
          <div>{name}</div>
          <div className="opacity-50">{authorName}</div>
        </Link>
      </div>
    );
  };
};
export default PodcastItem;

```

Листинг B.23 – client/src/components/podcasts/PodcastsList.tsx

```

import React from "react";
import PodcastItem from "../PodcastItem";
import Button from "../ui/Button";

interface IPodcastsListProps {
  podcasts: any[];
  paginationOnClick?: () => any;
  count?: number;
}

const PodcastsList: React.FC<IPodcastsListProps> = ({
  podcasts,
  paginationOnClick,
  count,
}) => {
  return (

```

```

    <div className="flex flex-col w-full">
      <div className="grid grid-cols-5 gap-10">
        {podcasts.map((podcast) => (
          <PodcastItem
            name={podcast.name}
            code={podcast.code}
            image={podcast.cover}
            authorName={podcast.author?.name}
          />
        ))}
      </div>
      {count && podcasts.length < count && (
        <Button type="secondary" styles="mt-5"
onClick={paginationOnClick}>
          Показать еще
        </Button>
      )}
    </div>
  );
};
export default PodcastsList;

```

Листинг B.24 – client/src/components/ui/Block.tsx

```

interface IBlockProps {
  className?: string;
  children: React.ReactNode;
}

const Block: React.FC<IBlockProps> = ({ className, children })
=> {
  return (
    <div className={`max-w-screen-xl w-full rounded-2xl bg-white
p-5 ${className}`}>{children}</div>
  );
};
export default Block;

```

Листинг B.25 – client/src/components/ui/Button.tsx

```

import React from "react";

interface IButtonAttributes {
  children?: React.ReactNode;
  styles?: string;
  onClick?: () => void;
  type: "primary" | "secondary" | "text";
}

```

```

    disabled?: boolean;
    width?: string;
  }

const Button: React.FC<IButtonAttributes> = ({
  children,
  styles,
  onClick,
  type = "primary",
  disabled = false,
  width = "full",
}: IButtonAttributes) => {
  const disabledStyle = `opacity-50`;
  const classesSet = {
    primary: `bg-main w-${width} rounded-2xl py-3 text-white
transition enabled: hover:-translate-y-px
enabled: hover: brightness-95 enabled: active: translate-y-px
disabled: cursor-not-allowed ${
      disabled ? disabledStyle : ""
    } ${styles ?? ""}` ,
    secondary: `bg-secondary w-${width} rounded-2xl py-3 text-
black transition enabled: hover:-translate-y-px
enabled: hover: brightness-95 enabled: active: translate-y-px
disabled: cursor-not-allowed ${
      disabled ? disabledStyle : ""
    } ${styles ?? ""}` ,
    text: `bg-white rounded-2xl w-${width} transition-all
enabled: hover: brightness-95 disabled: cursor-not-allowed
${styles}`
  };
  return (
    <button onClick={onClick} className={classesSet[type]}
disabled={disabled}>
      {children}
    </button>
  );
};

export default Button;

```

Листинг B.26 – client/src/components/ui/ImagePlayStatus.tsx

```

import Image from "next/image";
import { useEffect, useState } from "react";
import PlayButton from "../icons/PlayButton";

interface IImagePlayStatusProps {
  imageSrc: string;

```



```

    isPlaying: string;
    onClick: () => any;
  }

const ImagePlayStatus: React.FC<IImagePlayStatusProps> = ({
  imageSrc,
  isPlaying,
  onClick,
}) => {
  const [mouseOverImage, setMouseOverImage] =
    useState<boolean>(false);
  useEffect(() => {
    if (isPlaying) {
      setMouseOverImage(false);
    }
  }, [isPlaying]);
  return (
    <div className="h-10 w-10 rounded-md overflow-hidden
relative">
      {mouseOverImage && !isPlaying && (
        <div
          onClick={() => onClick()}
          onMouseEnter={() => setMouseOverImage(true)}
          onMouseLeave={() => setMouseOverImage(false)}
          className="backdrop-brightness-50 absolute top-0 left-
0 bottom-0 right-0 flex justify-center items-center"
        >
          <PlayButton className="h-6" color="#fff" />
        </div>
      )}
      {isPlaying && (
        <div className="backdrop-brightness-50 absolute top-0
left-0 bottom-0 right-0 flex justify-center items-center">
          <div className="animate-bounce bg-white rounded-full
h-3 w-3"></div>
        </div>
      )}
      <Image
        width={0}
        height={0}
        sizes="100vw"
        style={{ width: "100%", height: "auto", objectFit:
"contain" }}
        src={imageSrc}
        alt={imageSrc}
        onMouseEnter={() => setMouseOverImage(true)}

```

```

        onMouseLeave={() => setMouseOverImage(false)}
      />
    </div>
  );
};
export default ImagePlayStatus;

```

Листинг В.27 – client/src/components/ui/Input.tsx

```

interface IInputProps {
  placeholder?: string;
  value: string | number;
  onChange: (arg: any) => any;
  className?: string;
  onBlur?: () => any;
  disabled?: boolean;
  type?: string;
  min?: number;
  max?: number;
}

const Input: React.FC<IInputProps> = ({
  value,
  onChange,
  placeholder,
  className,
  min,
  max,
  type,
  disabled = false,
}) => {
  return (
    <input
      type={type}
      disabled={disabled}
      className={`outline-none rounded-2xl bg-secondary p-2
border border-secondary focus:bg-white focus:border-2
focus:border-main ${className}`}
      placeholder={placeholder}
      value={value}
      onChange={(e) => onChange(e)}
      min={min}
      max={max}
    />
  );
};
export default Input;

```

Листинг В.28 – client/src/components/ui/LoadMore.tsx

```
import Button from "../Button";

const LoadMore: React.FC<{ onClick: () => any }> = ({ onClick })
=> {
  return <Button type="secondary" onClick={onClick}>Загрузить
еще</Button>;
};
```

Листинг В.29 – client/src/components/ui/Modal.tsx

```
interface IModalProps {
  children?: React.ReactNode;
  modalToggler: (isOpen: boolean) => void;
  isModalOpen: boolean;
}

const Modal: React.FC<IModalProps> = ({
  children,
  modalToggler,
  isModalOpen,
}: IModalProps) => {
  return isModalOpen ? (
    <>
      <div className="fixed z-20 top-[50%] left-[50%] translate-
y-[-50%] translate-x-[-50%] p-12 rounded-3xl bg-white">
        {children}
      </div>
      <div
        onClick={() => modalToggler(false)}
        className="fixed z-10 top-0 left-0 right-0 bottom-0
backdrop-blur-sm backdrop-brightness-75"
      ></div>
    </>
  ) : null;
};

export default Modal;
```

Листинг В.30 –

client/src/components/ui/booksAndPodcasts/MainInfoDetailPage.tsx

```
import Image from "next/image";
import React, { useEffect, useState } from "react";
import { useSelector } from "react-redux";
import Button from "../Button";
import Link from "next/link";
import { addFavorite, removeFavorite } from "@helpers/axios";
```

```

interface IMainInfoDetailPage {
  entity: any;
  entityType: "book" | "podcast";
  children: React.ReactNode;
}

const MainInfoDetailPage: React.FC = ({
  entity,
  entityType,
  children,
}) => {
  const username = useSelector((state) => state.user.username);
  const [isFavorite, setIsFavorite] = useState(false);

  const isFavoriteChecker = () => {
    if (entity.favorites) {
      if (entity.favorites.length > 0) {
        setIsFavorite(true);
      } else {
        setIsFavorite(false);
      }
    }
  };

  useEffect(() => {
    isFavoriteChecker();
  }, [entity.favorites]);

  return (
    <div className="flex gap-8">
      <div className="w-72 flex flex-col gap-2">
        <div className="h-72 w-72 rounded-2xl overflow-hidden">
          <Image
            width={0}
            height={0}
            sizes="100vw"
            style={{ width: "100%", height: "auto", objectFit:
"contain" }}
            src={entity?.cover}
            alt={entity?.name}
          />
        </div>
        {username !== null &&
          (isFavorite ? (
            <Button
              onClick={() =>

```

```

        removeFavorite(entityType, entity.id).then(() =>
            setIsFavorite(false)
        )
    }
    type="secondary"
  >
    Убрать из избранного
  </Button>
) : (
  <Button
    onClick={() =>
      addFavorite(entityType, entity.id).then(() =>
        setIsFavorite(true)
      )
    }
    type="primary"
  >
    В избранное
  </Button>
))}
</div>
<div className="flex flex-col">
  <div className="flex gap-3 font-bold">
    {entity?.exclusive && (
      <span className="text-main uppercase text-sm">Эксклюзив</span>
    )}
    <span className="font-bold text-sm">{entity?.min_age +
  "+"}</span>
  </div>
  <h1 className="text-5xl font-bold">{entity?.name}</h1>
  {entity.author && (
    <div className="mt-2 opacity-50">
      <span className="mr-2">Автор:</span>
      <Link href={` /author/${entity.author.code}`}>
        {entity.author.name}
      </Link>
    </div>
  )}
  <div className="flex mt-4 gap-3">{children}</div>

  <h6 className="font-bold mt-4">
    О {entityType === "book" ? "книге" : "подкасте"}
  </h6>
  <p>{entity?.description}</p>
</div>

```

```

        </div>
    );
};
export default MainInfoDetailPage;

```

Листинг В.31 – client/src/components/ui/booksAndPodcasts/RoundedInfo.tsx

```

const RoundedInfo: React.FC<{ sign: string; value: string }> =
({
    sign,
    value,
}) => {
    return (
        <div className="bg-secondary rounded-2xl flex flex-col
items-center p-3">
            <div className="font-bold text-3xl">{value}</div>
            <div className="text-sm">{sign}</div>
        </div>
    );
};
export default RoundedInfo;

```

Листинг В.32 – client/src/components/ui/bottomHeaderIcons/book.tsx

```

import React from "react";

interface IBookIconProps {
    color: string;
    active?: boolean;
    text?: string;
}

const BookIcon: React.FC<IBookIconProps> = ({
    color,
    text,
    active = false,
}) => {
    return (
        <div className="flex flex-col justify-center">
            <svg
                className="h-8"
                viewBox="0 0 24 24"
                fill="none"
                xmlns="http://www.w3.org/2000/svg"
            >
                <g id="SVGRepo_bgCarrier" stroke-width="0"></g>
                <g
                    id="SVGRepo_tracerCarrier"

```

```

        stroke-linecap="round"
        stroke-linejoin="round"
    ></g>
    <g id="SVGRepo_iconCarrier">
        <path
            d="M4 19V6.2C4 5.0799 4 4.51984 4.21799
4.09202C4.40973 3.71569 4.71569 3.40973 5.09202 3.21799C5.51984
3 6.0799 3 7.2 3H16.8C17.9201 3 18.4802 3 18.908 3.21799C19.2843
3.40973 19.5903 3.71569 19.782 4.09202C20 4.51984 20 5.0799 20
6.2V17H6C4.89543 17 4 17.8954 4 19ZM4 19C4 20.1046 4.89543 21 6
21H20M9 7H15M9 11H15M19 17V21"
            stroke={active ? "#800080" : color}
            stroke-width={active ? 2 : 1}
            stroke-linecap="round"
            stroke-linejoin="round"
        ></path>
    </g>
</svg>
{text && (
    <div
        style={{
            fontSize: "10px",
            color: active ? "#800080" : color,
            fontWeight: active ? 700 : "normal",
        }}
    >
        {text}
    </div>
)}
</div>
);
};
export default BookIcon;

```

Листинг В.33 – client/src/components/ui/bottomHeaderIcons/mic.tsx

```

import React from "react";

interface IMicIconProps {
    color: string;
    active?: boolean;
    text?: string;
}

const MicIcon: React.FC<IMicIconProps> = ({ color, text, active
= false }) => {
    return (

```

```

<div className="flex flex-col justify-center">
  <svg
    className="h-8"
    viewBox="0 0 24 24"
    fill="none"
    xmlns="http://www.w3.org/2000/svg"
  >
    <g id="SVGRepo_bgCarrier" stroke-width="0"></g>
    <g
      id="SVGRepo_tracerCarrier"
      stroke-linecap="round"
      stroke-linejoin="round"
    ></g>
    <g id="SVGRepo_iconCarrier">
      <path
        d="M20 12V13C20 17.4183 16.4183 21 12 21C7.58172 21
4 17.4183 4 13V12M12 17C9.79086 17 8 15.2091 8 13V7C8 4.79086
9.79086 3 12 3C14.2091 3 16 4.79086 16 7V13C16 15.2091 14.2091
17 12 17Z"
        stroke={active ? "#800080" : color}
        stroke-width={active ? 2 : 1}
        stroke-linecap="round"
        stroke-linejoin="round"
      ></path>
    </g>
  </svg>
  {text && (
    <div
      style={{
        fontSize: "10px",
        color: active ? "#800080" : color,
        fontWeight: active ? 700 : "normal",
      }}
    >
      {text}
    </div>
  )}
</div>
);
};
export default MicIcon;

```

Листинг В.34 – client/src/components/ui/bottomHeaderIcons/user.tsx

```

import React from "react";

interface IUserIconProps {

```



```

    color: string;
    active?: boolean;
    text?: string;
  }

const UserIcon: React.FC<IUserIconProps> = ({
  color,
  text,
  active = false,
}) => {
  return (
    <div className="flex flex-col justify-center">
      <svg
        className="h-8"
        viewBox="0 0 24 24"
        fill="none"
        xmlns="http://www.w3.org/2000/svg"
      >
        <g id="SVGRepo_bgCarrier" stroke-width="0"></g>
        <g
          id="SVGRepo_tracerCarrier"
          stroke-linecap="round"
          stroke-linejoin="round"
        ></g>
        <g id="SVGRepo_iconCarrier">
          { " " }
          <path
            d="M5 21C5 17.134 8.13401 14 12 14C15.866 14 19
17.134 19 21M16 7C16 9.20914 14.2091 11 12 11C9.79086 11 8
9.20914 8 7C8 4.79086 9.79086 3 12 3C14.2091 3 16 4.79086 16 7Z"
            stroke={active ? "#800080" : color}
            stroke-width={active ? 2 : 1}
            stroke-linecap="round"
            stroke-linejoin="round"
          ></path>{ " " }
        </g>
      </svg>
      {text && (
        <div
          style={{
            fontSize: "10px",
            color: active ? "#800080" : color,
            fontWeight: active ? 700 : "normal",
          }}
        >
          {text}
        </div>
      )}
    </div>
  )
}

```

```

        </div>
      )}
    </div>
  );
};
export default UserIcon;

```

Листинг В.35 – client/src/components/ui/typography/headers/TypographyTitle.tsx

```

interface ITypographyTitleProps {
  children: React.ReactNode;
  className?: string;
  tag: "h1" | "h2" | "h3" | "h4" | "h5" | "h6";
}

const TypographyTitle: React.FC<ITypographyTitleProps> = ({
  children,
  className,
  tag,
}) => {
  const Tag = tag;
  const classNames = {
    h1: "text-5xl font-bold " + className,
    h2: "text-3xl font-bold " + className,
    h3: "text-2xl font-bold " + className,
    h4: "text-xl font-bold " + className,
    h5: "text-md font-bold " + className,
    h6: "text-md font-bold " + className,
  };
  return <Tag className={classNames[tag]}>{children}</Tag>;
};
export default TypographyTitle;

```

Листинг В.36 – client/src/helpers/axios.ts

```

import axios from "axios";

export const query = axios.create({
  baseURL: process.env.NEXT_PUBLIC_BACKEND_URL,
  withCredentials: true,
  timeout: 5000,
});

export const addFavorite = async (type: "podcast" | "book", id: number) => {
  return await query.post(`/favorites/${type}/${id}`);
};

```

```
export const removeFavorite = async (type: "podcast" | "book",
id: number) => {
  return await query.delete(`/favorites/${type}/${id}`);
};
```

Листинг В.37 – client/src/helpers/errorRedirect.ts

```
import { AxiosError } from "axios";

export const errorRedirect = (resultData: AxiosError) => {
  if (resultData.response?.status) {
    window.location.href = "/" + resultData.response?.status;
  }
};
```

Листинг В.38 – client/src/pages/404.tsx

```
import DocHead from "@components/head/DocHead";
import Layout from "@components/Layout";
import Block from "@components/ui/Block";
import TypographyTitle from
"@components/ui/typography/headers/TypographyTitle";

const Page404: React.FC = () => {
  return (
    <Layout>
      <DocHead title="Не найдено" index={false} />
      <Block className="mt-4 flex flex-col gap-5 items-center">
        <TypographyTitle tag="h1">404</TypographyTitle>
        <p>Такую страницу еще не написали :(</p>
      </Block>
    </Layout>
  );
};
export default Page404;
```

Листинг В.39 – client/src/pages/500.tsx

```
import DocHead from "@components/head/DocHead";
import Layout from "@components/Layout";
import Block from "@components/ui/Block";
import TypographyTitle from
"@components/ui/typography/headers/TypographyTitle";

const Page500: React.FC = () => {
  return (
    <Layout>
```

```

    <DocHead title="Не найдено" index={false} />
    <Block className="mt-4 flex flex-col gap-5 items-center">
      <TypographyTitle tag="h1">500</TypographyTitle>
      <p>На сервере что-то сломалось, но мы уже чиним</p>
    </Block>
  </Layout>
);
};
export default Page500;

```

Листинг В.40 – client/src/pages/books.tsx

```

import BooksList from "@components/books/BooksList";
import DocHead from "@components/head/DocHead";
import Layout from "@components/Layout";
import Block from "@components/ui/Block";
import TypographyTitle from
"@components/ui/typography/headers/TypographyTitle";
import { query } from "@helpers/axios";
import { useEffect, useState } from "react";

const BooksPages: React.FC = () => {
  const [page, setPage] = useState<number>(1);
  const [books, setBooks] = useState<any[]>([]);
  const [count, setCount] = useState<number>(0);
  const fetchBooks = async () => {
    await query.get("/books/list?page=" + page).then((res) => {
      if (page === 1) {
        setBooks(res.data.books);
        setCount(res.data.count);
      } else {
        setBooks([...books, ...res.data.books]);
      }
    });
  };

  const showMore = () => {
    setPage(page + 1);
  };

  useEffect(() => {
    fetchBooks();
    console.log(books);
  }, [page]);

  return (
    <Layout>

```

```

        <DocHead title="Список аудиокниг" description="Библиотека
аудиокниг" />
        <Block className="mt-4">
          <TypographyTitle tag="h1" className="mb-6">
            Аудиокниги
          </TypographyTitle>
          {books && (
            <BooksList
              books={books}
              count={count}
              paginationOnClick={showMore}
            ></BooksList>
          )}
        </Block>
      </Layout>
    );
  };
};
export default BooksPages;

```

Листинг В.41 – client/src/pages/index.tsx

```

import DocHead from "@components/head/DocHead";
import MainPageContent from
"@components/index/MainPageContent";
import Layout from "@components/Layout";

export default function Home() {
  return (
    <Layout>
      <DocHead title="Главная" />
      <MainPageContent />
    </Layout>
  );
}

```

Листинг В.42 – client/src/pages/podcasts.tsx

```

import PodcastsList from "@components/podcasts/PodcastsList";
import DocHead from "@components/head/DocHead";
import Layout from "@components/Layout";
import Block from "@components/ui/Block";
import TypographyTitle from
"@components/ui/typography/headers/TypographyTitle";
import { query } from "@helpers/axios";
import { useEffect, useState } from "react";

const PodcastsPages: React.FC = () => {

```

```

const [page, setPage] = useState<number>(1);
const [podcasts, setPodcasts] = useState<any[]>([]);
const [count, setCount] = useState<number>(1);
const fetchPodcasts = async () => {
  await query.get("/podcasts/list?page=" + page).then((res) =>
{
  if (page === 1) {
    setPodcasts(res.data.podcasts);
    setCount(res.data.count);
  } else {
    setPodcasts([...podcasts, ...res.data.podcasts]);
  }
  });
};

const showMore = () => {
  setPage(page + 1);
};

useEffect(() => {
  fetchPodcasts();
}, [page]);

return (
  <Layout>
    <DocHead title="Список аудиокниг" description="Библиотека
аудиокниг" />
    <Block className="mt-4">
      <TypographyTitle tag="h1" className="mb-6">
        Подкасты
      </TypographyTitle>
      {podcasts && (
        <PodcastsList
          podcasts={podcasts}
          count={count}
          paginationOnClick={showMore}
        ></PodcastsList>
      )}
    </Block>
  </Layout>
);
};
export default PodcastsPages;

```

Листинг В.43 – client/src/pages/search.tsx

```

import BooksList from "@components/books/BooksList";
import DocHead from "@components/head/DocHead";
import Layout from "@components/Layout";
import PodcastsList from "@components/podcasts/PodcastsList";
import Block from "@components/ui/Block";
import Button from "@components/ui/Button";
import Input from "@components/ui/Input";
import TypographyTitle from
"@components/ui/typography/headers/TypographyTitle";
import { query } from "@helpers/axios";
import { useState } from "react";

const SearchPage: React.FC = () => {
  const [books, setBooks] = useState<any[]>([]);
  const [podcasts, setPodcasts] = useState<any[]>([]);
  const [searchQuery, setSearchQuery] = useState<string>("");

  const search = async () => {
    await query.get("/search?query=" + searchQuery).then((res)
=> {
      setBooks(res.data.books);
      setPodcasts(res.data.podcasts);
    });
  };

  return (
    <Layout>
      <DocHead title="Поиск" index={false} />
      <Block className="mt-4">
        <TypographyTitle tag="h1">Поиск</TypographyTitle>
        <div className="flex gap-3 mt-4">
          <Input
            className="w-2/3"
            placeholder="Запрос"
            value={searchQuery}
            onChange={(e) => setSearchQuery(e.target.value)}
          />
          <Button
            disabled={searchQuery.length > 0 ? false : true}
            type="primary"
            width="1/3"
            onClick={search}
          >
            Найти
          </Button>
        </div>
      </Block>
    </Layout>
  );
};

```

```

</Block>

{books && books.length > 0 && (
  <Block className="mt-4">
    <TypographyTitle tag="h3" className="mb-4">
      Аудиокниги
    </TypographyTitle>
    <BooksList books={books} />
  </Block>
)}

{podcasts && podcasts.length > 0 && (
  <Block className="mt-4">
    <TypographyTitle tag="h3" className="mb-4">
      Подкасты
    </TypographyTitle>
    <PodcastsList podcasts={podcasts} />
  </Block>
)}
</Layout>
);
};
export default SearchPage;

```

Листинг В.44 – client/src/pages/_app.tsx

```

import Player from "@components/Player";
import { store } from "@redux/store";
import "@styles/globals.css";
import type { AppProps } from "next/app";
import { Provider } from "react-redux";

export default function App({ Component, pageProps }: AppProps)
{
  return (
    <Provider store={store}>
      <div className="flex flex-col items-center">
        <Component {...pageProps} />
        <Player/>
      </div>
    </Provider>
  );
}

```


Листинг В.45 – client/src/pages/_document.tsx

```
import { Html, Head, Main, NextScript } from "next/document";

export default function Document() {
  return (
    <Html lang="ru">
      <Head />
      <body>
        <Main />
        <NextScript />
      </body>
    </Html>
  );
}
```

Листинг В.46 – client/src/pages/auth/telegram/[code].tsx

```
import Error from "@components/auth/telegram/Error";
import Pending from "@components/auth/telegram/Pending";
import Success from "@components/auth/telegram/Success";
import DocHead from "@components/head/DocHead";
import Header from "@components/header/Header";
import { query } from "@helpers/axios";
import { useRouter } from "next/router";
import { useEffect, useState } from "react";

const TelegramCode: React.FC = () => {
  const router = useRouter();
  const html = {
    success: <Success />,
    pending: <Pending />,
    error: <Error />,
  };
  const [status, setStatus] = useState<"pending" | "success" | "error">(
    "pending"
  );

  const sendAuthRequest = async () => {
    await query
      .delete(
        process.env.NEXT_PUBLIC_BACKEND_URL +
          "/auth/telegram/link/" +
          router.query.code
      )
      .then(() => {
        setStatus("success");
      });
  };
}
```

```

        setTimeout(() => window.location.href = "/", 5000);
    })
    .catch(() => {
        setStatus("error");
        setTimeout(() => window.location.href = "/", 5000);
    });
};

useEffect(() => {
    sendAuthRequest();
}, [router]);
return (
    <>
        <DocHead title="Вход через Telegram"/>
        {html[status]}
    </>
);
};
export default TelegramCode;

```

Листинг В.47 – client/src/pages/author/[code].tsx

```

import BooksList from "@components/books/BooksList";
import DocHead from "@components/head/DocHead";
import Layout from "@components/Layout";
import PodcastsList from "@components/podcasts/PodcastsList";
import Block from "@components/ui/Block";
import TypographyTitle from
"@components/ui/typography/headers/TypographyTitle";
import { query } from "@helpers/axios";
import { errorRedirect } from "@helpers/errorRedirect";
import { useRouter } from "next/router";
import { useEffect, useState } from "react";

const AuthorPage: React.FC = () => {
    const router = useRouter();
    const [author, setAuthor] = useState({});

    const fetchAuthors = async () => {
        await query
            .get("/authors/one/" + router.query.code)
            .then((res) => setAuthor(res.data))
            .catch((e) => errorRedirect(e));
    };

    useEffect(() => {
        if (router.query.code) {

```

```

        fetchAuthors();
        console.log(author);
    }
}, [router.query.code]);

return (
  <Layout>
    <DocHead title={author.name} description={author.bio} />

    {author.books && (
      <>
        <Block className="mt-4">
          <TypographyTitle tag="h1">
            Книги автора {author.name}
          </TypographyTitle>
        </Block>

        <Block className="mt-4">
          <BooksList books={author?.books} />
        </Block>
      </>
    )}

    {author.podcasts && (
      <>
        <Block className="mt-4">
          <TypographyTitle tag="h1">
            Подкасты автора {author.name}
          </TypographyTitle>
        </Block>

        <Block className="mt-4">
          <PodcastsList podcasts={author?.podcasts} />
        </Block>
      </>
    )}
  </Layout>
);
};
export default AuthorPage;

```

Листинг В.48 – client/src/pages/book/[code].tsx

```

import DocHead from "@components/head/DocHead";
import Layout from "@components/Layout";
import EpisodeItem from "@components/parts/EpisodeItem";
import Tabs from "@components/parts/Tabs";

```

```

import Block from "@components/ui/Block";
import MainInfoDetailPage from
"@components/ui/booksAndPodcasts/MainInfoDetailPage";
import RoundedInfo from
"@components/ui/booksAndPodcasts/RoundedInfo";
import Button from "@components/ui/Button";
import { query } from "@helpers/axios";
import { errorRedirect } from "@helpers/errorRedirect";
import { playAudio, setMainInfo } from
"@redux/slices/playerSlice";
import Image from "next/image";
import Link from "next/link";
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import { useDispatch, useSelector } from "react-redux";

const BookPage: React.FC = () => {
  const [book, setBook] = useState({});
  const [parts, setParts] = useState([]);
  const router = useRouter();
  const dispatch = useDispatch();

  const getPartsOfBook = async () => {
    await query
      .get("/booksparts/list/" + book?.id)
      .then((res) => setParts(res.data))
      .catch();
  };

  const getBook = async () => {
    await query
      .get(`/books/one/${router.query.code}`)
      .then((res) => {
        setBook(res.data);
      })
      .catch((e) => {
        errorRedirect(e);
      });
  };

  useEffect(() => {
    if (router.query.code) {
      getBook();
    }
  }, [router.query.code]);

```

```

useEffect(() => {
  if (book.id) {
    getPartsOfBook();
  }
}, [book]);

const playBook = (sort) => {
  const queue = parts.map((part) => {
    return {
      ...part,
      src:
        process.env.NEXT_PUBLIC_BACKEND_URL +
        "/filesystem/stream/book/" +
        part.code,
    };
  });
  dispatch(
    setMainInfo({
      name: book.name,
      author: book.author?.name,
      cover: book?.cover,
      playQueue: queue,
      currentQueuePosition: sort,
    })
  );
};

{
  book.total_duration && (
    <RoundedInfo sign="общая продолжительность"
value={book.total_duration} />
  );
}
{
  book.rating > 0 && <RoundedInfo sign="рейтинг"
value={book.rating} />;
}
{
  book.trending > 0 && (
    <RoundedInfo sign="место в топе" value={book.trending} />
  );
}
{
  book.views > 0 && <RoundedInfo sign="просмотров"
value={book.views} />;
}

```

```

{
    book.listeners > 0 && (
        <RoundedInfo sign="слушателей" value={book.listeners} />
    );
}

return (
    <Layout>
        <DocHead title={book?.name}
description={book?.description} />
        <Block className="mt-4">
            <MainInfoDetailPage
                entity={book}
                entityType="book"
            >
                {book.total_duration && (
                    <RoundedInfo
                        sign="общая продолжительность"
                        value={book.total_duration}
                    />
                )}
                {book.rating > 0 && (
                    <RoundedInfo sign="рейтинг" value={book.rating} />
                )}
                {book.trending > 0 && (
                    <RoundedInfo sign="место в топе"
value={book.trending} />
                )}
                {book.views > 0 && (
                    <RoundedInfo sign="просмотров" value={book.views} />
                )}
                {book.listeners > 0 && (
                    <RoundedInfo sign="слушателей"
value={book.listeners} />
                )}
            </MainInfoDetailPage>
        </Block>
        <Block className="mt-4">
            <Tabs
                episodes={parts}
                episodeOnClickPlay={playBook}
                reviews={book.reviews ?? []}
                type="book"
                cover={book.cover}
                entityId={book.id}
            />

```

```

        </Block>
      </Layout>
    );
  };
export default BookPage;

```

Листинг В.49 – client/src/pages/podcast/[code].tsx

```

import DocHead from "@components/head/DocHead";
import Layout from "@components/Layout";
import EpisodeItem from "@components/parts/EpisodeItem";
import Tabs from "@components/parts/Tabs";
import Block from "@components/ui/Block";
import MainInfoDetailPage from
"@components/ui/booksAndPodcasts/MainInfoDetailPage";
import RoundedInfo from
"@components/ui/booksAndPodcasts/RoundedInfo";
import { query } from "@helpers/axios";
import { errorRedirect } from "@helpers/errorRedirect";
import { playAudio, setMainInfo } from
"@redux/slices/playerSlice";
import Image from "next/image";
import Link from "next/link";
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import { useDispatch } from "react-redux";

const PodcastPage: React.FC = () => {
  const [podcast, setPodcast] = useState({});
  const [parts, setParts] = useState([]);
  const router = useRouter();
  const dispatch = useDispatch();

  const getPartsOfPodcast = async () => {
    await query
      .get("/podcastsepisodes/list/" + podcast?.id)
      .then((res) => setParts(res.data))
      .catch();
  };

  const getPodcast = async () => {
    await query
      .get(`/podcasts/one/${router.query.code}`)
      .then((res) => {
        setPodcast(res.data);
      })
      .catch((e) => {

```

```

        errorRedirect(e);
    });
};

useEffect(() => {
    if (router.query.code) {
        getPodcast();
    }
}, [router.query.code]);

useEffect(() => {
    if (podcast.id) {
        getPartsOfPodcast();
    }
    console.log(podcast);
}, [podcast]);

const playPodcast = () => {
    const queue = parts.map((part) => {
        return {
            ...part,
            src:
                process.env.NEXT_PUBLIC_BACKEND_URL +
                "/filesystem/stream/podcast/" +
                part.code,
        };
    });
    dispatch(
        setMainInfo({
            name: podcast.name,
            author: podcast.author?.name,
            cover: podcast?.cover,
            playQueue: queue,
        })
    );
};

return (
    <Layout>
        <DocHead title={podcast?.name}
description={podcast?.description} />
        <Block className="mt-4">
            <MainInfoDetailPage
                entity={podcast}
                entityType="podcast"
            >

```



```

        {parts && <RoundedInfo sign="всего эпизодов"
value={parts.length} />}
        {podcast.rating > 0 && (
            <RoundedInfo sign="рейтинг" value={podcast.rating}
/>
        )}
        {podcast.trending > 0 && (
            <RoundedInfo sign="место в топе"
value={podcast.trending} />
        )}
        {podcast.views > 0 && (
            <RoundedInfo sign="просмотров" value={podcast.views}
/>
        )}
    </MainInfoDetailPage>
</Block>
<Block className="mt-4">
    <Tabs
        episodes={parts}
        episodeOnClickPlay={playPodcast}
        reviews={podcast.reviews ?? []}
        type="podcast"
        cover={podcast.cover}
        entityId={podcast.id}
    />
</Block>
</Layout>
);
};
export default PodcastPage;

```

Листинг В.50 – client/src/pages/user/history.tsx

```

import BooksList from "@components/books/BooksList";
import DocHead from "@components/head/DocHead";
import Layout from "@components/Layout";
import PodcastsList from "@components/podcasts/PodcastsList";
import Block from "@components/ui/Block";
import TypographyTitle from
"@components/ui/typography/headers/TypographyTitle";
import { query } from "@helpers/axios";
import { useEffect, useState } from "react";

const HistoryPage: React.FC = () => {
    const [booksPage, setBooksPage] = useState<number>(1);
    const [books, setBooks] = useState<any[]>([]);
    const [booksCount, setBooksCount] = useState<number>(0);

```

```

const [podcastsPage, setPodcastsPage] = useState<number>(1);
const [podcasts, setPodcasts] = useState<any[]>([]);
const [podcastsCount, setPodcastsCount] = useState<number>(0);

const universalFetch = async (type: "podcasts" | "books") => {
  return await query.get(`/history/${type}`).then((res) =>
res.data);
};

const fetchBooks = async () => {
  const data = await universalFetch("books");
  const newBooks = data.history.map((history) =>
history.book);
  setBooks([...books, ...newBooks]);
  setBooksCount(data.count);
};

const fetchPodcasts = async () => {
  const data = await universalFetch("podcasts");
  data.history.map((history) => setPodcasts([...podcasts,
history.podcast]));
  setPodcastsCount(data.count);
};

const showMoreBooks = () => {
  setBooksPage(booksPage + 1);
};

const showMorePodcasts = () => {
  setPodcastsPage(podcastsPage + 1);
};

useEffect(() => {
  fetchBooks();
}, [booksPage]);

useEffect(() => {
  fetchPodcasts();
}, [podcastsPage]);

return (
  <Layout>
    <DocHead title="История прослушиваний" index={false} />
    <Block className="mt-6">
      <TypographyTitle tag="h1">История
прослушиваний</TypographyTitle>

```

```

        <p>Здесь то, что вы прослушали за последнее время</p>
    </Block>
    <Block className="mt-4">
        <TypographyTitle tag="h3">Аудиокниги</TypographyTitle>
        <div className="mt-4">
            <BooksList books={books} count={booksCount} />
        </div>
    </Block>

    <Block className="mt-4">
        <TypographyTitle tag="h3">Подкасты</TypographyTitle>
        <div className="mt-4">
            <PodcastsList podcasts={podcasts}
count={podcastsCount} />
        </div>
    </Block>
</Layout>
);
};
export default HistoryPage;

```

Листинг В.51 – client/src/pages/user/[username].tsx

```

import BooksList from "@components/books/BooksList";
import DocHead from "@components/head/DocHead";
import Layout from "@components/Layout";
import PodcastsList from "@components/podcasts/PodcastsList";
import Block from "@components/ui/Block";
import TypographyTitle from
"@components/ui/typography/headers/TypographyTitle";
import { query } from "@helpers/axios";
import Image from "next/image";
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import { useSelector } from "react-redux";

const UserPage: React.FC = () => {
    const router = useRouter();
    const userState = useSelector((state) => state.user);
    const [userData, setUserData] = useState<{
        id?: number;
        username?: any;
        avatar?: any;
    }>({});
    const fetchUserData = async () => {
        await query.get("/users/user/" +
router.query.username).then((res) =>

```

```

    setUserData({
      id: res.data.id,
      username: res.data.username,
      avatar:
        res.data.avatar ??
        "https://www.kindpng.com/picc/m/22-223863_no-avatar-
png-circle-transparent-png.png",
    })
  );
};

const [books, setBooks] = useState([]);
const [booksCount, setBooksCount] = useState(0);
const [podcasts, setPodcasts] = useState([]);
const [podcastsCount, setPodcastsCount] = useState(0);
const [page, setPage] = useState(1);

const fetchFavorites = async () => {
  await query
    .get("/favorites/user/book/" + userData.id + "?page=" +
page)
    .then((res) => {
      setBooksCount(res.data.count);
      const newBooks = res.data.favorites.map((favorite) => {
        return favorite.book;
      });
      setBooks([...newBooks]);
    });
  await query
    .get("/favorites/user/podcast/" + userData.id + "?page=" +
page)
    .then((res) => {
      setPodcastsCount(res.data.count);
      const newPodcasts = res.data.favorites.map((favorite) =>
{
        return favorite.podcast;
      });
      setPodcasts([...newPodcasts]);
    });
  setPage(page + 1);
};

useEffect(() => {
  if (userState.username === router.query.username) {
    setUserData({
      username: userState.username,

```

```

        avatar: userState.avatar,
        id: userState.id,
    });
} else {
    if (router.query.username) {
        fetchUserData();
    }
}
}, [userState.username, router.query.username]);

useEffect(() => {
    if (userData.id) {
        fetchFavorites();
    }
}, [userData]);
return (
    <Layout>
        <DocHead title={router.query.username} />
        <Block className="flex flex-col gap-3 mt-5">
            <div className="flex gap-5">
                <Image
                    src={userData.avatar}
                    alt="avatar"
                    width={100}
                    height={100}
                    className="rounded-2xl"
                />
                <h1 className="text-2xl font-
bold">{userData.username}</h1>
            </div>
        </Block>

        {books.length !== 0 && (
            <Block className="mt-4">
                <TypographyTitle tag="h3" className="mb-3">
                    Избранные аудиокниги
                </TypographyTitle>
                <BooksList books={books} count={booksCount} />
            </Block>
        )}

        {podcasts.length !== 0 && (
            <Block className="mt-4">
                <TypographyTitle tag="h3" className="mb-3">
                    Избранные подкасты
                </TypographyTitle>

```

```

        <PodcastsList podcasts={podcasts}
count={podcastsCount} />
      </Block>
    )}
  </Layout>
);
};
export default UserPage;

```

Листинг B.52 – client/src/redux/store.ts

```

import { configureStore } from "@reduxjs/toolkit";
import userReducer from "../slices/userSlice";
import playerReducer from "../slices/playerSlice";

export const store = configureStore({
  reducer: {
    user: userReducer,
    player: playerReducer,
  },
});
export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;

```

Листинг B.53 – client/src/redux/slices/playerSlice.ts

```

import { compose, createSlice } from "@reduxjs/toolkit";

interface IPlayerInitialState {
  duration: string | null | number;
  name: string | null;
  author: string | null;
  cover: string | null;
  isPlaying: boolean;
  volume: number;
  currentTime: number;
  playQueue: any[] | null;
  currentQueuePosition: number | null;
}

const playerInitialState: IPlayerInitialState = {
  name: null,
  duration: 0,
  author: null,
  cover: null,
  isPlaying: false,
  volume: 100,

```

```

    currentTime: 0,
    playQueue: null,
    currentQueuePosition: 0,
  };

export const playerSlice = createSlice({
  name: "player",
  initialState: playerInitialState,
  reducers: {
    setMainInfo: (state, action) => {
      return { ...state, ...action.payload };
    },
    playAudio: (state) => {
      state.isPlaying = true;
    },
    stopAudio: (state) => {
      state.isPlaying = false;
    },
    setDuration: (state, action) => {
      state.duration = action.payload;
    },
    setVolume: (state, action) => {
      state.volume = action.payload;
    },
    setCurrentTime: (state, action) => {
      state.currentTime = action.payload;
    },
    setCurrentQueuePosition: (state, action) => {
      state.currentQueuePosition = action.payload;
    },
    setQueue: (state, action) => {
      state.playQueue = action.payload;
    },
  },
});

export const {
  playAudio,
  stopAudio,
  setCurrentTime,
  setDuration,
  setVolume,
  setQueue,
  setCurrentQueuePosition,
  setMainInfo,
} = playerSlice.actions;

```

```
export default playerSlice.reducer;
```

Листинг В.54 – client/src/redux/slices/userSlice.ts

```
import { query } from "@helpers/axios";
import { createAsyncThunk, createSlice } from
"@reduxjs/toolkit";

interface IUserInitialState {
  id: number | null;
  username: string | null;
  avatar: string | null;
  isFetching: boolean;
  authChecked: boolean;
}

export const userInitialState: IUserInitialState = {
  id: null,
  username: null,
  avatar: null,
  isFetching: true,
  authChecked: false,
};

export const getUserInfo = createAsyncThunk(
  "user/get",
  async function (_, { rejectWithValue }) {
    const response = await query.get("/auth/amiauthorized");
    if (response.status === 200) {
      return response.data;
    } else {
      return rejectWithValue(false);
    }
  }
);

export const userLogout = createAsyncThunk(
  "user/logout",
  async function name(_, { rejectWithValue }) {
    const response = await query.delete("/auth/logout");
    if (response.status === 200) {
      return response.data;
    } else {
      return rejectWithValue(false);
    }
  }
);
```



```

export const userSlice = createSlice({
  name: "user",
  initialState: userInitialState,
  reducers: {},
  extraReducers(builder) {
    builder
      .addCase(getUserInfo.pending, (state) => {
        state.isFetching = true;
      })
      .addCase(getUserInfo.fulfilled, (state, action) => {
        state.id = action.payload.id;
        state.username = action.payload.username;
        state.isFetching = false;
        state.avatar =
          action.payload.avatar ??
          "https://www.kindpng.com/picc/m/22-223863_no-avatar-
png-circle-transparent-png.png";
        state.authChecked = true;
      })
      .addCase(getUserInfo.rejected, (state) => {
        state.isFetching = false;
      })
      .addCase(userLogout.fulfilled, (state) => {
        (state.username = null), (state.avatar = null);
      });
  },
});
export default userSlice.reducer;

```